ALMI · CHAIN

ALMI-CHAIN · ASIR 1º · 2026-2027

Documentación Reto 1

Infraestructura informática de la cadena de restaurantes Almi-Chain: equipos, red, web en la nube, ERP con TPV, base de datos y copias de seguridad. Cada parte explicada paso a paso, como un manual, para que un técnico nuevo pueda montarla desde cero, y con la ruta de GitHub donde está cada cosa.

Equipo	Jon · Sendoa · Yeray · Iñigo · Alejandro · Jose
Centro	Centro de Estudios ALMI · Bilbao
Ciclo	CFGS Administración de Sistemas Informáticos en Red · 1.º
Entrega	13 de noviembre de 2026
Repositorio	github.com/JRevii/almi-chain

Índice

1. Introducción y arquitectura 2

2. Red: diseño en Packet Tracer, línea WAN y Wi-Fi 4

- Diseño de la red en Packet Tracer 5
- Necesidades de comunicación 5
- Línea WAN: comparativa y contratación 13
- Router y repetidor Wi-Fi del aula 15

3. Equipos: hardware, software y presupuesto 20

- Fichas de hardware y software y presupuesto 20

4. Equipo del gerente: arranque dual 26

- Equipo del gerente · Dual boot 26

5. TPV de barra e impresora de tickets 39

- TPV de barra e impresora de tickets 39
- TPV paso a paso: Windows 10 Pro en modo quiosco 44

6. Servidor, ERP y TPV (Odoo) 47

- Servidor Odoo (ERP + TPV) 48

7. Base de datos de los locales 56

- Base de datos de los locales (carpeta bd/) 56

8. Disco redundante y copias de seguridad 61

- Disco redundante y copias de seguridad 61
- Bucket S3 para la copia remota (Terraform) 68

9. Página web y hosting en AWS 69

- Bocetos de la web (diseño conceptual) 70
- Página web de Almi-Chain 72
- Código de la web 78
- Despliegue de la web en AWS con Terraform 79
- Infraestructura en AWS (Terraform) 84

10. Máquina recreativa 86

- Máquina recreativa (MAME) 87
- Pruebas de rendimiento · Recreativa MAME 92

11. Arcade stick con Arduino 93

- Arcade stick con Arduino 93

12. Digitalización 102

- Digitalización del restaurante 102

13. Organización y seguimiento del equipo 106

- Gestión del proyecto 107

14. Revisión de seguridad y pruebas 110

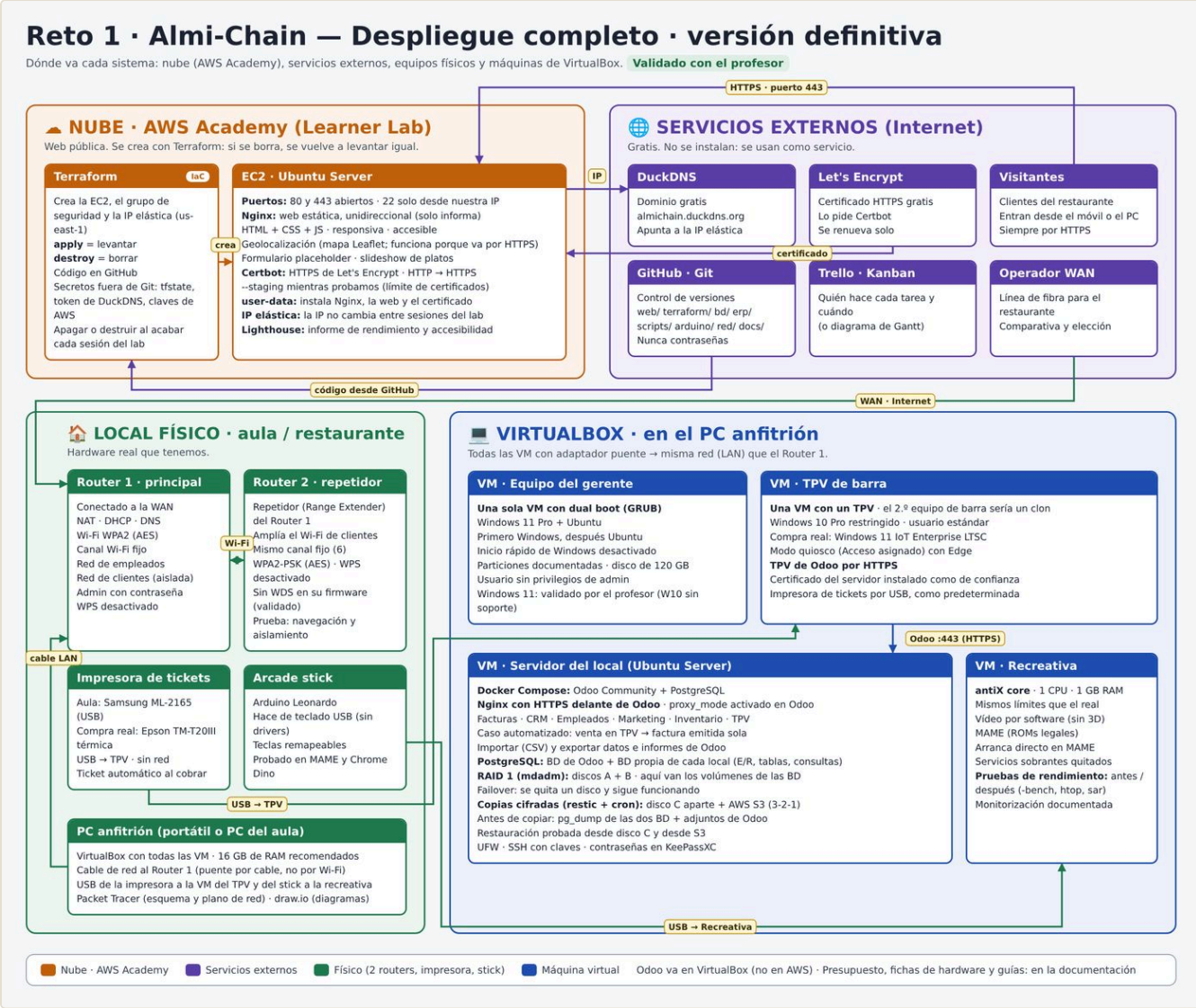
15. Conclusiones 112

16. Anexos 112

1. Introducción y arquitectura

Responsable: todos

Una cadena de restaurantes nos encarga su informática: equipos del gerente y de barra, máquina recreativa, red con Wi-Fi para clientes, web en la nube, ERP con TPV, base de datos de los locales y plan de copias de seguridad.



Arquitectura de la solución · GitHub: docs/diagramas/esquema_definitivo.png

El esquema reparte la solución en cuatro zonas. **En la nube (AWS Academy)** está la web pública, en un servidor EC2 que se crea con Terraform y se sirve por HTTPS con el dominio de DuckDNS. **En el local** están los dos routers (red de empleados y red de clientes aislada, con un repetidor para el Wi-Fi), la impresora de tickets y el arcade stick. **En VirtualBox** están el equipo del gerente (Windows 11 + Ubuntu), el TPV de barra en modo quiosco, la recreativa y el servidor del local con Odoo y PostgreSQL, RAID 1 y copias cifradas. El TPV y el gerente entran a Odoo por HTTPS dentro del local, las copias salen a un bucket S3 en AWS y los clientes llegan a la web desde internet. Los **servicios externos** (GitHub, Trello, DuckDNS, Let's Encrypt y el operador de fibra) se usan sin instalar nada.

Cómo está organizado el repositorio

Todo el trabajo está en GitHub: github.com/JRevii/almi-chain. Cada carpeta tiene su guía (`README.md`) con los pasos para montarla desde cero, sus pruebas y sus capturas.

Carpeta	Contenido	Tarea del reto
`docs/`	Documentación general y diagramas de arquitectura	Documentación reproducible
`gestion/`	Anexos (I, II, IX) y Gantt/Kanban	Seguimiento del trabajo
`sistemas/gerente-dualboot/`	Equipo del gerente: Windows 11 Pro + Ubuntu (dual boot)	Implantación de SO
`sistemas/tpv-barra/`	TPV de barra en Windows (una VM; el segundo TPV es un clon) + impresora de tickets, mínimos privilegios	Equipos, permisos, impresora
`recreativa/`	VM con MAME optimizada (1 núcleo, 1 GB RAM) y pruebas de rendimiento	Máquina recreativa
`arcade-stick/`	Código Arduino y montaje del arcade stick	Arcade stick
`redes/`	Línea WAN (`routers/wan/`), plano de red, Wi-Fi, repetidor (`routers/repetidor/`) y Packet Tracer	Conectividad
`web/`	Web estática responsiva y accesible	Página web
`infra/terraform/`	Despliegue en AWS (EC2 con IP elástica), Nginx + HTTPS con Let's Encrypt, DuckDNS	Hosting reproducible
`infra/terraform-copias/`	Bucket S3 cifrado para la copia remota de restic	Copias de seguridad (regla 3-2-1)
`erp/`	ERP + TPV y caso de uso automatizado (emisión de factura)	ERP
`bd/`	Modelo E/R, modelo relacional y scripts SQL	Base de datos
`backups/`	Disco redundante (RAID), plan de copias local y remoto (AWS S3) y pruebas de restauración	Copias de seguridad

Dónde está cada tarea del reto

Las 16 tareas de la hoja de seguimiento, con el documento que la explica y dónde están sus pruebas.

#	Tarea	Documento	Pruebas y evidencias
1	Identificar y documentar el hardware necesario	Fichas de hardware y software (apartados 1–7)	Presupuesto en Excel: 4.624,43 € + IVA = 5.595,56 €, con enlace de compra en cada fila
2	Identificar, documentar e instalar el software necesario	Fichas, apartado 8	La instalación de cada programa está en la guía de su equipo (columna «Instalación» de esa tabla)
3	Web en AWS	Guía de la web · Despliegue en AWS	Pruebas de la web · pruebas de AWS
4	ERP + TPV	ERP · TPV	Pruebas del ERP
5	Arcade stick con Arduino	Arcade stick	Pruebas
6	Configurar una impresora	TPV, impresora	Pruebas 5–8 del TPV

#	Tarea	Documento	Pruebas y evidencias
7	Instalación de SO con arranque dual	Dual boot	Resultado final
8	Elección de la conectividad WAN	Línea WAN	Comparativa (PDF)
9	Diseño y esquema de conectividad	Redes	Packet Tracer · pruebas
10	Ampliación de la cobertura Wi-Fi	Repetidor	Pruebas
11	Restricción de cuentas y equipos	Gerente · TPV · recreativa	Mínimo privilegio en cada parte
12	Optimizar y monitorizar la recreativa	Recreativa	Medidas
13	Plan de copias de seguridad	Copias	Pruebas 13–19
14	Disco redundante	Copias, RAID 1	Pruebas 11–12
15	Diseño del modelo E/R y tablas	Base de datos	Diagrama E/R · modelo relacional · requisitos
16	Implementación de la base de datos	Base de datos, orden de ejecución	Scripts SQL · modelo físico · capturas de las 11 consultas

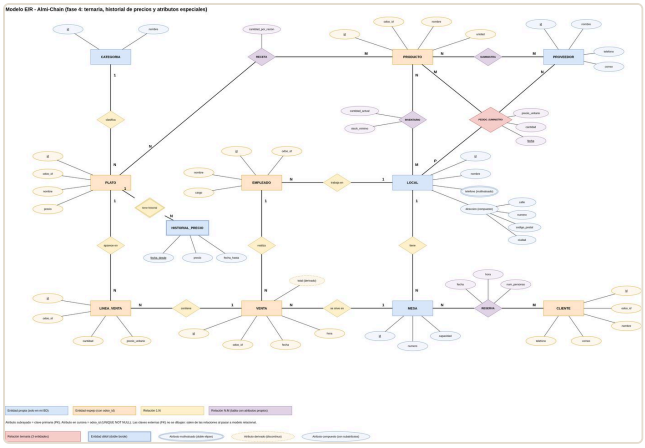
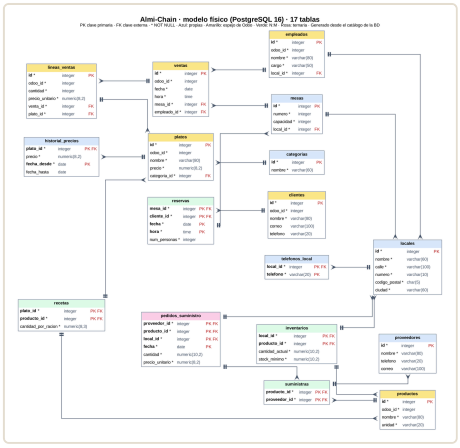


Diagrama E/R · GitHub: bd/er-conceptual.png



modelo físico · GitHub: bd/er-fisico.png

Resumen de todas las pruebas: documento final, apartado 14. Validaciones del profesor: `gestion/`.

Arquitectura: `docs/diagramas/esquema_definitivo.png` · **Planificación y anexos:** `gestion/`

Cada carpeta tiene su propio `README.md` con los pasos para reproducir esa parte desde cero.

2. Red: diseño en Packet Tracer, línea WAN y Wi-Fi

Responsable: Jon (Packet Tracer), Iñigo (necesidades), Yeray (WAN), Iñigo y Yeray (routers)

Dónde está en GitHub

Diseño en Packet Tracer y pruebas	`redes/README.md`
Proyecto de Packet Tracer validado	`redes/packet-tracer/almichain.pkt`
Topología del proyecto	`redes/packet-tracer/topologia_validada.png`
Configuración de los routers y el switch	`redes/packet-tracer/config`
Capturas de Packet Tracer	`redes/packet-tracer/capturas`
Necesidades de comunicación	`docs/necesidades/necesidades-comunicacion.md`
Línea WAN	`redes/routers/wan/README.md`
Router y repetidor del aula	`redes/routers/router-y-repetidor.md`
Capturas del router y del repetidor	`redes/routers`

Resumen

Diseño de la red en Packet Tracer (Jon). El proyecto (`almichain.pkt`) tiene todas las IP configuradas en cada equipo y cinco zonas: el operador WAN, internet (DuckDNS, Let's Encrypt, GitHub, Trello y un visitante, en `203.0.113.0/24`), la EC2 de AWS, el local y las máquinas de VirtualBox. El restaurante tiene un router propio, **R1-AlmiChain** (ISR4331), detrás de la fibra, con **dos redes separadas**: la de **empleados** (`10.13.10.0/24`, por cable: servidor de Odoo, TPV, gerente, recreativa y PC anfitrión, unidos por el switch `Puente-VirtualBox`) y la Wi-Fi de **clientes** (`10.13.20.0/24`), con dos routers Wi-Fi como puntos de acceso (`AlmiChain-Clientes`, **WPA2-PSK con AES**, canal 6). R1 hace NAT, reparte IP por DHCP y tiene la **ACL** `CLIENTES-AISLADOS`: los clientes salen a internet pero no entran en la red de empleados ni pueden intentar el SSH. R1 solo se administra por **SSH v2** y solo desde la red de empleados. El switch tiene *port-security* en cada puerto, los puertos libres apagados y no se administra en remoto. La configuración de cada equipo está en `redes/packet-tracer/config/`. En la prueba de aislamiento, el móvil de clientes no llega al servidor (100 % de pérdida). La profesora validó el proyecto.

Necesidades de comunicación (Iñigo). El local tiene dos redes. En la de **empleados** (`10.13.10.0/24`, por cable y con IP fijas) están el servidor, los dos TPV, el PC del gerente y la recreativa. La Wi-Fi de **clientes** (`10.13.20.0/24`, por DHCP) va aparte, con el router principal y el repetidor. La impresora de tickets va por USB al TPV, sin red, y la web está en AWS, fuera del local. Para cada equipo se indica con quién habla, por qué puerto y si va cifrado: los TPV y el gerente usan Odoo por HTTPS (443), el técnico administra por SSH solo con clave, y los clientes solo salen a internet. También se define el tráfico que **no** debe existir: de la Wi-Fi de clientes a la red de empleados, PostgreSQL y Odoo accesibles desde fuera del servidor, y SSH desde la red de clientes. El riesgo principal es que la recreativa, de uso público, comparte red con los TPV, y se mitiga dejándola sin ningún servicio escuchando.

Línea WAN (Yeray). Se compararon Movistar, Vodafone y MásMóvil para empresas, y se eligió **Vodafone con fibra de 1 Gb simétrica**. El primer año cuesta casi lo mismo que MásMóvil, y a partir del mes 13 es la más barata (35 €/mes con IVA), con la instalación sin coste y un router Wi-Fi 6 incluido. Que sea simétrica importa porque cada noche se suben las copias de seguridad a AWS. La permanencia es de 12 meses. No se contrata IP fija porque ningún servicio del local se ofrece a internet: la web está en AWS.

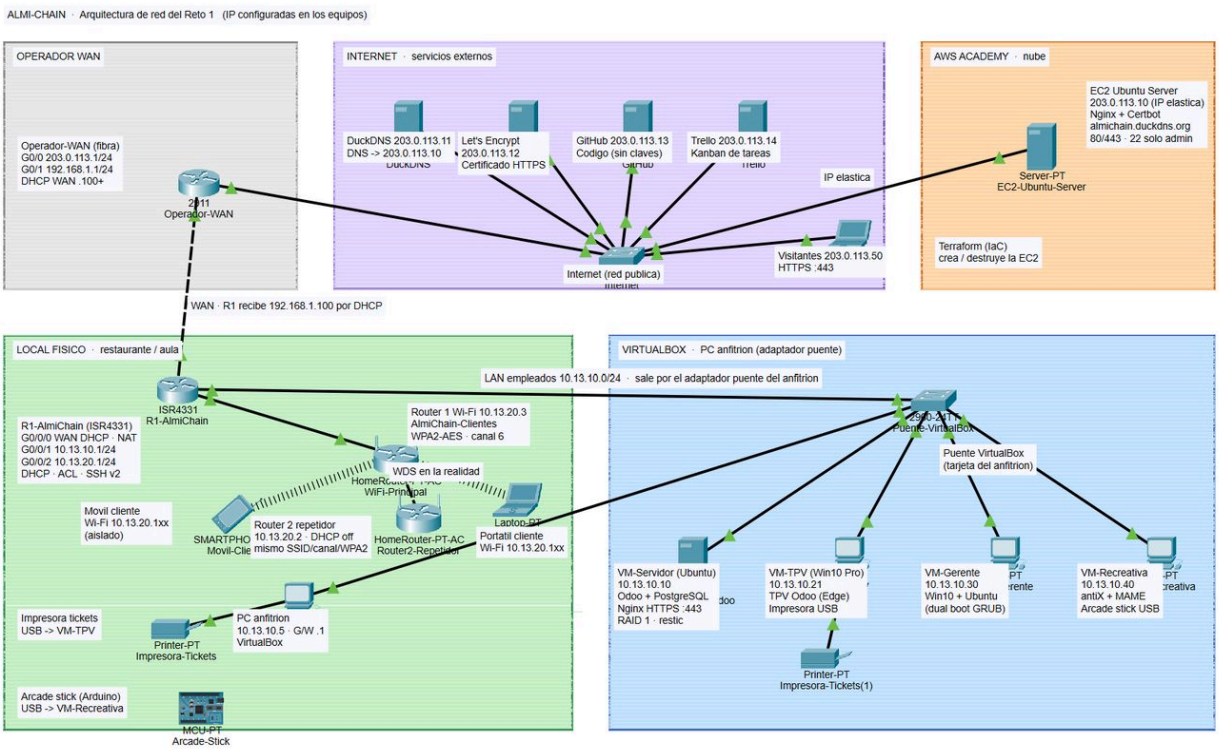
Routers del aula (Iñigo y Yeray). Se montaron con dos TP-Link TL-WR841N, reconfigurados desde cero. El router principal da la Wi-Fi `AlmiChain-Clientes` (canal 6, **WPA2-PSK con AES**) y reparte IP de `192.168.0.100` a `.199`. Tiene la contraseña de administración cambiada, el **WPS** desactivado y la gestión remota cerrada. El enunciado pedía enlazar el segundo router por **WDS**, pero este modelo no tiene esa opción, y el profesor indicó usarlo como **repetidor** (*Range Extender*): amplía la misma red, elegida por el BSSID del router para no confundirla con las de otros grupos. Para **aislar a los clientes**, una regla de *Access Control* bloquea todo el tráfico de la Wi-Fi hacia la red del aula. En la prueba, internet responde y el servidor de Odoo no (100 % de pérdida, y HTTPS sin conexión). La profesora lo validó el 12/11.

Diseño de la red en Packet Tracer

> **En GitHub:** `redes/README.md`

Contratación de la línea WAN, esquema de conectividad en Packet Tracer, plano del local, Wi-Fi segura y ampliación de la Wi-Fi de clientes con un segundo router.

Archivo	Qué es
`packet-tracer/almichain.pkt`	Proyecto de Packet Tracer (vista lógica + vista física) con las IP configuradas en cada equipo. **Es la versión que ha validado la profesora**; las anteriores quedan en el historial de Git
`packet-tracer/topologia_validada.png`	Captura de la vista lógica del proyecto validado, con las IP de cada equipo
`packet-tracer/plano_local.png`	Plano del local: ubicación de equipos, cableado y cobertura Wi-Fi. Es la imagen de fondo de la vista física
`packet-tracer/config/`	Configuración de los dos routers Cisco y del switch, sacada del proyecto validado y lista para pegar en la CLI
`packet-tracer/capturas/`	Capturas de las pruebas en el simulador
`routers/wan/`	Comparativa y elección de la línea WAN (Yeray)
`routers/router-y-repetidor.md`	Routers físicos del aula: red de clientes y repetidor Wi-Fi (Iñigo y Yeray)
`../docs/Comparativa y contratación de la línea WAN.pdf`	Comparativa de ofertas de fibra y elección de la línea WAN (Yeray)



Vista lógica del proyecto validado en Packet Tracer · GitHub: redes/packet-tracer/topologia_validada.png

1. Qué representa el proyecto

El proyecto tiene cinco zonas, las mismas que el esquema general del reto:

- **Operador WAN:** el router de la fibra (`Operador-WAN`). Da a R1 su IP de WAN por DHCP y le da salida a internet.

- **Internet:** un switch (`Internet`) con los servicios externos que usa el reto: DuckDNS (DNS), Let's Encrypt (certificado), GitHub, Trello y un visitante de la web.
- **AWS Academy:** el servidor EC2 con la web pública (`203.0.113.10`, que hace el papel de la IP elástica).
- **Local físico:** el router propio del restaurante (`R1-AlmiChain`), los dos routers Wi-Fi de clientes, los clientes Wi-Fi, el PC anfitrión, la impresora de tickets y el arcade stick.
- **VirtualBox:** las cuatro máquinas virtuales del reto, unidas por el adaptador puente del PC anfitrión, que en el simulador es el switch `Puente-VirtualBox`.

Como las IP públicas reales cambian, internet se simula con el rango `203.0.113.0/24`, reservado para documentación.

2. Direccionamiento

Red	Rango	Puerta de enlace	Uso
Internet (simulado)	203.0.113.0/24	203.0.113.1	Servicios externos y web en AWS
WAN	192.168.1.0/24	192.168.1.1	Enlace entre el operador y R1. R1 recibe `192.168.1.100` por DHCP
Empleados	10.13.10.0/24	10.13.10.1	Servidor de Odoo, TPV, gerente, recreativa y PC anfitrión. Por cable
Clientes	10.13.20.0/24	10.13.20.1	Wi-Fi `AlmiChain-Clientes`, **aislada** de la red de empleados

En las dos redes del local, las IP de la `.1` a la `.99` y de la `.200` a la `.254` son fijas, y el DHCP de R1 reparte de la `.100` a la `.199`. El DNS que reparte es el de DuckDNS (`203.0.113.11`).

2.1 Equipos, IP y cables

Equipo	Modelo en Packet Tracer	IP	Conectado a
Operador-WAN	Router Cisco 2911	Gi0/0 `203.0.113.1` · Gi0/1 `192.168.1.1`	Gi0/0 → Internet Gi0/1 · Gi0/1 → R1 Gi0/0/0 (cruzado)
Internet	Switch 2960	—	Servidores externos en Fa0/1 a Fa0/6
EC2-Ubuntu-Server	Server	`203.0.113.10`	Internet Fa0/1
DuckDNS	Server	`203.0.113.11`	Internet Fa0/2
LetsEncrypt	Server	`203.0.113.12`	Internet Fa0/3
Visitante-Internet	Laptop	`203.0.113.50`	Internet Fa0/4
GitHub	Server	`203.0.113.13`	Internet Fa0/5
Trello	Server	`203.0.113.14`	Internet Fa0/6
R1-AlmiChain	Router Cisco ISR4331	Gi0/0/0 por DHCP · Gi0/0/1 `10.13.10.1` · Gi0/0/2 `10.13.20.1`	Gi0/0/1 → Puente-VirtualBox Gi0/1 · Gi0/0/2 → WiFi-Principal (puerto LAN 1)
Puente-VirtualBox	Switch 2960	—	VM en Fa0/1 a Fa0/4 y PC anfitrión en Fa0/5
VM-Servidor-Odoo	Server	`10.13.10.10`	Puente Fa0/1
VM-TPV	PC	`10.13.10.21`	Puente Fa0/2
VM-Gerente	PC	`10.13.10.30`	Puente Fa0/3

Equipo	Modelo en Packet Tracer	IP	Conectado a
VM-Recreativa	PC	`10.13.10.40`	Puente Fa0/4
PC-Anfitrion	PC	`10.13.10.5`	Puente Fa0/5
WiFi-Principal	HomeRouter-PT-AC	LAN `10.13.20.3`	R1 Gi0/0/2 (LAN 1) · Router2-Repetidor (LAN 3, cruzado)
Router2-Repetidor	HomeRouter-PT-AC	LAN `10.13.20.2`	WiFi-Principal (LAN 1)
Movil-Cliente	Smartphone	DHCP (`10.13.20.100`)	Wi-Fi
Portatil-Cliente	Laptop	DHCP (`10.13.20.101`)	Wi-Fi
Impresora-Tickets e Impresora-Tickets(1)	Printer	—	Cable serie al PC anfitrión y a la VM-TPV (en la realidad, USB)
Arcade-Stick	MCU-PT	—	Nota: va por USB a la VM-Recreativa

Los equipos de la red de empleados llevan máscara `255.255.255.0` y puerta de enlace `10.13.10.1`.

3. Configuración de cada equipo

3.1 R1-AlmiChain (router del restaurante)

Archivo: `packet-tracer/config/R1-AlmiChain.txt`

- **Tres interfaces:** WAN por DHCP (`ip nat outside`), LAN de empleados y LAN de clientes (las dos `ip nat inside`).
- **DHCP:** pools `EMPLEADOS` y `CLIENTES`, con las IP fijas excluidas.
- **NAT con sobrecarga (PAT):** las dos LAN salen a internet con la IP de la WAN (ACL `NAT-LAN`). Desde fuera no se ve ninguna IP interna.
- **ACL `CLIENTES-AISLADOS`**, de entrada en Gi0/0/2:
  1. Deja pasar el DHCP (`bootps`), para que los clientes reciban IP.
  2. Bloquea todo lo que va de `10.13.20.0/24` a `10.13.10.0/24`.
  3. Bloquea el puerto 22: los clientes no pueden intentar administrar el router.
  4. Deja pasar el resto, es decir, internet.
- **Administración:** solo SSH v2, con usuario local y solo desde la red de empleados (ACL `SSH-ADMIN` en las líneas vty). Consola y vty se cierran tras 5 minutos sin uso.
- **Contraseñas:** `enable secret`, `service password-encryption` y longitud mínima de 10 caracteres (`security passwords min-length 10`). Mensaje de acceso restringido.

3.2 Operador-WAN

Archivo: `packet-tracer/config/Operador-WAN.txt`. Es la puerta de enlace de internet (`203.0.113.1`) y da por DHCP la IP de WAN a R1 (pool `WAN`, desde la `.100`).

3.3 Puente-VirtualBox (switch de empleados)

Archivo: `packet-tracer/config/Puente-VirtualBox.txt`

- **Port-security *sticky*** en Fa0/1 a Fa0/5: cada puerto aprende la MAC de su equipo. Si se conecta otro, su tráfico se descarta y queda registrado (`violation restrict`).
- **PortFast y BPDU Guard** en esos puertos: el equipo tiene red al momento y, si alguien conecta otro switch, el puerto se bloquea.
- **Puertos libres apagados** (Fa0/6 a Fa0/24 y Gi0/2), con la descripción «Sin uso».

- **Sin administración remota:** `transport input none` en las vty. Solo se entra por consola, con usuario local y cierre a los 5 minutos.

3.4 Routers Wi-Fi (WiFi-Principal y Router2-Repetidor)

Se configuran en la pestaña **GUI** de cada HomeRouter:

Ajuste	WiFi-Principal	Router2-Repetidor
IP de LAN	`10.13.20.3`	`10.13.20.2`
Conexión	Puerto LAN 1 a R1 Gi0/0/2	Puerto LAN 1 al LAN 3 del principal
SSID	`AlmiChain-Clientes`	`AlmiChain-Clientes`
Seguridad	WPA2-PSK, cifrado AES	WPA2-PSK, cifrado AES, misma clave
Canal (2,4 GHz)	6	6
DHCP	Lo da R1 (`10.13.20.1`)	Lo da R1 (`10.13.20.1`)

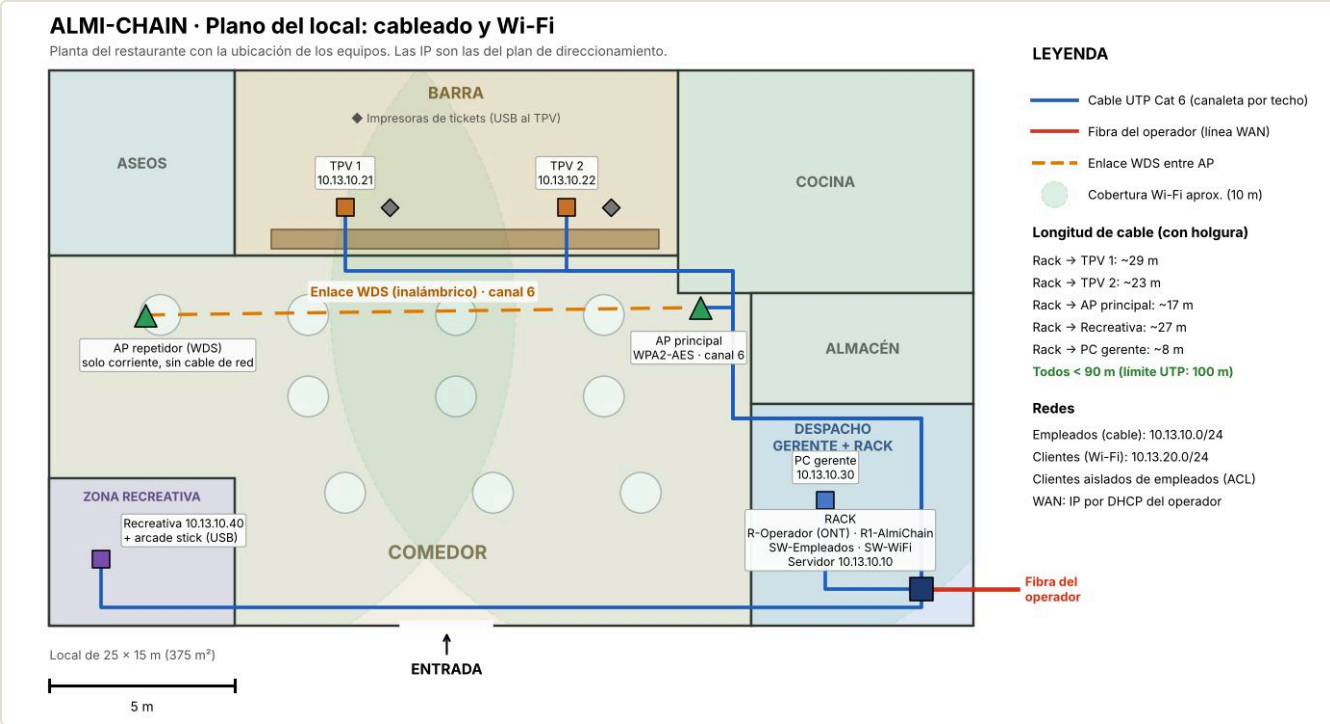
Los dos routers van conectados por sus puertos LAN, así que funcionan como **puntos de acceso**: los clientes reciben la IP del DHCP de R1 y pasan por la ACL de aislamiento. Con el mismo SSID, la misma clave y el mismo canal, un móvil pasa de uno a otro sin reconectarse.

WDS y repetidor. Packet Tracer no tiene WDS ni modo repetidor entre estos routers, así que el enlace entre los dos se simula con un cable (nota «WDS en la realidad» en el proyecto). En el aula, el segundo router (TP-Link TL-WR841N v14) tampoco tiene WDS en su firmware, y el profesor indicó configurarlo como **repetidor** (*Range Extender*). La configuración real, las capturas y las pruebas están en `routers/router-y-repetidor.md`.

4. Montaje en Packet Tracer, paso a paso

1. Coloca los equipos de la tabla 2.1 y únelos según la columna «Conectado a». Usa cable directo, salvo Operador-WAN ↔ R1 y WiFi-Principal ↔ Router2-Repetidor, que van con cable cruzado.
2. En **Operador-WAN**, **R1-AlmiChain** y **Puente-VirtualBox**: pestaña **CLI**, pulsa Enter, escribe `enable` y `configure terminal`, y pega su archivo de `packet-tracer/config/`. Antes, cambia `CLAVE_ENABLE` y `CLAVE_ADMIN` por tus claves (en R1, de 10 caracteres o más).
3. Configura los dos HomeRouter en la pestaña **GUI** con la tabla 3.4.
4. En el móvil y el portátil: *Config* → *Wireless0*, el SSID `AlmiChain-Clientes`, WPA2-PSK y la misma clave. En IP, marca **DHCP**.
5. En los equipos de empleados y de internet: *Config* → *FastEthernet0*, la IP fija de la tabla y su puerta de enlace (`10.13.10.1` o `203.0.113.1`).
6. En el servidor EC2, activa el servicio **HTTP** para poder abrir la web desde el navegador de los clientes.
7. Guarda el proyecto como `packet-tracer/almichain.pkt`.

5. Vista física (plano)



Plano del local · GitHub: redes/packet-tracer/plano_local.png

1. Pulsa **Physical** en la barra superior.
2. Entra en el edificio del local y ponle de fondo `plano_local.png`, con el botón de imagen de fondo.
3. Arrastra cada equipo a su sitio según el plano: el rack en el despacho, los TPV en la barra, la recreativa en su zona y los dos routers Wi-Fi en el comedor.
4. Comprueba en la vista lógica que el móvil sigue conectado a la Wi-Fi.

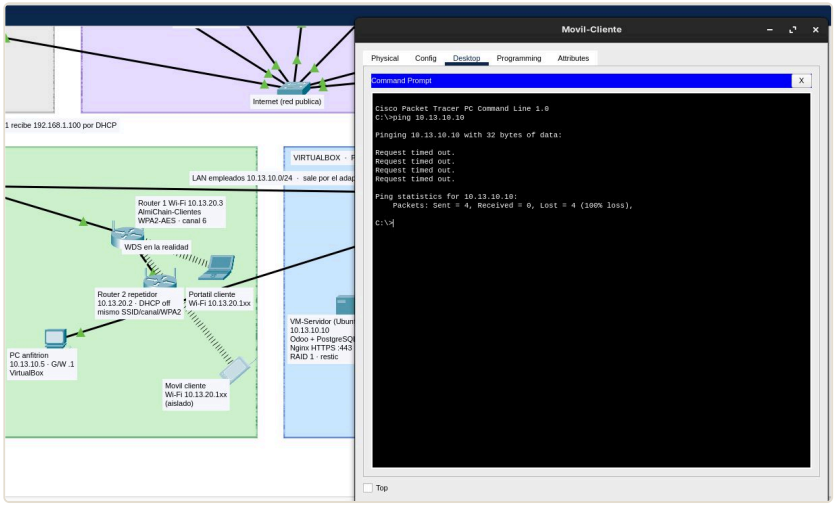
6. Seguridad aplicada

Medida	Dónde
La Wi-Fi de clientes no llega a la red de empleados; solo sale a internet	ACL `CLIENTES-AISLADOS` en R1 Gi0/0/2
Wi-Fi con WPA2-PSK y cifrado AES, igual en los dos routers	WiFi-Principal y Router2-Repetidor
El router solo se administra por **SSH v2**, con usuario y contraseña, y solo desde la red de empleados	R1: `transport input ssh` y `access-class SSH-ADMIN in`
Los clientes ni siquiera pueden intentar el SSH	R1: `deny tcp any any eq 22` en la ACL de clientes
Contraseñas cifradas, `enable secret`, mínimo 10 caracteres y mensaje de acceso restringido	R1 y Puente-VirtualBox
El switch no se administra en remoto, solo por consola	Puente-VirtualBox: `transport input none`
Cada puerto solo acepta su equipo; puertos libres apagados; BPDU Guard	Puente-VirtualBox
NAT: desde internet no se ve ninguna IP interna	R1
Sesiones que se cierran solas tras 5 minutos sin uso	`exec-timeout 5 0`

7. Pruebas

La profesora pide el esquema en Packet Tracer y ya lo ha validado (prueba 9). Las pruebas 1 a 7 sirven para enseñar en la defensa que la configuración funciona en el simulador.

#	Prueba	Resultado esperado	Estado
1	`ipconfig` en el móvil	IP `10.13.20.1xx` dada por R1	Opcional
2	Desde el móvil, `ping 203.0.113.10` y abrir `http://203.0.113.10`	Responde y carga la web: hay salida a internet	Opcional
3	Desde el móvil, `ping 10.13.10.10`	**Falla**: la Wi-Fi de clientes está aislada	✅ 05/11 · 4 enviados, 0 recibidos (`100% loss`) · captura
4	Desde la VM-TPV, `ping 10.13.10.10` y `ping 203.0.113.10`	Responden los dos	Opcional
5	Desde el PC anfitrión, `ssh -l admin 10.13.10.1`	Pide la contraseña y entra	Opcional
6	Desde el portátil del cliente, `ssh -l admin 10.13.20.1`	**Rechazado**	Opcional
7	En R1: `show ip nat translations` , `show ip dhcp binding` y `show access-lists`	Traducciones, IP repartidas y bloqueos de la ACL	Opcional
8	Routers físicos del aula: un cliente de la Wi-Fi navega y no llega a la red del aula	Navega y el ping al servidor (`192.168.1.50`) falla	✅ 12/11 · internet responde (0 % de pérdida) y el servidor no (100 %, y HTTPS sin conexión). Validado por la profesora · `router-y-repetidor.md`
9	**Revisión de la profesora:** proyecto de Packet Tracer, esquema, plano y configuración	Le parece correcto	✅ Validado por la profesora (primera revisión 26/10; versión final con las IP configuradas, 12/11)



Desde el móvil, ping 10.13.10.10 · GitHub: redes/packet-tracer/capturas/pt_03_movil_no_llega_al_servidor.png

Necesidades de comunicación

> **En GitHub:** `docs/necesidades/necesidades-comunicacion.md`

1. Inventario de equipos y servicios

Elemento	Quién lo usa	Dónde está	Cómo se conecta
PC del gerente (dual boot)	Gerente	Despacho	Cable, red de empleados (10.13.10.30)
TPV de barra 1	Camareros	Barra	Cable, red de empleados (10.13.10.21)
TPV de barra 2 (clon del 1)	Camareros	Barra	Cable, red de empleados (10.13.10.22)
Impresora de tickets	TPV de barra	Barra	USB al TPV (sin red)
Servidor (Odoo + PostgreSQL)	Gerente y TPV; administra el técnico	Despacho	Cable, red de empleados (10.13.10.10)
Máquina recreativa	Clientes	Sala	Red de empleados (10.13.10.40)
Router 1	Toda la red	Despacho	Fibra del operador
Router 2 (repetidor)	Clientes	Sala	Inalámbrico (repetidor; ver `router-y-repetidor.md`)
Wi-Fi de clientes	Clientes	Local	Wi-Fi, red de clientes aislada
Web corporativa	Público	Nube (AWS)	Internet

2. Redes y direccionamiento (plan)

Red	Rango	Puerta de enlace	Reparto
Empleados	10.13.10.0/24	10.13.10.1	Fijas .2-.99, DHCP .100-.199
Clientes (aislada)	10.13.20.0/24	10.13.20.1	DHCP .50-.250, Router 2 en .2

Equipo	IP	Red
Anfitrión	10.13.10.5	Empleados
Servidor	10.13.10.10	Empleados
TPV de barra 1 (2 = clon)	10.13.10.21 (.22)	Empleados
Gerente	10.13.10.30	Empleados
Recreativa	10.13.10.40	Empleados
Router 2 (gestión)	10.13.20.2	Clientes

Riesgo: la recreativa es de uso público y comparte red con TPV y servidor. Mitigación: servicios sobrantes eliminados, nada escuchando.

3. Tráfico por equipo

Origen	Destino	Protocolo y puerto	Para qué	¿Cifrado?
TPV de barra	Servidor 10.13.10.10	HTTPS 443	Abrir el TPV de Odoo	Sí
Gerente	Servidor 10.13.10.10	HTTPS 443	Usar Odoo	Sí

Origen	Destino	Protocolo y puerto	Para qué	¿Cifrado?
Técnico	Servidor	SSH 22 (solo claves)	Administrar	Sí
Servidor	Internet	HTTP/HTTPS 80/443	Actualizaciones	Sí
Equipos de empleados	Router 1	DNS 53	Resolver nombres	No
Móviles de clientes	Router 1	DHCP 67/68, DNS 53	Obtener IP y nombres	No
Clientes	Internet	80/443	Navegar	Depende
Público	Web en AWS	80 (redirige) y 443	Ver la web	Sí
Técnico (nuestra IP)	EC2 en AWS	SSH 22	Administrar la web	Sí
Let's Encrypt	EC2 en AWS	HTTP 80	Validar el dominio	No
Recreativa	Router 1	DNS 53	Solo nombres si hiciera falta	No

Tráfico que NO debe existir

- Red de clientes (10.13.20.0/24) hacia la de empleados (10.13.10.0/24).
- PostgreSQL (5432) y Odoo (8069) accesibles desde fuera del servidor.
- SSH al servidor desde la red de clientes.
- Acceso desde Internet al servidor local (sin redirección de puertos en el Router 1).

Línea WAN: comparativa y contratación

> **En GitHub:** `redes/routers/wan/README.md`

Responsable: Yeray. Comparativa de ofertas de fibra para empresa y elección de la línea del local. El documento completo está en `Comparativa y contratación de la línea WAN.pdf`, y el diseño de la red en `../../README.md`.

Elección

Vodafone Empresas, fibra 1 Gb simétrica, según la propuesta de Yeray al equipo (captura):

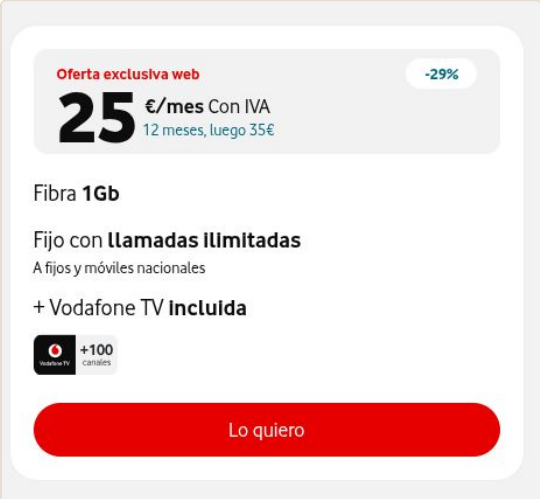
Concepto	Valor
Cuota	20,66 €/mes sin IVA los 12 primeros meses; después, 28,93 €/mes sin IVA
Instalación	Sin coste
Router	Wi-Fi 6 incluido
Permanencia	**12 meses**. Si se da de baja antes, penalización proporcional al tiempo que falte (Selectra y Comparaiso, consultado el 06/11/2026)
IP fija	Es un servicio aparte de pago. **No se contrata, porque no hace falta:** la web está en AWS, la copia remota sale del local hacia S3 y no se administra nada desde fuera. Ningún servicio del local se ofrece a internet

Comparativa

Criterio	Movistar Empresas	Vodafone Empresas	MásMóvil Empresas
Precio año 1	29,90 €/mes	25 € con IVA (20,66 € sin IVA)	24,80 €/mes
Precio desde mes 13	46,20 €/mes	35 € con IVA	38 €/mes
Instalación	A confirmar	Sin coste, en 24-48 h	A confirmar
Router	A confirmar	Wi-Fi 6 incluido	Alquiler gratis
Permanencia	Sin permanencia	A confirmar (comparadores dicen 12 meses)	12 meses
IP fija	A confirmar	Servicio aparte (tiene página propia en la web)	A confirmar
SLA / soporte	A confirmar	A confirmar	A confirmar
Fuente	Comparador	Oficial	Comparador

Tabla comparativa · GitHub: redes/routers/wan/capturas/16_wan_tabla.png

Captura	Qué muestra
13_wan_movistar	Oferta de Movistar Empresas
14_wan_vodafone	Oferta de Vodafone Empresas
15_wan_masmovil	Oferta de MásMóvil Empresas
16_wan_tabla	Tabla comparativa de las tres
17_wan_mensaje_jose	Elección comunicada al equipo para el presupuesto



Oferta de Vodafone Empresas · GitHub: redes/routers/wan/capturas/14_wan_vodafone.png

Por qué Vodafone

Necesidad del restaurante	Cómo la cubre la línea
TPV y servidor de Odoo	Funcionan dentro del local, así que una caída de la fibra no deja sin cobrar. La línea solo se usa para salir a internet
Copia remota en AWS S3 cada noche	1 Gb **simétrico**: la subida es igual de rápida que la bajada, y subir las copias a la nube no satura la línea
Wi-Fi de clientes en hora punta	1 Gb de sobra para muchos móviles a la vez navegando

Necesidad del restaurante	Cómo la cubre la línea
Coste	El primer año, casi igual que MásMóvil (25 € frente a 24,80 €/mes). A partir del mes 13, **la más barata**: 35 €/mes con IVA (28,93 € sin IVA), frente a 38 € de MásMóvil y 46,20 € de Movistar. Además, la instalación no cuesta nada y el router Wi-Fi 6 va incluido

La permanencia de 12 meses no es un problema para un local que abre para quedarse. Movistar no tiene permanencia, pero cuesta bastante más cada mes.

Nunca se suben contraseñas ni la copia de configuración del router (`.bin`, `.cfg`): llevan la contraseña de administración y la clave del Wi-Fi.

Router y repetidor Wi-Fi del aula

> **En GitHub:** `redes/routers/router-y-repetidor.md`

Responsables: Iñigo y Yeray. Estado: ✅ **validado por la profesora de Redes el 12/11**.

1. Objetivo y diseño

Dar Wi-Fi a los clientes del local, aislada de los equipos de la empresa, y ampliar su cobertura con un segundo aparato.

Elemento	Valor
Red del aula (WAN del router)	192.168.1.0/24, IP por DHCP
Red de clientes (LAN del router)	192.168.0.0/24, puerta de enlace 192.168.0.1
DHCP del router	192.168.0.100 a 192.168.0.199
Wi-Fi	AlmiChain-Clientes, canal 6 fijo, WPA2-PSK con AES
Aislamiento	Regla de Access Control: clientes (192.168.0.100-199) hacia la red del aula (192.168.1.1-254)

Las claves y contraseñas están solo en KeePassXC. No se escriben aquí.

2. Aparatos

Los dos son TP-Link TL-WR841N v14 (firmware 0.9.1 4.16 Build 180319).

Aparato	MAC	Papel
Router	LAN C0:C9:E3:B3:9B:BA	NAT, DHCP y Wi-Fi de clientes
Repetidor	Etiqueta 50:D4:F7:3D:9A:6E	Amplía la Wi-Fi del router

Los papeles se intercambiaron respecto a la primera configuración: el aparato que antes hacía de router pasó a ser el repetidor.

3. Decisión: repetidor en lugar de WDS

El reto pedía un segundo router enlazado por WDS. El panel de este modelo no tiene WDS: ofrece los modos Wireless Router, WISP, Access Point y Range Extender, y el menú Wireless solo tiene Basic Settings, WPS, Wireless Security, MAC Filtering, Wireless Advanced y Wireless Statistics. Se consultó al profesor, que indicó que el aparato hiciera de repetidor.

4. Router

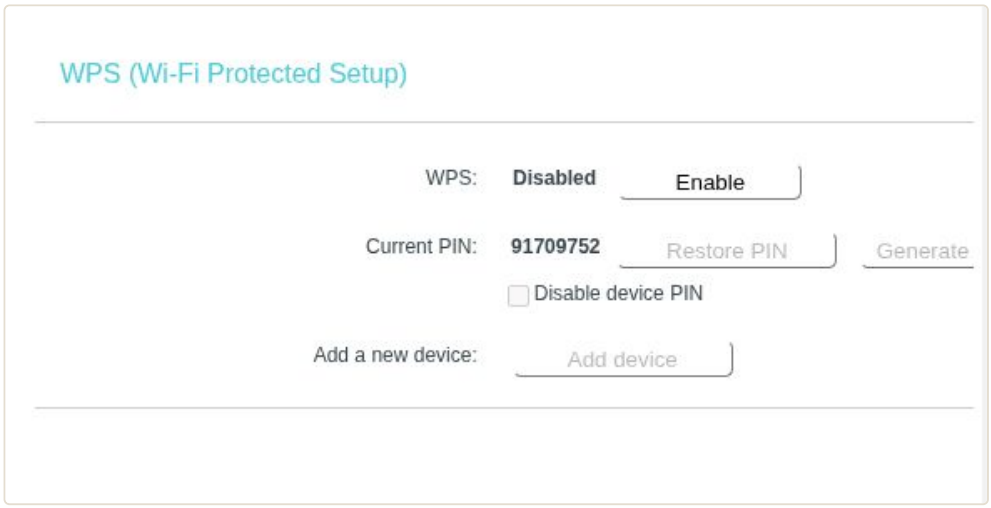
Todo se configura por cable.

1. Reset de fábrica (RESET 10 segundos con el router encendido). El cable del aula va al puerto WAN y el PC a un puerto LAN.
2. Contraseña de administración generada en KeePassXC.
3. Operation Mode: Wireless Router. Network, WAN: Dynamic IP. El router cogió una IP de la red del aula.
4. Wi-Fi: SSID AlmiChain-Clientes, modo 11bgn mixed, canal 6 fijo, seguridad WPA2-PSK con AES y clave generada en KeePassXC.



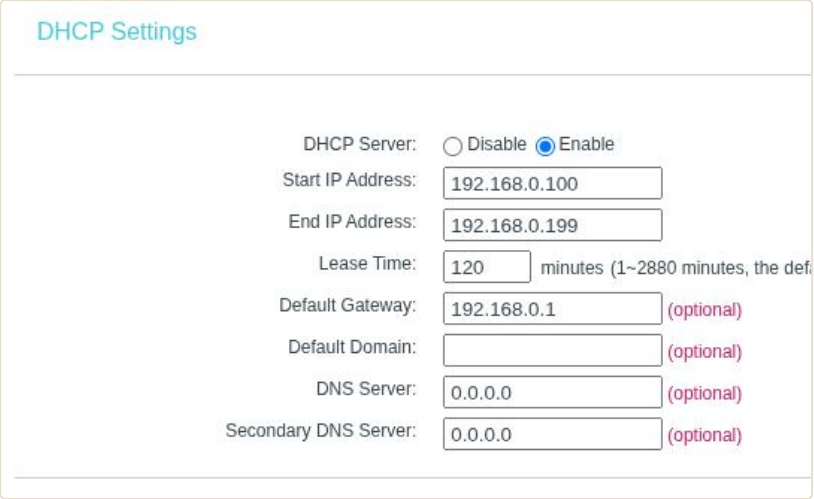
SSID y canal · GitHub: redes/routers/router/capturas/r1_03_wifi_ssid_canal.png

5. WPS desactivado, gestión remota cerrada (IP 0.0.0.0) y red de invitados desactivada, porque toda la Wi-Fi del router es de clientes.



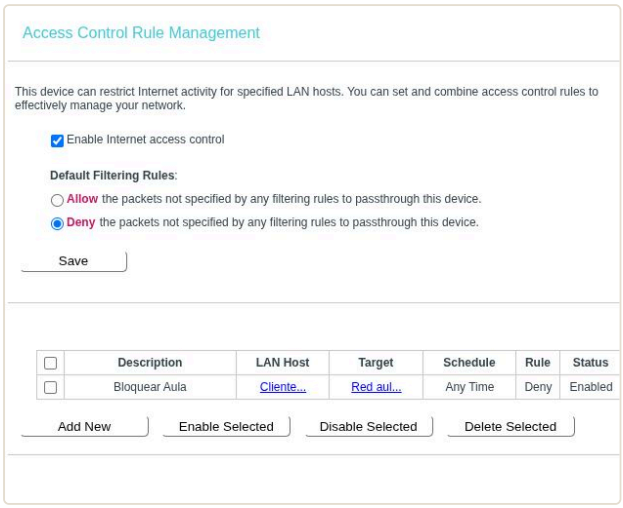
WPS desactivado · GitHub: redes/routers/router/capturas/r1_04_wps_desactivado.png

6. DHCP activado, de 192.168.0.100 a 192.168.0.199.



DHCP · GitHub: redes/routers/router/capturas/r1_08_dhcp.png

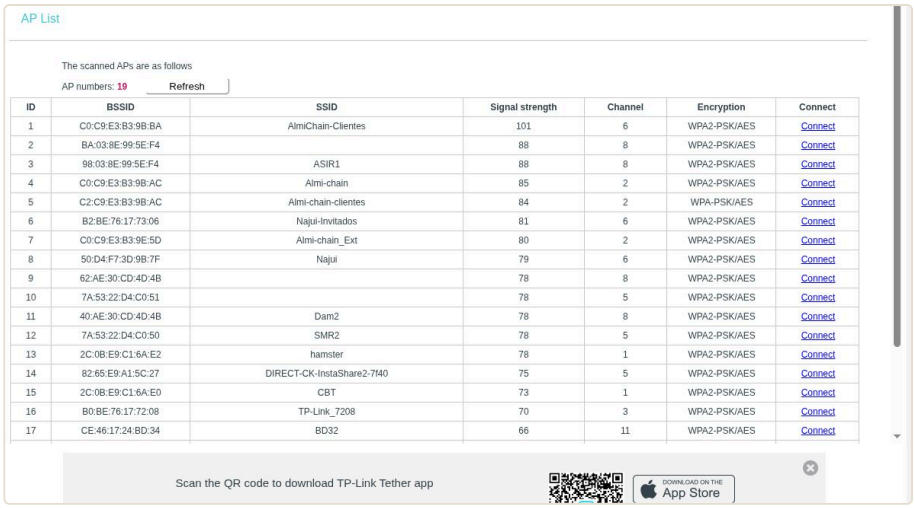
7. Aislamiento con Access Control: un host “Clientes” (192.168.0.100-199), un destino “Red aula” (192.168.1.1-254, todos los puertos) y una regla “Bloquear aula” siempre activa. Internet sigue funcionando porque las páginas web no tienen IP 192.168.1.x, pero los paquetes de un cliente hacia un equipo del aula deben bloquearse.



Regla · GitHub: redes/routers/router/capturas/r1_06_access_control.png

5. Repetidor

1. Reset de fábrica y contraseña de administración generada, distinta de la del router.
2. WPS desactivado antes de pasar a modo repetidor. En modo Range Extender el menú de Wireless no muestra la opción WPS, así que no se puede volver a comprobar desde el panel.
3. Quick Setup, Range Extender. En el escaneo salen muchas redes de otros grupos con nombres casi iguales, así que se elige la red comprobando que su BSSID es el del router (C0:C9:E3:B3:9B:BA, entero, hasta el último byte) y no solo por el nombre.



Elección de la red por su BSSID · GitHub: redes/routers/repetidor/capturas/r2_02_range_extender.png

4. Se escribe la clave de la Wi-Fi del router (la misma, no una nueva) pegándola desde KeePassXC, y se deja el mismo nombre de red.
5. Tras reiniciar, se comprueba en Status el modo Range Extender, el canal 6 y la conexión con el router.

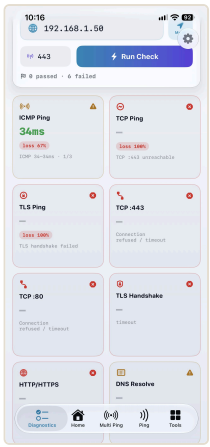
6. Prueba de aislamiento

La regla de Access Control del router impide que los clientes de la Wi-Fi lleguen al servidor de Odoo (`192.168.1.50`), y el resto de internet sigue funcionando.

#	Prueba	Resultado esperado	Resultado	Captura
1	PC conectado al router (IP `192.168.0.100`): `ping 8.8.8.8`	Responde (hay internet)	✅ 0 % de pérdida	p1_pruebas
2	Mismo PC: `ping 192.168.0.1`	Responde (la red local funciona)	✅ 0 % de pérdida	p1_pruebas
3	Mismo PC: `ping 192.168.1.50`	Falla (servidor aislado)	✅ 100 % de pérdida, «Destination Host Unreachable»	p1_pruebas
4	Mismo PC: `curl https://192.168.1.50`	No conecta	✅ «Couldn’t connect to server»	p1_pruebas
5	Móvil conectado a `AlmiChain-Clientes`	Conectado	✅	p1_movil
6	Móvil: Odoo (`192.168.1.50`, puerto 443)	No conecta	✅ TCP 443 bloqueado al 100 %. La app marca 1 de 3 respuestas en el ping ICMP; la prueba 3, con `ping` desde el PC, confirma que el servidor no responde	p2_aislamiento
7	Móvil: navegar por internet	Carga	✅	p3_buscador



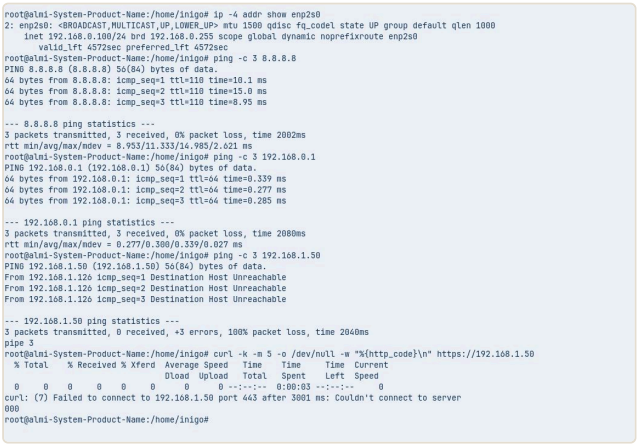
Móvil conectado a AlmiChain-Clientes · GitHub: redes/routers/repetidor/capturas/p1_movil_conectado.PNG



Móvil: Odoo (192.168.1.50, puerto 443) · GitHub: redes/routers/repetidor/capturas/p2_aislamiento_movil.PNG



Móvil: navegar por internet · GitHub: redes/routers/repetidor/capturas/p3_buscador_funcionable.PNG



```
root@almi-System-Product-Name:/home/inigo# ip -4 addr show enp2s0
2: enp2s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    inet 192.168.0.100/24 brd 192.168.0.255 scope global dynamic noprefixroute enp2s0
       valid_lft 4572sec preferred_lft 4572sec
root@almi-System-Product-Name:/home/inigo# ping -c 3 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=110 time=10.1 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=110 time=15.0 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=110 time=8.95 ms

--- 8.8.8.8 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2002ms
rtt min/avg/max/mdev = 8.953/11.333/14.985/2.621 ms
root@almi-System-Product-Name:/home/inigo# ping -c 3 192.168.0.1
PING 192.168.0.1 (192.168.0.1) 56(84) bytes of data.
64 bytes from 192.168.0.1: icmp_seq=1 ttl=64 time=0.339 ms
64 bytes from 192.168.0.1: icmp_seq=2 ttl=64 time=0.277 ms
64 bytes from 192.168.0.1: icmp_seq=3 ttl=64 time=0.285 ms

--- 192.168.0.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2080ms
rtt min/avg/max/mdev = 0.277/0.300/0.339/0.027 ms
root@almi-System-Product-Name:/home/inigo# ping -c 3 192.168.1.50
PING 192.168.1.50 (192.168.1.50) 56(84) bytes of data.
From 192.168.1.126 icmp_seq=1 Destination Host Unreachable
From 192.168.1.126 icmp_seq=2 Destination Host Unreachable
From 192.168.1.126 icmp_seq=3 Destination Host Unreachable

--- 192.168.1.50 ping statistics ---
3 packets transmitted, 0 received, +3 errors, 100% packet loss, time 2040ms
pipe 3
root@almi-System-Product-Name:/home/inigo# curl -k -m 5 -o /dev/null -w "%{http_code}\n" https://192.168.1.50
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:--  0:00:03 --:--:--     0
curl: (7) Failed to connect to 192.168.1.50 port 443 after 3001 ms: Couldn't connect to server
000
root@almi-System-Product-Name:/home/inigo#
```

Pruebas desde el PC · GitHub: redes/routers/repetidor/capturas/p1_pruebas_repetidor.png

7. Incidencias

- Error 80002 (Unknown error) al guardar cambios de Wi-Fi desde el móvil: la Wi-Fi se reinicia al guardar y se corta la conexión con el panel. Solución: configurar siempre por cable.
- El reset de fábrica reactiva el WPS y vuelve a la contraseña de fábrica: hay que desactivarlo y cambiarla.
- En la primera configuración, el servidor seguía siendo accesible desde la red de clientes (respondían el ping y la web). Se corrigió la regla de Access Control y se repitió la prueba.
- Con la regla ya bien, se cortó también internet: en *Default Filtering Rules* estaba marcado **Deny**, que bloquea todo lo que no está en una regla. Se cambió a **Allow** y la regla «Bloquear Aula» se dejó en Deny: así solo se bloquea la red del aula.
- Una contraseña inicial débil llegó a escribirse en un chat. Se cambió tras el reset y no se reutiliza.

8. Riesgos y desviaciones

- No hay WDS: se usa repetidor, por indicación del profesor.
- No hay red de empleados y red de clientes separadas en el router: toda su Wi-Fi es de clientes, y los equipos y el servidor siguen en la red del aula (192.168.1.x). El plano preveía 10.13.10.x y 10.13.20.x.
- La regla solo protege al rango del DHCP del router. Un equipo con IP fija fuera de ese rango no cae en ella.
- Solo se prueba el ping y HTTPS al servidor. Otros puertos no se han comprobado.
- No hay captura de la contraseña de administración ni de la pantalla de *Wireless Security* (WPA2-PSK con AES): se configuraron, pero no se capturaron.
- Firmware de 2018 sin parches de seguridad en los dos aparatos.

- El nombre AlmiChain-Clientes se parece al de redes de otros grupos, por eso se compara la MAC y no el nombre.
- Canal 6 compartido con otras redes del aula, lo que puede bajar la velocidad.

9. Cómo repetirlo

1. Reset de los dos aparatos. Contraseña de administración distinta en cada uno, generadas en KeePassXC.
2. Router: modo Wireless Router, WAN Dynamic IP, Wi-Fi con WPA2-PSK y AES en canal 6 fijo, WPS desactivado, gestión remota cerrada y DHCP 192.168.0.100-199.
3. Router: regla de Access Control (host, destino y regla) para bloquear la red del aula, con *Default Filtering Rules* en **Allow**, y probarla con ping y curl.
4. Repetidor: WPS desactivado, Quick Setup, Range Extender, red elegida por su BSSID y la misma clave.
5. Guardar las capturas y los backups de los aparatos en KeePassXC (nunca en GitHub).

3. Equipos: hardware, software y presupuesto

Responsable: Jose

> **Dónde está en GitHub**
>
> | | |
> |---|---|
> | Fichas de hardware y software | `docs/presupuesto/fichas_hw_sw.md` |
> | Presupuesto en Excel | `docs/presupuesto/presupuesto_local_plano.xlsx` |

Resumen

Para cada equipo (gerente, TPV, impresora de tickets, servidor, red, rack y cableado, recreativa y arcade stick) se indica el modelo, las características, el software, el precio real con su enlace y fecha, y por qué se ha elegido. Todo el software es libre o viene incluido con el equipo. En vez de Windows 10, que ya no tiene soporte, se usa Windows 11. El presupuesto da los precios sin IVA, con el 21 % de IVA aparte, y separa la compra inicial de los gastos mensuales (fibra y AWS).

Fichas de hardware y software y presupuesto

> **En GitHub:** `docs/presupuesto/fichas_hw_sw.md`

Responsable	Fechas (Gantt)	Carpeta en el repo	Precios consultados
Jose	27/10 – 29/10	`docs/`	22/10/2026 (sin IVA + 21 % de IVA aparte)

El presupuesto detallado, sin IVA y con el IVA aparte, está en `presupuesto_local_plano.xlsx`. Sigue el **plano del local (cableado y Wi-Fi)** y el **esquema de red** del equipo. Este documento explica **qué** se compra para cada equipo, **qué software** lleva y **por qué**.

> **Laboratorio frente a compra real.** En el reto montamos los equipos como máquinas virtuales en VirtualBox y la web en AWS Academy, así que a nosotros no nos cuesta nada. Este presupuesto es lo que tendría que comprar **el restaurante** para tenerlo funcionando de verdad.

Resumen

Concepto	Importe
Compra inicial de hardware	**5.595,56 €** con IVA (4.624,43 € + IVA)
Software	**0 €** (libre o incluido en el equipo)
Costes recurrentes	**≈ 47 €/mes** con IVA (≈ 568 €/año)
Primer año	**≈ 6.164 €** con IVA

Equipo	Total
Gerente	1.053,59 €
TPV de barra (×2, con impresora)	1.899,36 €
Servidor	1.950,78 €
Red (router, switches y puntos de acceso)	201,98 €
Rack y cableado	431,40 €
Recreativa y arcade stick	58,45 €

Por qué Windows 11 y no Windows 10

El enunciado de 2022-2023 pide Windows 10, pero **Windows 10 dejó de tener soporte el 14/10/2025**. Desde entonces solo recibe parches de seguridad pagando las actualizaciones ampliadas (ESU): para empresas cuestan **61 $** el primer año (terminó el 13/10/2026), **122 $** el segundo y **244 $** el tercero, por equipo.

Por eso, en la compra real, el equipo decidió usar **Windows 11** en el gerente y en los TPV: - **Seguridad por defecto:** sigue recibiendo actualizaciones gratis. Un TPV que cobra a clientes no puede llevar un sistema sin parches. - **Sin coste extra:** los equipos presupuestados ya lo traen, el del gerente con Windows 11 Pro y los TPV con Windows 11 IoT Enterprise LTSC (soporte hasta 2034 y pensado para equipos de función fija como un TPV). Así nos ahorramos el ESU.

En el reto (máquinas virtuales del aula): el gerente se montó con **Windows 11 Pro** (validado por el profesor el 26/10/2026). El TPV se montó con **Windows 10 Pro**, porque así se hizo la VM en el aula; el usuario estándar, el quiosco y el certificado se configuran igual en las dos versiones. En la compra real, los TPV ya traen Windows 11 IoT Enterprise LTSC.

---

1. Equipo del gerente

Modelo	Lenovo ThinkCentre neo 50s Gen 5 (SFF)
Procesador	Intel Core i5-14400 (10 núcleos, 16 hilos, hasta 4,7 GHz)
RAM	16 GB DDR5 (ampliable a 64 GB)
Disco	SSD NVMe de 512 GB
Red	Ethernet 1 Gbps, Wi-Fi 6E, Bluetooth 5.3
Monitor	AOC 24B2XH de 23,8'', IPS Full HD
Precio	962,60 € + 90,99 €

Software: Windows 11 Pro y Ubuntu Desktop 26.04 LTS en arranque dual con GRUB, Microsoft Edge y Firefox, el navegador para entrar en Odoo y KeePassXC.

Por qué: el i5 y los 16 GB sobran para ofimática y Odoo durante años. 512 GB dan para dos sistemas (unos 250 GB cada uno). La garantía y los repuestos de una gama profesional (ThinkCentre) son mejores que los de un equipo doméstico.

Seguridad: el gerente usa una cuenta **sin permisos de administrador** en los dos sistemas. El inicio rápido de Windows va desactivado (si no, Windows deja el disco bloqueado y Ubuntu no puede montarlo). Se recomienda cifrar el disco con BitLocker y LUKS.

2. TPV de barra (×2)

Modelo	10POS 10T 15″, todo en uno táctil
Procesador	Intel Core i5, 4 núcleos
RAM / disco	8 GB DDR4 / SSD de 256 GB
Pantalla	15″ táctil capacitiva, 1024×768
Puertos	6 USB, 2 RS-232, Ethernet Gigabit, HDMI y VGA (para un visor de cliente)
Sistema incluido	Windows 11 IoT Enterprise LTSC
Precio	708,48 € cada uno

Software: Windows 10 Pro en la VM del reto; Windows 11 IoT Enterprise LTSC en la compra real, Edge en **modo quiosco** abriendo el TPV de Odoo por HTTPS, el certificado del servidor instalado como de confianza y el driver de la impresora Epson.

Por qué: un TPV todo en uno ocupa poco en la barra y aguanta grasa y golpes mejor que un PC con monitor. La pantalla táctil es más rápida para el camarero. Como Odoo funciona en el navegador, no hace falta más potencia.

Seguridad: un usuario estándar sin administrador y en modo quiosco (solo puede usar el TPV), los USB que no se usan bloqueados y conexión por cable a la red de empleados, nunca al Wi-Fi de clientes.

3. Impresora de tickets (×2)

Modelo	Epson TM-T20III
Tecnología	Térmica directa (no gasta tinta), papel de 80 mm
Velocidad	250 mm/s
Conexión	USB + serie RS-232
Cortador	Automático (1,5 millones de cortes)
Precio	241,20 € cada una

Por qué: Epson es el estándar en hostelería y la usan casi todos los programas de TPV. Es térmica, así que solo hay que comprar rollos de papel. Va por USB a su TPV, sin estar en la red, lo que reduce la superficie de ataque.

4. Servidor

Modelo	Lenovo ThinkSystem ST50 V3 (torre)
Procesador	Intel Xeon E-2434
RAM	16 GB
Discos	2 × 1 TB SATA (**discos A y B, RAID 1**) + Seagate IronWolf de 4 TB (**disco C, copias**)
Bahías	3 × 3,5″
Fuente	500 W
SAI	APC Back-UPS de 700 VA / 405 W

Precio	1.604,84 € + 214,95 € + 130,99 €

Software: Ubuntu Server 24.04 LTS, Docker y Docker Compose, Odoo Community y PostgreSQL, Nginx con HTTPS delante de Odoo, `mdadm` (RAID 1), `restic` y `cron` (copias cifradas), UFW (cortafuegos) y SSH con claves.

Por qué: - Un **servidor de verdad** (Xeon) admite memoria ECC, que corrige errores de RAM, y está pensado para funcionar 24 horas. - **RAID 1:** si se rompe el disco A, el B tiene lo mismo y el restaurante sigue funcionando (failover). - **Disco C aparte:** el RAID **no es una copia de seguridad** (si se borra algo, se borra en los dos discos). Por eso las copias van a otro disco. - **SAI:** un corte de luz en plena escritura puede corromper la base de datos. El SAI da tiempo para apagar el servidor bien.

Seguridad: solo están abiertos los puertos necesarios (443, y 22 solo desde la red de administración), SSH sin contraseña (solo con claves) y sin acceso de `root`. PostgreSQL no se expone a la red y cada base de datos tiene su propio usuario. Las contraseñas se guardan en KeePassXC, nunca en GitHub.

5. Red

Así queda la red según el plano: la fibra entra en el **rack** del despacho del gerente. Allí están la ONT del operador, nuestro router **R1-AlmiChain** y dos switches. De ahí salen los cables a cada equipo y al punto de acceso del comedor.

Equipo	Modelo	Función	Precio
R-Operador (ONT)	El que da el operador	Convierte la fibra en red. Da la IP pública por DHCP	Incluido en la fibra
R1-AlmiChain	TP-Link ER605 (5 puertos Gigabit, VLAN)	Router propio: NAT, DHCP, DNS. Red de empleados 10.13.10.0/24 y red de clientes 10.13.20.0/24, aisladas con ACL	65,00 €
SW-Empleados	TP-Link TL-SG108E (8 puertos, gestionable)	Conecta servidor, PC del gerente, TPV 1, TPV 2 y recreativa	29,99 €
SW-WiFi	TP-Link TL-SG105 (5 puertos)	Conecta el AP principal (red de clientes)	16,99 €
AP principal	TP-Link Archer C6 AC1200 en modo punto de acceso	Wi-Fi de clientes en el comedor, WPA2-AES, canal 6	45,00 €
AP repetidor	TP-Link Archer C6 AC1200	**Repetidor** (modo *Range Extender*) al otro lado del comedor, solo con corriente. Repite la Wi-Fi de clientes sin cable	45,00 €

Por qué un router propio: el router del operador no deja hacer ACL ni separar bien las dos redes. Con R1 controlamos nosotros la red, y si cambiamos de operador solo cambia la ONT.

Por qué dos switches: así los equipos de empleados y el Wi-Fi de clientes van por cables y switches distintos. Un cliente del Wi-Fi no puede llegar al servidor ni a los TPV.

Seguridad: WPA2-AES (o WPA3 si los dos AP lo admiten), WPS desactivado, contraseñas de administración cambiadas y DHCP desactivado en el repetidor.

Por qué un repetidor y no WDS: el enunciado pedía enlazar el segundo router por WDS, pero el router del aula (TP-Link TL-WR841N v14) no tiene esa opción en su firmware, y el profesor indicó usarlo como **repetidor**. El repetidor se conecta por Wi-Fi a la red de clientes del AP principal y vuelve a emitirla, así que amplía la

cobertura sin tender cable hasta el otro lado del comedor. Los clientes que se conectan a él siguen en la red de clientes, aislados de los equipos de la empresa. La configuración y las pruebas están en `redes/routers/router-y-repetidor.md`.

> Los precios de R1 y de los switches son orientativos.

6. Rack y cableado

Elemento	Cantidad	Precio
Armario rack mural 19″ de 9U con puerta de cristal y llave	1	89,90 €
Bandeja fija 1U (para la ONT, R1 y los switches)	1	14,90 €
Regleta de rack de 8 enchufes	1	24,90 €
Patch panel de 12 puertos Cat 6	1	24,90 €
Bobina de cable UTP Cat 6 de 100 m (cobre)	2	99,80 €
Toma de red Cat 6 de superficie	5	27,50 €
Latiguillos Cat 6 (6 de 0,5 m y 5 de 2 m)	11	32,50 €
Canaleta de PVC de 2 m para el techo	30	117,00 €
Total		**431,40 €**

Cuánto cable: según el plano, del rack a cada equipo hay TPV 1 29 m, TPV 2 23 m, AP principal 17 m, recreativa 27 m y PC del gerente 8 m. En total son **104 m**, por eso hacen falta 2 bobinas de 100 m. Todos los tramos miden menos de 90 m, y el límite del cable UTP es de 100 m.

Por qué Cat 6: llega a 1 Gbps sin problemas y es lo normal hoy en una instalación nueva. Va por canaleta en el techo para que no se pise ni se moje.

Por qué rack con llave: los equipos de red y el servidor quedan juntos, en el despacho y cerrados. Nadie de la sala puede desconectarlos ni enchufar algo. El servidor es de torre, así que va al lado del rack, y el SAI protege también la ONT, el router y los switches.

> La instalación la hace el propio equipo, así que no se presupuesta mano de obra. Los precios de esta tabla son orientativos.

7. Máquina recreativa y arcade stick

Equipo	El que ya tiene el cliente: 1 núcleo y 1 GB de RAM (coste 0 €)
Controlador	Arduino Leonardo Rev3 (ATmega32u4): 25,40 €
Mandos	Kit de 1 jugador con Eurojoystick 2, 10 botones y cableado: 33,05 €

Software: antiX core (una distribución Linux muy ligera, sin escritorio), MAME arrancando directamente y el IDE de Arduino para programar el stick.

Por qué: - **antiX core:** con 1 GB de RAM no cabe un escritorio normal. antiX arranca con unos 100 MB y deja el resto para MAME. - **Arduino Leonardo:** su chip (ATmega32u4) tiene USB nativo, así que el ordenador lo reconoce como un **teclado o mando** sin instalar drivers y funciona con cualquier emulador. Un Arduino Uno no puede hacerlo. - El kit trae su propio encoder USB, pero lo sustituimos por el Arduino porque el enunciado pide que el control se pueda personalizar.

8. Software (todo de coste 0)

Software	Dónde	Licencia	Por qué	Instalación
Windows 11 Pro / Windows 11 IoT Enterprise LTSC	Gerente, TPV	Incluida con el equipo (OEM)	Con soporte, a diferencia de Windows 10 (en el reto, la VM del TPV usa Windows 10 Pro)	Gerente §4 · TPV
Ubuntu Desktop 26.04 LTS / Ubuntu Server 24.04 LTS	Gerente / servidor	Libre (GPL)	Soporte de 5 años (hasta 2031 y 2029)	Gerente §6 · servidor, `01-base.sh`
Odoo Community	Servidor	Libre (LGPL)	ERP y TPV en uno (lo justifica Alejandro)	ERP, `04-odoo-docker.sh`
PostgreSQL	Servidor	Libre (licencia PostgreSQL)	Base de datos que usa Odoo	ERP · BD
Docker / Docker Compose	Servidor	Libre (Apache 2.0)	Despliegue reproducible	ERP, `04-odoo-docker.sh`
Nginx, Certbot	Servidor, AWS	Libre	HTTPS	ERP · AWS
mdadm, restic, UFW	Servidor	Libre	RAID, copias cifradas, cortafuegos	Copias y RAID
KeePassXC	Gerente, administración	Libre (GPL)	Gestor de contraseñas: nada de contraseñas en papel ni en GitHub	Paquete del sistema (`apt install keepassxc`)
antiX, MAME	Recreativa	Libre (GPL)	Ligero / emulador	Recreativa
Arduino IDE	Para programar el stick	Libre		Arcade stick
Terraform	Para AWS	Gratis (BSL)	Infraestructura como código	AWS · copias en S3
VirtualBox	Solo en el laboratorio	Libre (GPL)	No se usa en la compra real	—

9. Costes recurrentes

Concepto	Al mes
Fibra Vodafone Empresas 1 Gb simétrica (la eligió Yeray; precio desde el mes 13)	35,00 € con IVA (28,93 € + IVA). El primer año, 25 € con IVA por oferta
AWS: EC2 t3.micro + IP elástica + disco de 8 GB	≈ 11,90 $ (≈ 10,20 €) + IVA
DuckDNS + Let's Encrypt	0 €

En el reto, AWS Academy Learner Lab no nos cobra nada. El precio de AWS es lo que pagaría el restaurante con una cuenta normal.

10. Fuentes (consultadas el 22/10/2026)

Todos los enlaces de compra están en el Excel, en cada fila. Además: - Fin del soporte de Windows 10 y precios de ESU (motivo para usar Windows 11): https://www.techrepublic.com/article/news-windows-10-esu-cheat-sheet/ - Precio de la IPv4 pública en AWS: https://aws.amazon.com/blogs/aws/new-aws-public-ipv4-address-charge-public-ip-insights - Comparativa de fibra para empresas: https://selectra.es/internet-telefono/empresas/fibra - Elección de la fibra: `docs/Comparativa y contratación de la línea WAN.pdf` (Yeray)

> Los precios cambian. Antes de entregar, vuelve a abrir los enlaces, actualiza los que hayan cambiado y pon la nueva fecha.

4. Equipo del gerente: arranque dual

Responsable: Sendoa

> **Dónde está en GitHub**
>
Guía completa	`sistemas/gerente-dualboot/README.md`
Capturas	`sistemas/gerente-dualboot/capturas`

Resumen

El equipo del gerente es una VM de VirtualBox con firmware **EFI, TPM 2.0 y arranque seguro**, 6 GB de RAM, 2 núcleos y un disco de 120 GB. Lleva **Windows 11 Pro** y **Ubuntu 26.04 LTS** en el mismo disco, y al encender **GRUB** deja elegir el sistema. Se usa Windows 11 en lugar de Windows 10 porque este ya no tiene parches de seguridad (validado por el profesor). Los dos sistemas están **cifrados**: BitLocker con contraseña de arranque en Windows y LUKS2 en Ubuntu. En cada uno hay un usuario `tecnico` que administra y un usuario `gerente` **sin permisos de administrador** (estándar en Windows y sin `sudo` en Ubuntu), con el UAC al máximo. El sistema está **endurecido**: se han desactivado los servicios que el gerente no usa (compartir carpetas, Escritorio remoto, UPnP, avahi...), el cortafuegos bloquea todo lo que entra en los dos sistemas y no queda ningún servicio accesible desde la red. En Windows, los servicios en marcha bajan de 98 a 79. Las cinco incidencias (instalación colgada, Windows que no arrancaba, errores de lectura de la ISO, la VM que arrancaba directamente en Windows y la entrada de Ubuntu perdida en el arranque UEFI) están documentadas con su causa, su solución y capturas. Las contraseñas y la clave de recuperación de BitLocker solo están en KeePassXC.

Equipo del gerente · Dual boot

> **En GitHub:** `sistemas/gerente-dualboot/README.md`

Equipo del gerente con **Windows 11 Pro** y **Ubuntu 26.04.1 LTS** en el mismo disco, con GRUB para elegir el sistema al arrancar. Los dos sistemas están cifrados y en cada uno el gerente trabaja con un usuario sin permisos de administrador.

Responsable: Sendoa · Fecha: 22–26 de octubre de 2026 · Montado en una VM de VirtualBox en el PC del aula.

> 🔐 En este documento no hay contraseñas ni claves. Todas están en el gestor KeePassXC del equipo (`almi-chain.kdbx` , grupo *Gerente – Dual boot*).

Índice

1. Por qué Windows 11 y no Windows 10
2. Máquina virtual
3. ISOs y verificación
4. Instalación de Windows 11
5. Preparar Windows para el dual boot
6. Instalación de Ubuntu
7. Configuración de Ubuntu
8. Cifrado de Windows con BitLocker

9. Resultado final
.0. Incidencias y cómo se resolvieron
.1. Seguridad aplicada
.2. Lista de capturas

---

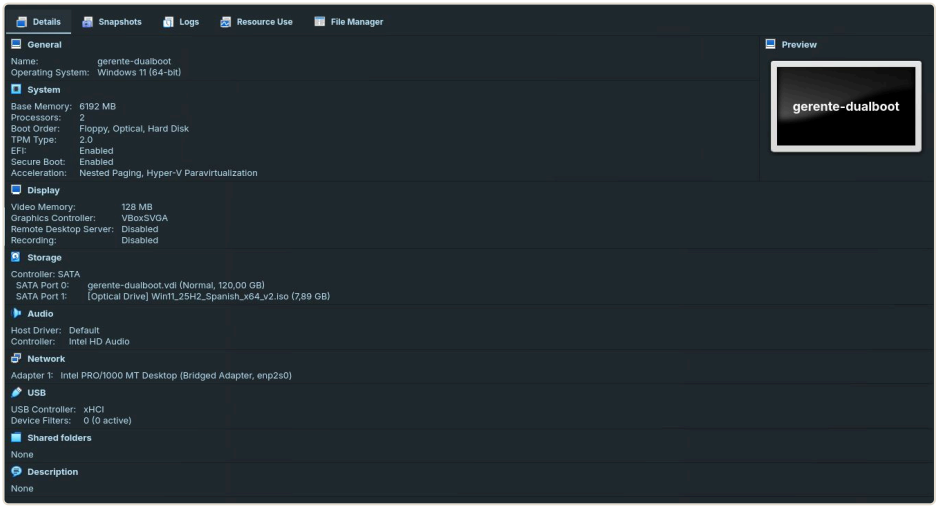
1. Por qué Windows 11 y no Windows 10

El enunciado pide Windows 10, pero **Windows 10 dejó de tener soporte el 14 de octubre de 2025**: ya no recibe parches de seguridad. En una empresa real no se instalaría un sistema sin soporte en el equipo del gerente, que maneja datos económicos. Por eso se usa Windows 11 Pro, que además permite BitLocker y directivas de grupo (`gpedit.msc`).

> ✅ **Validado por el profesor (26/10/2026):** Windows 11 en lugar de Windows 10 es válido para el reto.

2. Máquina virtual

Parámetro	Valor	Motivo
Software	VirtualBox 7.2	Anfitrión Linux del aula
Nombre	`gerente-dualboot`	
Firmware	**EFI**	Windows 11 lo exige; arranque UEFI como un PC actual
TPM	**2.0**	Requisito de Windows 11
Arranque seguro	Activado	Requisito de Windows 11
RAM	6192 MB (~6 GB)	Windows 11 pide 4 GB como mínimo; con 6 va fluido
CPU	2 núcleos	
Vídeo	128 MB	
Disco	120 GB VDI dinámico	~70 GB Windows + ~50 GB Ubuntu
Controlador	SATA (AHCI), **caché de E/S del anfitrión activada**	Ver incidencia 3
Red	Adaptador puente	La VM recibe IP de la red como un equipo más



G02 Detalles de la VM · GitHub: sistemas/gerente-dualboot/capturas/G02-detalles-vm.png

3. ISOs y verificación

Sistema	ISO	Origen
Windows 11 Pro 25H2	ISO multiedición x64, español	microsoft.com/es-es/software-download/windows11
Ubuntu 26.04.1 LTS	`ubuntu-26.04.1-desktop-amd64.iso`	ubuntu.com

Antes de instalar hay que comprobar el hash SHA-256 de cada ISO contra el oficial, para asegurar que no está corrupta ni manipulada.

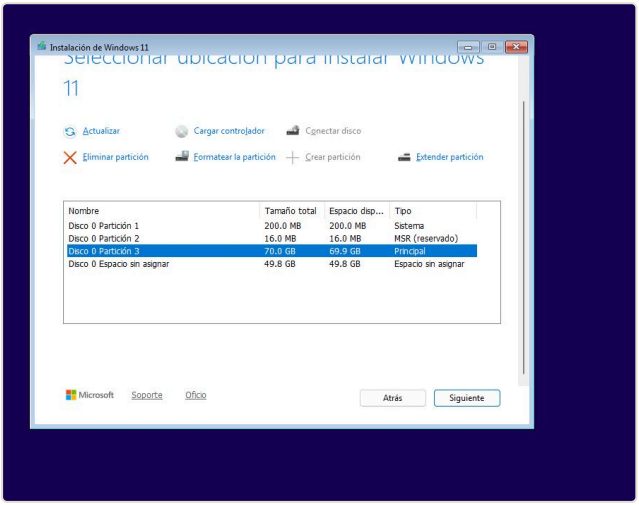
```
sha256sum ubuntu-26.04.1-desktop-amd64.iso
# 601e30fbf5d97759367c632e2c33630665039b7e2158fd068403da3ccf1bda1f
```

El resultado coincide con el publicado en `https://releases.ubuntu.com/26.04/SHA256SUMS`.

La ISO de Windows se descargó directamente de la web oficial de Microsoft, pero su hash no se llegó a comprobar. Queda como mejora para la próxima vez (ver *Lecciones aprendidas*).

4. Instalación de Windows 11

1. Arrancar desde la ISO e instalar **Windows 11 Pro**.
2. En el disco vacío de 120 GB, crear una partición de **71 680 MB** (70 GB) con el botón *Crear partición*. El instalador añade solo la EFI (200 MB), la reservada de Microsoft (16 MB) y la de recuperación. El resto se deja **sin asignar** para Ubuntu.
3. Cuenta **local**, sin cuenta de Microsoft: en la pantalla de conexión, `Shift + F10` y `start ms-cxh:localonly`. Se crea el usuario `tecnico` (administrador).
4. Privacidad: todas las opciones de telemetría y publicidad **desactivadas**.
5. Instalar las **Guest Additions** de VirtualBox (portapapeles compartido solo *Host To Guest*).
6. **Windows Update** hasta que no quede nada pendiente.
7. Desinstalar las aplicaciones preinstaladas que no se usan (juegos, noticias...).



Particiones creadas · G03 Particiones de Windows · GitHub: sistemas/gerente-dualboot/capturas/G03-particiones-windows.png

La contraseña de `tecnico` se genera y se guarda en KeePassXC (en la captura sale oculta):

5. Preparar Windows para el dual boot

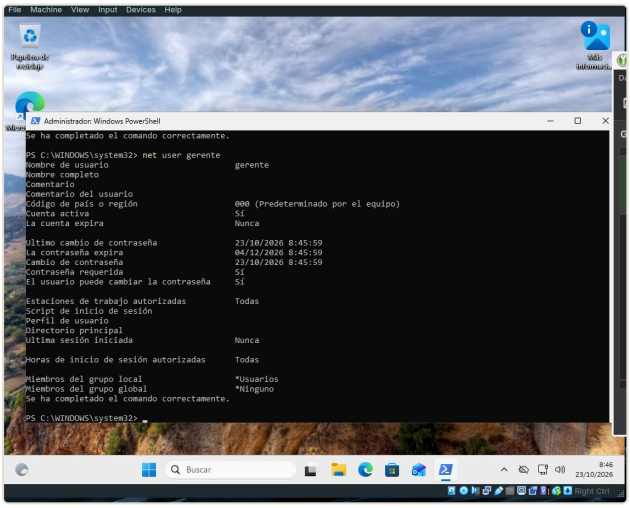
5.1 Desactivar el inicio rápido

El inicio rápido deja Windows medio hibernado al apagar. En un dual boot, eso hace que el disco de Windows quede “en uso” y que Linux pueda corromperlo si lo monta. En la VM no aparecía la casilla en Opciones de energía, así que se desactivó la hibernación (y con ella el inicio rápido) desde la terminal de administrador:

```
powercfg /h off
powercfg /a    # "Inicio rápido: la hibernación no está disponible"
```

5.2 Usuario gerente estándar

```
net user gerente * /add   # el * pide la contraseña sin mostrarla ni guardarla en el historial
net user gerente          # comprobar: solo grupo *Usuarios
```

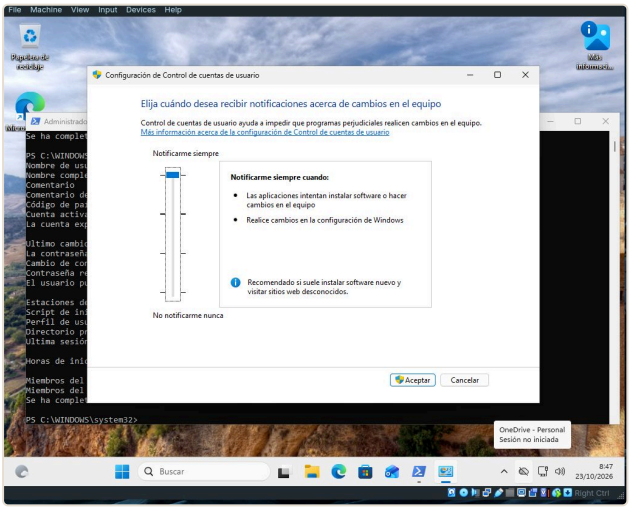


G08 gerente es usuario estándar · GitHub: sistemas/gerente-dualboot/capturas/G08-gerente-estandar-windows.png

La contraseña del gerente debe ser distinta de la del técnico. Al principio se puso la misma por error y se corrige con `net user gerente *` (desde una consola de administrador), poniéndole una contraseña propia.

5.3 Control de cuentas de usuario (UAC) al máximo

Nivel **“Notificarme siempre”**: si el gerente intenta instalar algo, Windows pide las credenciales del técnico.



G09 UAC al máximo · GitHub: sistemas/gerente-dualboot/capturas/G09-uac-maximo.png

5.4 Comprobar el espacio libre

`diskmgmt.msc`: quedan **49,79 GB sin asignar** al final del disco.

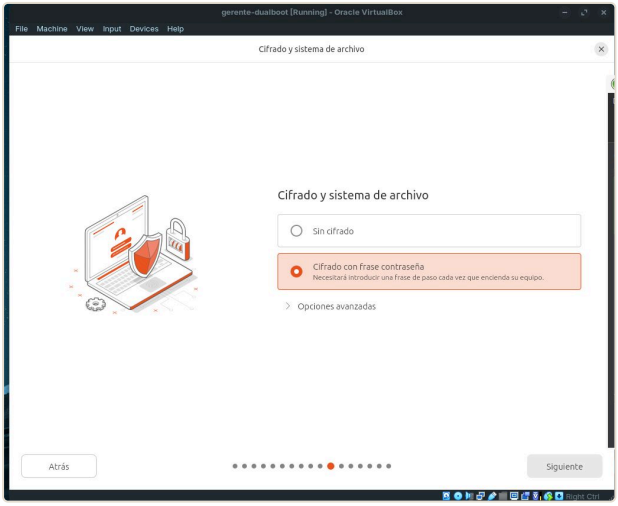
5.5 BitLocker antes de instalar Ubuntu

Windows 11 cifra el disco automáticamente al detectar el TPM ("esperando activación", sin protectores). El instalador de Ubuntu **no permite instalar al lado** de una partición con BitLocker, así que se descifró antes de instalar Ubuntu y se volvió a cifrar al final (ver apartado 8).

```
manage-bde -status C:
manage-bde -off C:     # descifrar; repetir -status hasta "Descifrado completo, 0,0%"
```

6. Instalación de Ubuntu

1. Antes de nada, **instantánea de la VM** (`Windows listo - antes de Ubuntu`) para poder volver atrás si algo falla.
2. Montar la ISO de Ubuntu y arrancarla desde el menú UEFI (`Esc` al arrancar → *Boot Manager* → *UEFI VBOX CD-ROM*), porque Windows Boot Manager arranca primero.
3. Idioma y teclado español, instalación interactiva, selección de aplicaciones **por defecto** (menos software, menos superficie de ataque), sin software de terceros.
4. Tipo de instalación: **"Instalar Ubuntu junto a Windows Boot Manager"**. Usa el espacio sin asignar y no toca Windows.
5. Cifrado: **"Cifrado con frase contraseña"** (LUKS + LVM). No se usa el cifrado con TPM porque en Ubuntu es experimental y da problemas con el dual boot.



G11b Cifrado con frase contraseña · GitHub: sistemas/gerente-dualboot/capturas/G11b-ubuntu-cifrado-frase.png

6. Usuario **`tecnico`** (administrador con `sudo`), equipo **`gerente-ubuntu`**, pedir contraseña al iniciar sesión.
7. Al terminar, quitar la ISO y reiniciar.

7. Configuración de Ubuntu

7.1 Actualizar

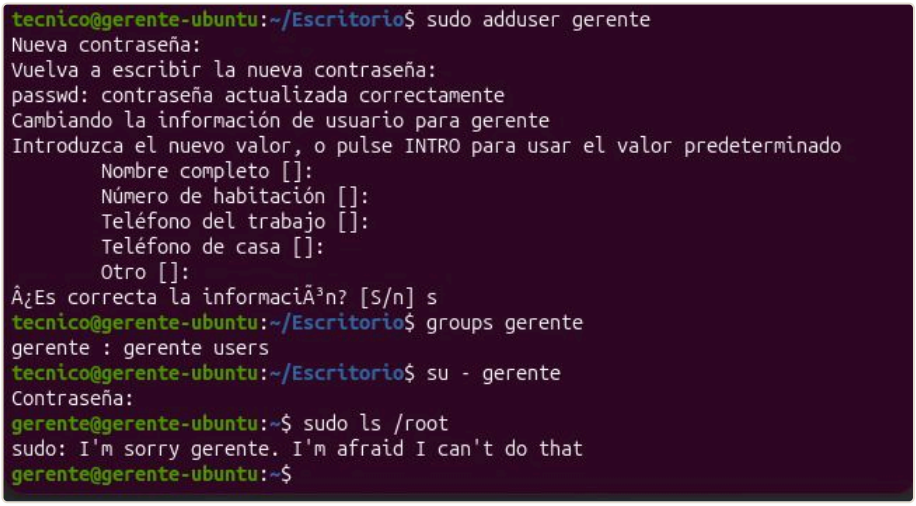
```
sudo apt update && sudo apt full-upgrade -y
sudo snap refresh
```

7.2 Usuario gerente sin sudo

```
sudo adduser gerente
groups gerente        # sin el grupo sudo
su - gerente
```

```
sudo ls /root         # "gerente no está en el archivo sudoers"
exit
```

`groups gerente` muestra solo `gerente users`, y al intentar `sudo` responde *"I'm sorry gerente. I'm afraid I can't do that"*:



G13 gerente sin sudo · GitHub: sistemas/gerente-dualboot/capturas/G13-gerente-sin-sudo.png

7.3 Orden de arranque

Ubuntu queda el primero en el firmware UEFI (`Boot0006 Ubuntu`), así que siempre aparece GRUB:

```
sudo efibootmgr
```

8. Cifrado de Windows con BitLocker

Al intentar proteger C: con el TPM apareció este error:

```
ERROR 0xc0310116: Los valores PCR de TPM no coinciden con lo que se calculó a partir del registro de
TCG.
```

El TPM mide toda la cadena de arranque. Ahora Windows arranca a través de shim/GRUB y el TPM virtual de VirtualBox no acepta esas medidas. En un PC real con TPM físico esto suele funcionar, pero aquí no. La solución fue usar una **contraseña de arranque**, igual que en Ubuntu, de forma que los dos sistemas piden una frase antes de arrancar.

1. `gpedit.msc` → *Configuración del equipo* → *Plantillas administrativas* → *Componentes de Windows* → *Cifrado de unidad BitLocker* → *Unidades del sistema operativo* → **"Requerir autenticación adicional al iniciar"** → *Habilitada* → marcar **"Permitir BitLocker sin un TPM compatible"**.
2. Aplicar y añadir los protectores:

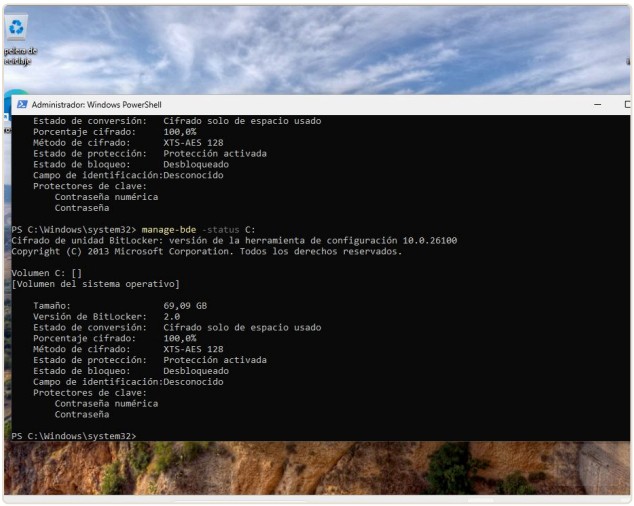
   ```
   gpupdate /force
   manage-bde -protectors -add C: -RecoveryPassword   # clave de recuperación de 48 dígitos → KeePassXC
   manage-bde -protectors -add C: -Password           # contraseña de arranque (solo minúsculas y números)
   manage-bde -on C: -UsedSpaceOnly
   ```

3. Reiniciar, elegir Windows en GRUB e introducir la contraseña de BitLocker.
4. `manage-bde -status C:` hasta **100 %, "Protección activada"**.

La contraseña de arranque usa solo minúsculas y números porque en la pantalla previa al arranque el teclado puede estar en inglés.

> ⚠️ Si Ubuntu actualiza GRUB, Windows puede pedir la clave de recuperación al arrancar. Es el comportamiento esperado de BitLocker ante un cambio en el arranque, y por eso la clave está guardada en KeePassXC.

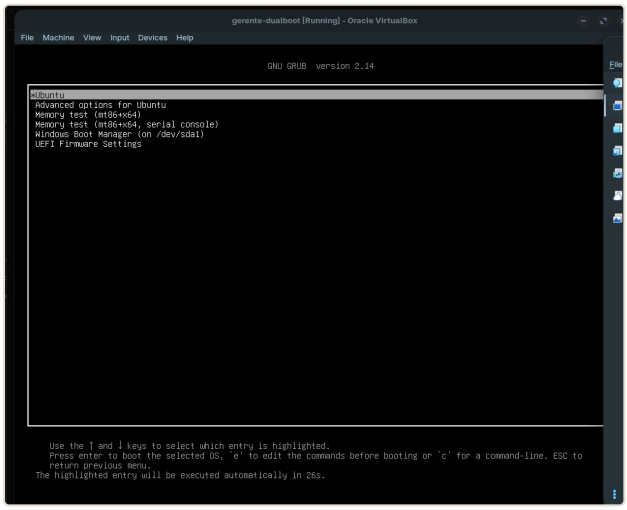
Al arrancar Windows, BitLocker pide la contraseña:



G15 BitLocker activado · GitHub: sistemas/gerente-dualboot/capturas/G15-bitlocker-activado.png

9. Resultado final

Menú de arranque



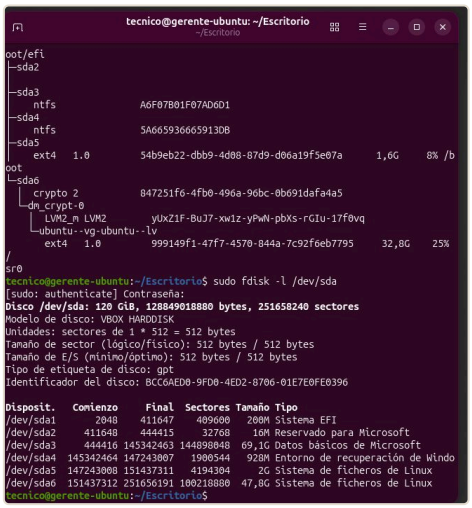
G12 GRUB con Ubuntu y Windows · GitHub: sistemas/gerente-dualboot/capturas/G12-grub-dual-boot.png

Particiones

```
lsblk -f
sudo fdisk -l /dev/sda
```

Partición	Tamaño	Sistema de archivos	Uso
sda1	200 MB	FAT32	**EFI**, compartida por Windows Boot Manager y GRUB
sda2	16 MB	—	Reservada de Microsoft
sda3	69,1 GB	NTFS + BitLocker	**Windows 11** (C:)
sda4	928 MB	NTFS	Recuperación de Windows

Partición	Tamaño	Sistema de archivos	Uso
sda5	2 GB	ext4	`/boot` de Ubuntu (sin cifrar, para poder pedir la frase al arrancar)
sda6	47,8 GB	LUKS2 → LVM → ext4	**Ubuntu** (`/`) cifrado



G14 Particiones · GitHub: sistemas/gerente-dualboot/capturas/G14-particiones-lsblk-fdisk.png

Usuarios

Sistema	Usuario	Permisos
Windows 11	`tecnico`	Administrador
Windows 11	`gerente`	Usuario estándar (UAC pide credenciales del técnico)
Ubuntu	`tecnico`	Grupo `sudo`
Ubuntu	`gerente`	Sin `sudo`

Cada usuario y cada cifrado tiene una contraseña distinta, todas en KeePassXC.

Demostración

Un arranque normal del equipo del gerente queda así:

1. Se enciende la VM y aparece **GRUB** (5 segundos para elegir; por defecto, Ubuntu).
2. **Windows:** GRUB → *Windows Boot Manager* → BitLocker pide su contraseña de arranque → inicio de sesión con el usuario *gerente*.
3. **Ubuntu:** GRUB → *Ubuntu* → se pide la frase de LUKS para descifrar el disco → inicio de sesión con el usuario *gerente*.

En los dos sistemas el disco está cifrado antes de llegar al inicio de sesión, y el gerente entra siempre sin permisos de administrador.

10. Incidencias y cómo se resolvieron

Incidencia 1: la instalación de Ubuntu se quedó colgada

Qué pasó: el primer intento de instalación se quedó más de una hora parado en el paso `curtin command in-target`, sin escribir nada nuevo en el registro.

Qué se hizo: apagado forzado de la VM. Al reintentar, el instalador avisó de que Windows tenía BitLocker y pidió reiniciar Windows.

Incidencia 2: Windows dejó de arrancar

Qué pasó: al arrancar Windows apareció *"El dispositivo tuvo un problema y no se pudo reparar"*. La reparación automática falló.

Diagnóstico y solución desde *Solucionar problemas → Opciones avanzadas → Símbolo del sistema*:

```
diskpart
list vol                      # C: sigue ahí, NTFS 69 GB, Correcto
select vol 2                  # partición EFI (FAT32, 200 MB)
assign letter=S
exit
chkdsk C: /f                  # sin errores, 0 KB en sectores defectuosos
bcdboot C:\Windows /s S: /f UEFI   # "Archivos de arranque creados correctamente"
```

Los datos de Windows estaban intactos y solo se había roto el arranque. `bcdboot` lo volvió a crear y Windows arrancó. Después se descifró C: (`manage-bde -off C:`) y se borraron desde `diskmgmt.msc` las dos particiones que el intento fallido había dejado a medias (2 GB y 47,79 GB), hasta volver a tener los ~49,8 GB sin asignar.

Incidencia 3: errores de lectura de la ISO durante la instalación

Qué pasó: en el segundo intento apareció `rsync: read errors mapping ".../firefox_8763.snap": Input/output error (5)`.

Diagnóstico: se comprobó el hash de la ISO en el anfitrión y **coincidía con el oficial**, así que el archivo estaba bien. El fallo venía de la lectura del CD virtual: el disco del PC del aula es lento (extraer el sistema tardó unos 7 minutos) y VirtualBox daba errores de E/S. Seguramente era también la causa del cuelgue de la incidencia 1.

Al principio la opción estaba desactivada:

Solución: la instalación terminó correctamente. Para que no vuelva a pasar, se recomienda activar **"Use Host I/O Cache"** en el controlador SATA de la VM, que hace los accesos a disco más rápidos en un PC con disco lento. Al terminar, se comprobó que Firefox estaba instalado y que el sistema se actualizaba sin errores.

Incidencia 4: la VM arrancaba directamente en Windows

Qué pasó: tras instalar Ubuntu, la VM entraba en Windows sin mostrar GRUB, porque `bcdboot` había puesto Windows Boot Manager el primero. Además, la ISO seguía montada.

Solución: quitar la ISO de la unidad óptica y comprobar con `efibootmgr` que la entrada *Ubuntu* estaba la primera en `BootOrder`. Desde entonces aparece GRUB en cada arranque.

Incidencia 5: Ubuntu desapareció del arranque (26 de octubre)

Qué pasó: al encender la VM salía directamente la contraseña de BitLocker y arrancaba Windows, sin menú GRUB. En el menú de la UEFI (tecla **Esc** → *Boot Manager*) ya no aparecía la entrada *ubuntu*.

Causa: las entradas de arranque UEFI no se guardan en el disco, sino en la NVRAM de la VM (el archivo `.nvram` de VirtualBox). La entrada *ubuntu* se perdió de ahí, así que el firmware pasó a arrancar el primero que quedaba: *Windows Boot Manager*. Los archivos de GRUB seguían intactos en la partición EFI (`\EFI\ubuntu\`).

Solución:

1. Arrancar Ubuntu a mano: **Esc** → *Boot Maintenance Manager* → *Boot From File* → partición EFI → `EFI\ubuntu\shimx64.efi`.
2. Comprobar el orden con `sudo efibootmgr`: el `BootOrder` empezaba por `0003` (Windows) y no había entrada *ubuntu* permanente.
3. Volver a crear la entrada. Se arranca `shimx64.efi` (y no `grubx64.efi` directamente) para que siga funcionando con el arranque seguro:

```
sudo efibootmgr -c -d /dev/sda -p 1 -L ubuntu -l '\EFI\ubuntu\shimx64.efi'
```

La nueva entrada `0006 ubuntu` queda la primera (`BootOrder: 0006,0003,...`).

4. Borrar la entrada temporal que creó *Boot From File*: `sudo efibootmgr -b 0005 -B`.
5. Reiniciar: vuelve a aparecer GRUB con Ubuntu y Windows Boot Manager.

Cómo evitarlo: no borrar ni separar el archivo `.nvram` de la carpeta de la VM, y copiar o exportar siempre la carpeta completa de la máquina. Si vuelve a pasar, los pasos de arriba lo arreglan en dos minutos y sin tocar los datos.

Lecciones aprendidas

- **Verificar el hash de las ISOs antes de instalar**, no después. Así se descarta enseguida que el archivo esté corrupto.
- **Quitar BitLocker antes de instalar otro sistema** al lado de Windows.
- **Hacer una instantánea antes de cada cambio arriesgado**: volver atrás lleva segundos en vez de horas reparando.
- Las entradas de arranque UEFI viven en la NVRAM de la VM, no en el disco: si desaparece GRUB, el sistema no se ha borrado y se recupera con `efibootmgr`.
- Si una instalación se para, revisar los indicadores de disco y red de VirtualBox antes de esperar más.

11. Seguridad aplicada

- Sistema operativo **con soporte** (Windows 11) y los dos sistemas **actualizados**.
- **Mínimo privilegio:** el gerente no es administrador en ninguno de los dos sistemas. El técnico solo se usa para administrar.
- **Cifrado completo** en los dos sistemas: BitLocker (XTS-AES 128) con contraseña de arranque en Windows, y LUKS2 en Ubuntu.
- **Contraseñas distintas** por usuario, sistema y cifrado, generadas y guardadas en KeePassXC. Nunca se suben al repositorio.
- **Clave de recuperación de BitLocker** solo en KeePassXC. La primera quedó visible en una captura, así que se regenera (`manage-bde -protectors -delete` + `-add -RecoveryPassword`) y la nueva se guarda solo en KeePassXC.
- **UAC al máximo**, inicio de sesión con contraseña (sin inicio automático) y telemetría de Windows desactivada.
- Portapapeles de VirtualBox solo del anfitrión al invitado.
- **Servicios reducidos y cortafuegos** en los dos sistemas: solo quedan los servicios que el gerente usa y no hay ningún servicio accesible desde la red. Detalle en el apartado 11.1.

11.1 Servicios reducidos y cortafuegos (04/11/2026)

Objetivo: que el equipo del gerente no ofrezca nada a la red. El gerente solo navega, imprime y se actualiza, y todo eso es tráfico de **salida**. Por eso en los dos sistemas se desactivan los servicios que no usa y el cortafuegos bloquea todo lo que entra.

Antes de empezar se hizo la instantánea «antes de servicios reducidos», para poder volver atrás.

Ubuntu

Servicio desactivado	Para qué sirve	Por qué se quita
`avahi-daemon` (+ socket, **enmascarado**)	Anuncia el equipo en la red local (mDNS)	Era el único servicio escuchando en la red (`0.0.0.0:5353`)
`cups-browsed`	Busca impresoras de red automáticamente	La impresora la instala el técnico a mano

Servicio desactivado	Para qué sirve	Por qué se quita
`openvpn`	VPN	No se usa
`sssd`	Unirse a un dominio/LDAP	No hay dominio
`bluetooth`	Bluetooth	La VM no tiene bluetooth
`ModemManager`	Módems 3G/4G	No hay módem

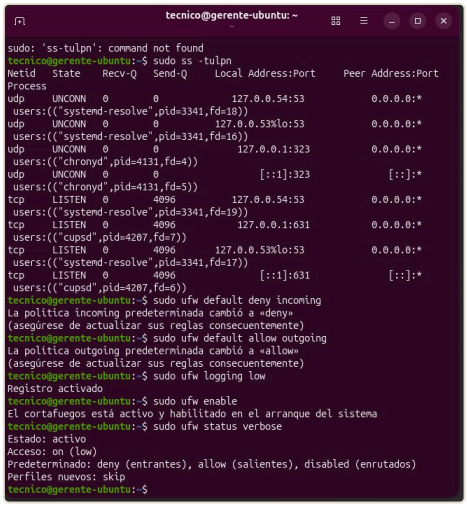
Se mantienen `cups` (imprimir, solo escucha en `127.0.0.1`), `snapd` (Firefox es un snap), `NetworkManager`, `wpa_supplicant` y `unattended-upgrades` (actualizaciones de seguridad automáticas).

```
# Con el usuario tecnico
for s in avahi-daemon.service avahi-daemon.socket cups-browsed.service openvpn.service sssd.service
bluetooth.service ModemManager.service; do sudo systemctl disable --now "$s"; done
sudo systemctl mask avahi-daemon.service avahi-daemon.socket   # que nadie lo vuelva a arrancar

# Cortafuegos: bloquear lo que entra, permitir lo que sale
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw logging low
sudo ufw enable

# Comprobación
systemctl is-enabled avahi-daemon cups-browsed openvpn sssd bluetooth ModemManager   # masked +
disabled
sudo ufw status verbose                                                              # activo, deny
(entrantes)
sudo ss -tulpn                                                                       # solo 127.0.0.x y
::1
```

	Antes	Después
Puertos escuchando hacia la red	1 (`5353/udp`, avahi)	**0** (todo en `127.0.0.x` / `::1`)
Cortafuegos (ufw)	Inactivo	Activo: `deny` entrantes, `allow` salientes, registro `low`
Tras reiniciar	—	Igual: ufw activo, avahi `inactive`, sin puertos hacia la red



Cortafuegos · G16c · GitHub: sistemas/gerente-dualboot/capturas/G16c-ufw-activo.png

Windows 11

Servicio desactivado	Para qué sirve	Por qué se quita
`LanmanServer` (Servidor)	Compartir carpetas (SMB, puerto **445**)	El gerente no comparte nada; es la vía de gusanos como WannaCry

Servicio desactivado	Para qué sirve	Por qué se quita
NetBIOS sobre TCP/IP	Nombres NetBIOS (puerto **139**)	Protocolo antiguo, innecesario
`SSDPSRV`, `upnphost`	Descubrimiento UPnP (puerto **2869**)	No se publican dispositivos en la red
`TermService` + reglas del cortafuegos	**Escritorio remoto**	Nadie debe entrar en remoto al equipo del gerente
`lfsvc`, `MapsBroker`	Ubicación y mapas	No se usan; privacidad
`XblAuthManager`, `XblGameSave`, `XboxNetApiSvc`, `XboxGipSvc`	Xbox	Equipo de trabajo
`RetailDemo`, `WMPNetworkSvc`, `SharedAccess`, `bthserv`	Demo de tienda, compartir multimedia, ICS, bluetooth	No se usan
Optimización de distribución (P2P)	Comparte actualizaciones con otros PC (puerto **7680**)	Se desactiva «Permitir descargas desde otros dispositivos»

`RemoteRegistry` (Registro remoto) ya venía deshabilitado.

```
# PowerShell como administrador (tecnico)
$s =
'lfsvc','MapsBroker','XblAuthManager','XblGameSave','XboxNetApiSvc','XboxGipSvc','RetailDemo','WMPNetw
orkSvc','SharedAccess','TermService','bthserv','LanmanServer','SSDPSRV','upnphost'
foreach ($n in $s) { Stop-Service $n -Force -ErrorAction SilentlyContinue; Set-Service $n -StartupType
Disabled }

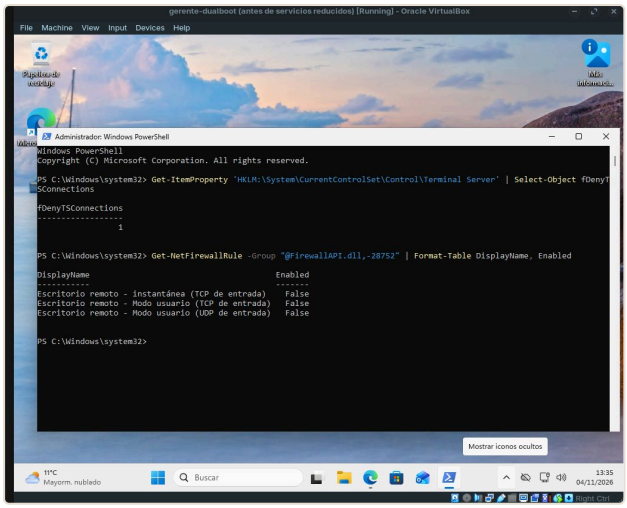
# NetBIOS desactivado en los adaptadores
Get-CimInstance Win32_NetworkAdapterConfiguration -Filter "IPEnabled=true" | Invoke-CimMethod -
MethodName SetTcpipNetbios -Arguments @{TcpipNetbiosOptions=2}

# Cortafuegos explícito y con registro de bloqueos
Set-NetFirewallProfile -All -Enabled True -DefaultInboundAction Block -DefaultOutboundAction Allow -
LogBlocked True

# Escritorio remoto cerrado en el sistema y en el cortafuegos
Set-ItemProperty 'HKLM:\System\CurrentControlSet\Control\Terminal Server' -Name fDenyTSConnections -
Value 1
Disable-NetFirewallRule -Group "@FirewallAPI.dll,-28752"
```

Optimización de distribución: *Configuración → Windows Update → Opciones avanzadas → Optimización de distribución → Permitir descargas desde otros dispositivos: Desactivado*.

	Antes	Después (tras reiniciar)
Servicios en marcha	98	**79**
Puertos abiertos a la red	135, 139, 445, 2869, 5040, 7680, 49664–49674	135, 5040, 49664–49670 (RPC y plataforma de Windows, bloqueados desde fuera por el cortafuegos)
Cortafuegos	Activo, entrada `NotConfigured`	Activo en los 3 perfiles: entrada `Block`, salida `Allow`, registro de bloqueos
Escritorio remoto	Servicio en manual	Deshabilitado y sin reglas en el cortafuegos



Escritorio remoto cerrado · G17f · GitHub: sistemas/gerente-dualboot/capturas/G17f-escritorio-remoto-cerrado.png

Al terminar se hizo la instantánea «servicios reducidos» (04/11/2026, 13:37).

12. Lista de capturas

Código	Archivo	Estado
G01	`G01-hash-iso-ubuntu.png`	✅
G02	`G02-detalles-vm.png`	✅
G03	`G03-particiones-windows.png`	✅
G03a–d	`G03a…G03d-*.png` (disco vacío, cuenta local, KeePassXC)	✅
G04	`G04-escritorio-windows.png`	✅
G04b–c	`G04b-red-conectada.png`, `G04c-guest-additions.png`	✅
G05	`G05-windows-update.png`	✅
G06	`G06-menu-inicio-limpio.png`, `G06b-escritorio-limpio.png`	✅
G07	`G07-inicio-rapido-desactivado.png`	✅
G08	`G08-gerente-estandar-windows.png`	✅
G09	`G09-uac-maximo.png`	✅
G10	`G10-espacio-libre-ubuntu.png`	✅
G10b	`G10b-bitlocker-esperando-activacion.png`	✅
G11a	`G11a-boot-manager-uefi.png`	✅
G11b	`G11b-ubuntu-cifrado-frase.png`	✅
G11c	`G11c-ubuntu-actualizado.png`	✅
G12	`G12-grub-dual-boot.png`	✅
G12b	`G12b-efibootmgr.png`	✅
G13	`G13-gerente-sin-sudo.png`	✅
G14	`G14-particiones-lsblk-fdisk.png`	✅
G15a	`G15a-gpedit-bitlocker.png`	✅
G15b	`G15b-bitlocker-pide-contrasena.png`	✅

Código	Archivo	Estado
G15	`G15-bitlocker-activado.png`	✅
G16a1–a3	`G16a1-servicios-antes.png`, `G16a2-servicios-antes.png`, `G16a3-puertos-ufw-antes.png`	✅
G16b	`G16b-servicios-desactivados.png`	✅
G16c	`G16c-ufw-activo.png`	✅
G16d	`G16d-tras-reinicio.png`	✅
G16e	`G16e-internet-y-actualizaciones.png`	✅
G17a1–a3	`G17a1-windows-cortafuegos-antes.png`, `G17a2-windows-puertos-antes.png`, `G17a3-windows-servicios-antes.png`	✅
G17b	`G17b-windows-servicios-desactivados.png`	✅
G17c	`G17c-optimizacion-distribucion.png`	✅
G17d	`G17d-windows-tras-reinicio.png`	✅
G17e	`G17e-windows-internet-edge.png`	✅
G17f	`G17f-escritorio-remoto-cerrado.png`	✅
INC1–INC12	`INC*-*.png`	✅

> Antes de subir una captura, comprobar que no aparece ninguna contraseña ni la clave de recuperación de BitLocker.

5. TPV de barra e impresora de tickets

Responsable: Iñigo (TPV) y Jon (impresora)

> **Dónde está en GitHub**
>
Resumen del TPV y pruebas	`sistemas/tpv-barra/README.md`
Guía del quiosco paso a paso	`sistemas/tpv-barra/tpv-quiosco.md`
Capturas y vídeo de la impresión	`sistemas/tpv-barra/capturas`

Resumen

TPV (Iñigo). En el aula, el TPV es una VM con Windows 10 Pro; en la compra real llevaría Windows 11 IoT Enterprise LTSC. El cajero entra con la cuenta `tpv`, que es **estándar** y queda encerrada en un **quiosco de Edge** (acceso asignado de Windows): a pantalla completa, sin barra de direcciones ni pestañas, y abriendo solo el punto de venta de Odoo. Odoo va por **HTTPS** con el certificado de nuestra CA instalado como de confianza, así que no sale ningún aviso. En Odoo, el cajero solo tiene permiso de punto de venta: si intenta crear un producto, Odoo se lo deniega. En la prueba de restricción, ninguna de las siete combinaciones probadas (tecla Windows, Alt+Tab, Alt+F4, Ctrl+Shift+Esc, entre otras) saca del quiosco. Se trabajó con instantáneas, porque la primera vez no se pudo salir del quiosco y hubo que rehacer la VM.

Impresora de tickets (Jon). La impresora va por USB a la VM del TPV. Con el controlador USB 3.0 de VirtualBox, Windows daba el error *Código 10*, y se resolvió pasando a **USB 1.1 (OHCI)**. El driver lo instala solo el administrador, nunca el usuario `tpv`, y la impresora queda como **predeterminada**. En Odoo está activada la **impresión automática del recibo**: al validar el pago, el ticket se imprime solo, con el logo de Almi-Chain y el IVA al 10 %. El profesor la validó el 27/10 como impresión y factura automáticas, y está grabada en vídeo. En el aula se usa la Samsung ML-2165 (láser A4), y en la compra real dos Epson TM-T20III térmicas de 80 mm, que se configuran igual.

TPV de barra e impresora de tickets

> **En GitHub:** `sistemas/tpv-barra/README.md`

Equipo de barra que solo sirve para cobrar: **Windows 10 Pro** con un usuario estándar, encerrado en un **quiosco de Edge** que abre el punto de venta de Odoo por HTTPS, con la impresora de tickets por USB.

Responsable: Iñigo (TPV) · Jon (impresora).

La guía completa, paso a paso, está en `tpv-quiosco.md`: máquina virtual, cuentas, certificado de la CA, modo quiosco, usuario cajero de Odoo, impresora, pruebas de restricción, riesgos y resumen para un técnico nuevo.

Equipo del reto frente a la compra real

	En el reto (VM del aula)	En la compra real (presupuesto)
TPV	**Una** VM con **Windows 10 Pro**; el segundo TPV sería un clon	2 TPV todo en uno 10POS con **Windows 11 IoT Enterprise LTSC**
Impresora	**Samsung ML-2165** (láser A4, la disponible en el aula)	2 **Epson TM-T20III** térmicas de 80 mm

El usuario estándar, el quiosco, el certificado y la impresión se configuran igual en las dos versiones de Windows y con cualquier impresora instalada como predeterminada. El hardware y el software de la compra real, con precios, están en el presupuesto.

Mínimo privilegio

Medida	Cómo	Detalle
Usuario de Windows `tpv`	Cuenta local **estándar**, sin administrador	`tpv-quiosco.md` §3
Una sola aplicación	Acceso asignado (quiosco) con Edge a pantalla completa, sin barra de direcciones ni pestañas	§5
Usuario de Odoo cajero	Solo *Punto de venta · Usuario*: no puede crear productos ni ver facturación	§6
Drivers	Solo los instala el administrador, nunca `tpv`	§7
Comunicaciones cifradas	Odoo por HTTPS con el certificado de nuestra CA instalado como de confianza	§4
Red	Cable a la red de empleados, nunca al Wi-Fi de clientes	`../../redes/`

Impresora de tickets

1. **USB a la VM:** con la VM apagada, *Configuración → USB*, controlador **USB 1.1 (OHCI)** y un filtro con la impresora. Con USB 3.0 Windows daba *Código 10* (ver §7).

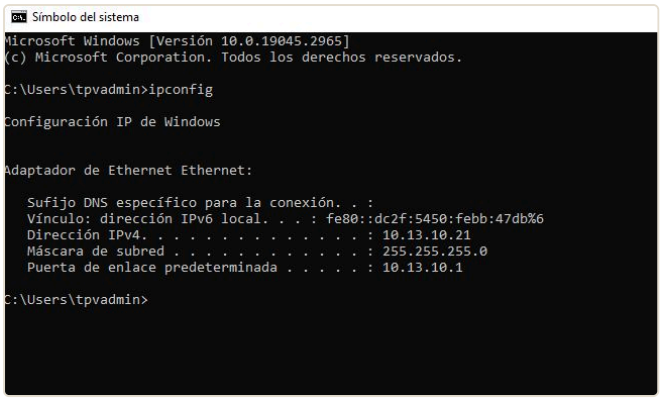
2. **Driver y predeterminada:** con la cuenta de administrador se instala el driver y se pone como **predeterminada**, desactivando que Windows la gestione. Papel: A4 con la ML-2165 del aula; 80 mm con la Epson térmica.
3. **Odoo:** *Punto de venta → Configuración → Ajustes → Facturas y recibos* → **Impresión automática del recibo**. Al validar el pago, Odoo lanza la impresión solo, sin pulsar «Imprimir recibo». ✅ **Validado por el profesor el 27/10 como impresión automática.**
4. **Mejora opcional, sin la ventana de imprimir:** Odoo Community sin *IoT Box* imprime a través del navegador, y el navegador muestra su diálogo. Edge lo omite si se abre con `--kiosk-printing`, que envía el ticket directo a la impresora predeterminada:

   ```
   "C:\Program Files (x86)\Microsoft\Edge\Application\msedge.exe" --kiosk
   https://192.168.1.50/pos/ui/2 --edge-kiosk-type=fullscreen --kiosk-printing
   ```

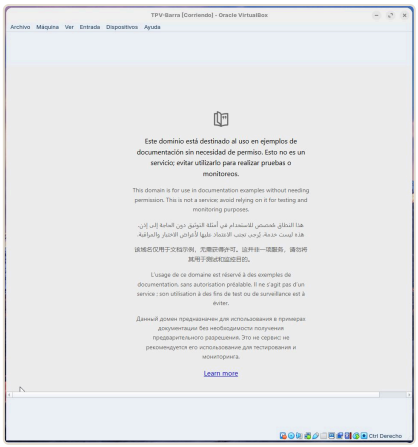
   Ojo: con el *Acceso asignado* de Windows, es Windows quien arranca Edge con sus propios parámetros, así que este acceso directo no sirve tal cual. Hay que probarlo primero fuera del quiosco y después buscar cómo pasarle el parámetro dentro.
5. **Ticket de prueba de la impresora:** `impresion_prueba_almichain_80mm.pdf`, con tildes, ñ, € y código de barras. Se imprime a tamaño real (100 %).

Pruebas

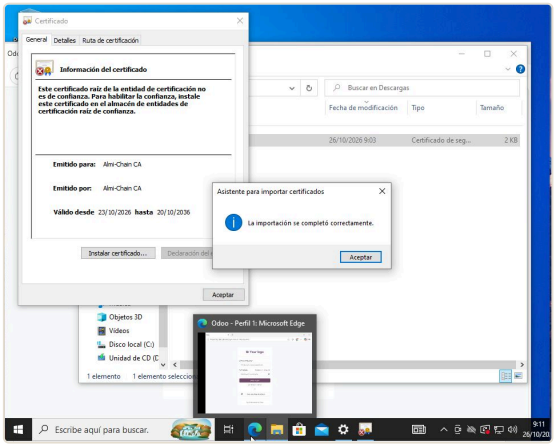
#	Prueba	Resultado esperado	Estado	Evidencia
1	IP fija del TPV	`10.13.10.21` en la red de empleados	✅	captura
2	Entrar con `tpv`	Edge se abre solo en el punto de venta, a pantalla completa	✅	captura · plano de mesas
3	Odoo por HTTPS desde el TPV	Sin aviso de certificado	✅	captura
4	Pruebas de restricción (tecla Windows, Alt+Tab, Alt+F4, Ctrl+Shift+Esc…)	No se puede salir del quiosco	✅ 06/11 · ninguna de las 7 combinaciones saca del quiosco	§8
5	Impresora reconocida en la VM	Aparece en el Administrador de dispositivos	✅	captura
6	**Venta en el TPV → ticket impreso**	El ticket sale por la impresora	✅ 27/10, con el diálogo de imprimir de por medio	vídeo · diálogo
7	Ticket impreso legible	En el papel se leen los caracteres especiales y el código	✅ 27/10 · en el ticket impreso del vídeo se leen el **€** («1,82 €»), los signos **¿?** («¿Necesita una factura?») y el **código QR**. El PDF de prueba con ñ y código de barras queda como comprobación extra para la Epson térmica	vídeo (final)
8	**Venta → impresión automática**	Al validar el pago, Odoo lanza la impresión del ticket solo	✅ 27/10 · validado por el profesor	vídeo



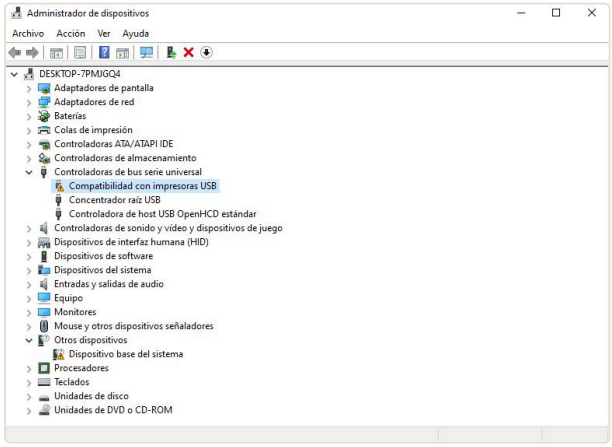
IP fija del TPV · GitHub: sistemas/tpv-barra/capturas/03-ip-fija.png



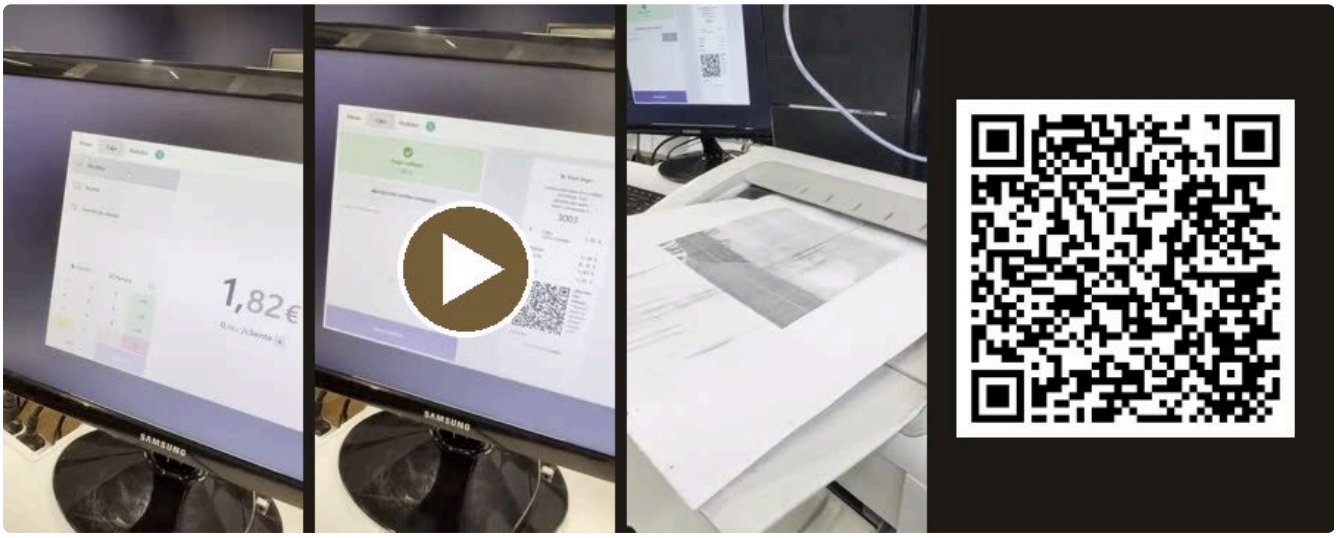
Entrar con tpv · GitHub: sistemas/tpv-barra/capturas/07-quiosco-edge.png



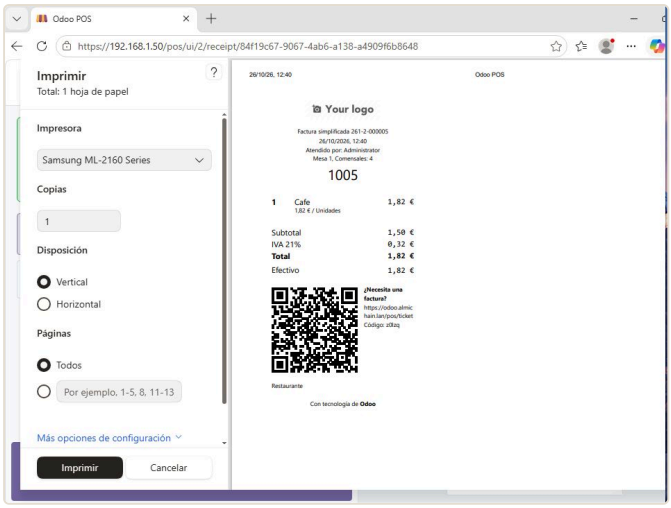
Odoo por HTTPS desde el TPV · GitHub: sistemas/tpv-barra/capturas/05-certificado-confianza.png



Impresora reconocida en la VM · GitHub: sistemas/tpv-barra/capturas/09c-administrador-dispositivos.png



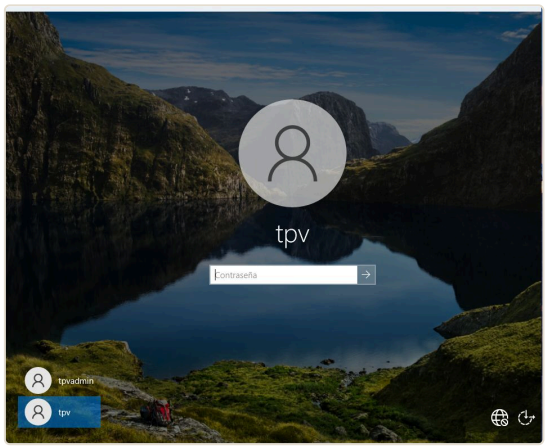
▶ **Vídeo:** Venta en el TPV → ticket impreso · verlo en GitHub (o con el QR): sistemas/tpv-barra/capturas/imp_01_venta_e_impresion.mp4



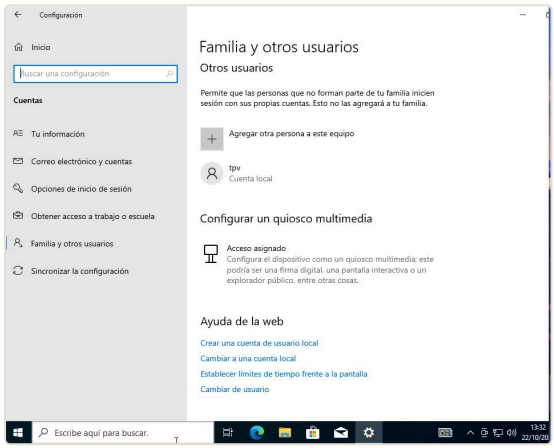
Venta en el TPV → ticket impreso · GitHub: sistemas/tpv-barra/capturas/21-odoo-prueba-ticket.png

Capturas

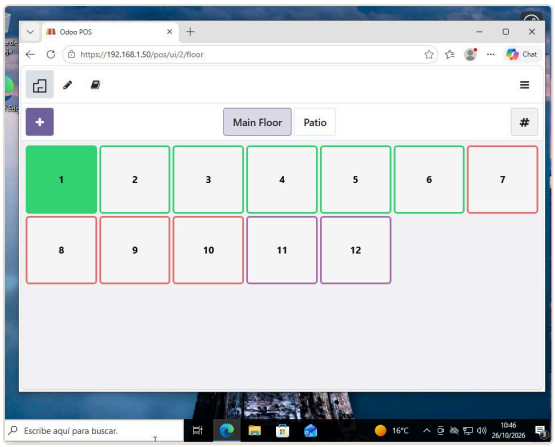
Captura	Qué muestra
01 · 02 · 03	Configuración de la VM en VirtualBox
02-windows-10-pro	Edición de Windows
03-ip-fija	`ipconfig` con la IP del plano
04-usuario-estandar	Cuenta `tpv` estándar
05-certificado-confianza · 13-certificado-detalles	CA instalada como de confianza
06-acceso-asignado(quiosco)	Configuración del acceso asignado
06-acceso-asignado · 07-quiosco-edge	El quiosco abierto en el punto de venta
09-quiosco-navega-enlaces	Prueba inicial: el quiosco seguía enlaces externos (ver riesgos en §9)
09c-administrador-dispositivos	Impresora USB en el Administrador de dispositivos
16-odoo-restaurante-activado	Módulo Restaurante instalado en Odoo
18-odoo-usuario-cajero	Usuario cajero con permisos solo de punto de venta
19-odoo-tpv-cajero · 20-odoo-prueba-pedido	Plano de mesas y pedido de prueba
21-odoo-prueba-ticket	Diálogo de impresión del ticket
imp_01_venta_e_impresion	Vídeo: venta, pago, diálogo de impresión y ticket impreso



Cuenta tpv estándar · GitHub: sistemas/tpv-barra/capturas/04-usuario-estandar.png



Configuración del acceso asignado · GitHub: sistemas/tpv-barra/capturas/06-acceso-asignado(quiosco).png



Plano de mesas y pedido de prueba · GitHub: sistemas/tpv-barra/capturas/19-odoo-tpv-cajero.png

TPV paso a paso: Windows 10 Pro en modo quiosco

> **En GitHub:** `sistemas/tpv-barra/tpv-quiosco.md`

Responsable: Iñigo. Estado: funcionando contra el Odoo del servidor. En el aula, el TPV y el servidor están en la red del aula (`192.168.1.x`) y no en la red de empleados del plano (`10.13.10.x`); es una desviación justificada, ver §9.

1. Objetivo

Un equipo de barra que solo sirva para cobrar: Windows 10 Pro con un usuario estándar, encerrado en un quiosco de Edge que abre el punto de venta de Odoo por HTTPS, con una impresora para los tickets. Todo con el mínimo privilegio posible.

2. Máquina virtual

Elemento	Valor
Hipervisor	VirtualBox 7.2.16
Nombre	TPV Barra
Sistema	Windows 10 Pro 22H2 (64 bits), instalado con una cuenta local
RAM / CPU	4096 MB / 2 núcleos
Disco	50 GB, reservado dinámicamente
Red	Adaptador puente sobre la tarjeta por cable del anfitrión
USB	Controlador USB 1.1 (OHCI), ver apartado de la impresora

Al crear la VM se marcó “Skip Unattended Installation” para que VirtualBox no creara un administrador con una contraseña elegida por él.

Después de instalar Windows se instalaron los Guest Additions, se activó el portapapeles compartido y se aplicaron las actualizaciones.

3. Cuentas

Cuenta	Tipo	Para qué
TPV-admin	Administrador	Instalar y configurar. No se usa a diario

Cuenta	Tipo	Para qué
tpv	Estándar	El que queda encerrado en el quiosco

Cada cuenta tiene una contraseña distinta, generada en KeePassXC. Ninguna se sube al repositorio.

El usuario `tpv` es una cuenta local estándar: para instalar programas Windows le pide la contraseña de un administrador, que no tiene.

4. Certificado de la CA

El servidor usa un certificado firmado por una entidad propia, **Almi-Chain CA** (RSA 4096, válida del 23/10/2026 al 20/10/2036). El certificado del servidor lleva la IP 192.168.1.50 en el campo "Nombre alternativo del firmante", que es lo que Edge comprueba. Se instala la CA y no el certificado del servidor: así, si este se renueva, no hay que repetirlo en el TPV.

Pasos:

1. Pedir a quien administra el servidor solo el certificado público de la CA, un archivo `.crt`. Nunca la clave privada (`.key`).
2. Abrirlo y comprobar en Detalles que el Asunto es `Almi-Chain CA`, que la fecha de validez es futura y que lleva la restricción "Tipo de asunto = Entidad de certificación".
3. Comparar la huella digital con la que obtiene el administrador en el servidor. Windows muestra la SHA-1, así que se usa `openssl x509 -in ca.crt -noout -fingerprint -sha1`. ✅ La huella coincide.
4. Con el administrador: Instalar certificado, Equipo local, y colocarlo en "Entidades de certificación raíz de confianza".
5. Cerrar Edge y abrir `https://192.168.1.50`: el aviso desaparece y sale el candado.

Antes y después:

5. Modo quiosco

Se configura con el administrador en Configuración, Cuentas, Familia y otros usuarios, "Configurar un quiosco" (acceso asignado):

1. Cuenta existente: `tpv`.
2. Aplicación: Microsoft Edge.
3. Modo: firma digital o pantalla interactiva (pantalla completa, sin barra de direcciones ni pestañas).
4. URL: `https://192.168.1.50/pos/ui/2`. Es la IP del certificado, para que no aparezca ningún aviso.
5. Se pulsa Guardar, se cierra sesión y se entra con `tpv`: Edge se abre solo.

Lección aprendida: la primera vez que se montó el quiosco no se pudo salir de él y hubo que rehacer la VM. Por eso se trabaja con instantáneas (`Windows limpio`, `Antes del quiosco`, `Antes de impresora`) y la prueba de salida se hace nada más configurarlo. Para salir como administrador se usa Máquina, Insertar Ctrl-Alt-Supr (o Ctrl derecho + Supr), y Cerrar sesión.

6. Odoo: usuario del cajero

El Odoo es compartido por todo el equipo. Para el TPV se creó un usuario de Odoo con permisos solo de **Punto de venta, nivel Usuario**, y sin ningún otro permiso. Su contraseña está en KeePassXC.

Para comprobarlo, el cajero intentó crear un producto y Odoo lo denegó (error de acceso por falta del grupo "Productos/Crear"). Esa es la evidencia de que, si alguien tomara el control del quiosco, solo podría cobrar.

Los productos los crea un administrador de Odoo. Edge no debe guardar la contraseña del cajero.

7. Impresora

Modelo: Samsung ML-2165. Es una **láser A4 monocromo**, no una térmica de tickets, pero es la única disponible, así que se usa como sustituta.

Pasos:

1. En el anfitrión, el usuario está en el grupo `vboxusers`.
2. Con la VM apagada se añade la impresora como filtro en Configuración, USB.
3. Con el administrador se comprueba en el Administrador de dispositivos y se instala el driver (los controladores se instalan solo con el administrador, nunca con `tpv`).
4. Se pone como impresora predeterminada, desactivando que Windows la gestione, y se imprime una página de prueba.

Problema encontrado: con el controlador USB 3.0 (xHCI) el dispositivo "Compatibilidad con impresoras USB" daba el error "Código 10: este dispositivo no puede iniciar". **Solución:** cambiar el controlador de la VM a USB 1.1 (OHCI), que no necesita el Extension Pack y basta para una impresora. ✅ Con ese cambio quedó funcionando: la impresora aparece sin errores en el Administrador de dispositivos (captura) e imprime los tickets desde Odoo (vídeo).

Impresión desde el TPV (27/10): al validar el pago, Odoo abre el **diálogo de impresión** de Edge con la Samsung ML-2160 Series seleccionada, y al pulsar *Imprimir* sale el ticket (vídeo). El profesor lo validó el 27/10 como impresión automática. Quitar también el diálogo queda como mejora opcional en el README.

8. Pruebas de restricción

Se hacen con el usuario `tpv` dentro del quiosco.

Prueba	Esperado	Resultado
Tecla Windows	No aparece el menú Inicio	✅ No aparece el menú Inicio
Alt + Tab	No hay otra aplicación	✅ No hay otra aplicación
Alt + F4	No cierra, o se reabre solo	✅ No cierra
Ctrl + Shift + Esc	No abre el administrador de tareas	✅ No lo abre
Ctrl + L / Ctrl + T	No hay barra ni pestañas	✅ No hay barra ni pestañas
Instalar un programa	Pide credenciales de administrador	✅ No deja instalar nada
Cambiar de usuario o cerrar sesión desde dentro	Solo con Ctrl+Alt+Supr	✅ Solo con Ctrl+Alt+Supr

9. Riesgos y desviaciones

- **Windows 10 sin soporte de seguridad** desde octubre de 2025. Se reduce con usuario estándar, quiosco y una única aplicación.
- **El quiosco dejó seguir enlaces** en una prueba inicial con una página externa. Con Odoo no hay enlaces externos, pero se podría cerrar con una lista de direcciones permitidas en Edge. Queda como mejora.
- **Red:** el plano prevé servidor en 10.13.10.10 y TPV en 10.13.10.21. Ahora mismo ambos están en la red del aula (servidor 192.168.1.50), y el certificado lleva esa IP. Cuando todo cuelgue del Router 1 habrá que cambiar las IPs y regenerar el certificado del servidor.
- **Impresora:** láser A4 en lugar de térmica de tickets.
- **Inicio de sesión automático** del quiosco: es aceptable porque `tpv` es estándar y está limitado a una sola aplicación.
- **Clave de la CA:** quien la tenga puede fabricar certificados que este TPV aceptaría. Se queda en el servidor y nunca se copia.

10. Cómo repetirlo (resumen para un técnico nuevo)

1. Crear la VM (4 GB, 2 CPU, 50 GB, adaptador puente, USB 1.1) y instalar Windows 10 Pro con cuenta local. Instalar Guest Additions y actualizar. Instantánea.
2. Crear el usuario `tpv` como estándar. Instantánea.
3. Instalar el certificado de la CA en las entidades raíz de confianza, tras comprobar su huella.
4. Configurar el quiosco con Edge y la URL del punto de venta. **Probar la salida enseguida.**
5. Crear en Odoo el usuario cajero con permisos solo de punto de venta.
6. Instalar la impresora con el administrador y dejarla como predeterminada.
7. Hacer las pruebas de restricción y guardar las capturas.

6. Servidor, ERP y TPV (Odoo)

Responsable: Sendoa (servidor), Alejandro (configuración de Odoo) e Iñigo y Yeray (factura automática)

Dónde está en GitHub

Guía del servidor y de Odoo	`erp/README.md`
Scripts de instalación	`erp/scripts`
Docker Compose y Nginx	`erp/docker`
CSV de ejemplo	`erp/datos`
Capturas	`erp/capturas`

Resumen

Configuración de Odoo (Alejandro). Usamos **Odoo 19 Community** con seis ámbitos: Facturación, CRM, Empleados, Marketing, Inventario y TPV. La carta, los ingredientes, los clientes y los empleados se cargaron desde ficheros CSV con **ID externo**, así que volver a importar actualiza los registros en lugar de duplicarlos, y antes de importar se pulsa «Probar», que avisa de los errores. También se exportaron datos a CSV y XLSX. Hay tres usuarios por rol: **Gerente Almi** (facturación, marketing y empleados) y **Barra 1** y **Barra 2**, que solo pueden usar el TPV (sin Ventas, Contabilidad ni Contactos, y sin poder exportar), para cumplir el mínimo privilegio. El CRM tiene un flujo de cuatro etapas (Nuevo, Calificado, Propuesta y Ganado) con cuatro oportunidades de ejemplo, y en Marketing hay una lista de correo de clientes y una campaña en borrador, sin enviar. Las contraseñas se asignan a mano y no están en el repositorio. Las capturas de cada paso están en `erp/capturas/`.

Servidor (Sendoa). Es una VM con Ubuntu Server 24.04 LTS que ejecuta con **Docker Compose** tres contenedores: Odoo 19, PostgreSQL 16 y Nginx. Solo **Nginx** se publica (puertos 80 y 443): redirige de HTTP a HTTPS y usa el certificado de nuestra CA. Odoo y PostgreSQL no tienen ningún puerto abierto, y se comprobó con `nmap`. El cortafuegos (UFW) solo deja pasar 22, 80 y 443 desde la red local; SSH solo admite claves, y el gestor de bases de datos de Odoo está desactivado. Los datos están en un RAID 1 y se copian cada noche (apartado 12). Se instala con cinco scripts numerados, así que se puede repetir desde cero.

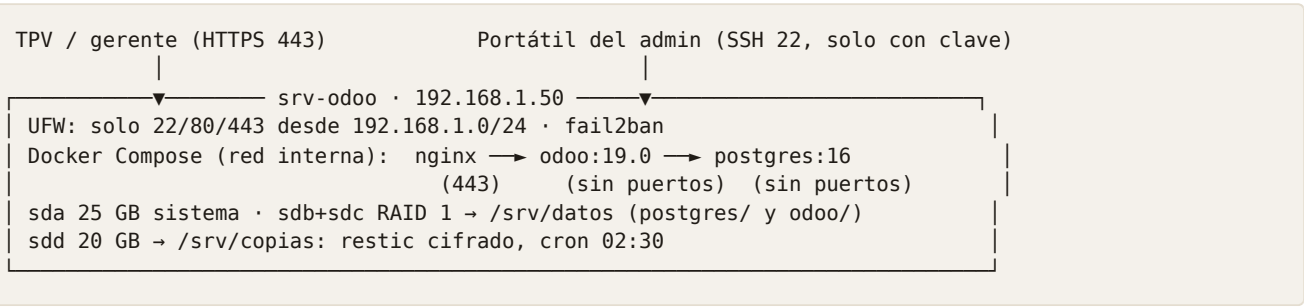
Factura automática (Iñigo y Yeray). Al validar el pago en el TPV, Odoo imprime el ticket solo, con el logo de Almi-Chain y el IVA al 10 %. El profesor lo validó como factura automática. Como mejora, queda documentado el código de una regla de Odoo para emitir también la factura con número.

Servidor Odoo (ERP + TPV)

> **En GitHub:** `erp/README.md`

VM con **Ubuntu Server 24.04 LTS** que ejecuta con **Docker Compose** tres contenedores: **Odoo 19 Community**, **PostgreSQL 16** y **Nginx con HTTPS**. Los datos están en un **RAID 1** y hay **copias cifradas diarias** (restic + cron) en un tercer disco. Cada venta del TPV emite su factura sola.

La guía paso a paso, desde crear la VM hasta las pruebas, está en el documento «Guía · Servidor Odoo Almi-Chain». Este README es el resumen técnico.



Archivo	Qué es
`scripts/red.conf`	**IP del servidor y red** (en el aula: `192.168.1.50`, `192.168.1.0/24`). Único sitio donde se cambia
`scripts/01-base.sh`	Actualizaciones automáticas, SSH solo con clave, UFW y fail2ban
`scripts/02-raid.sh`	RAID 1 (discos A y B) en `/srv/datos` para los volúmenes
`scripts/03-certificados.sh`	CA propia «Almi-Chain CA» y certificado de `odoo.almichain.lan`
`scripts/04-odoo-docker.sh`	Docker, secretos, `odoo.conf`, BD `almichain` con los módulos y arranque
`scripts/05-copias.sh`	Disco C, restic y copia diaria con cron
`scripts/emitir_factura.py`	Extra: emitir facturas por la API JSON-2 de Odoo
`docker/`	`docker-compose.yml`, Nginx, plantilla de `odoo.conf`, usuarios de PostgreSQL (`initdb/`) y configuración inicial
`datos/`	CSV de ejemplo para importar: carta (`productos.csv`) y clientes (`clientes.csv`)
`../backups/`	Copia, prueba de restauración y recuperación

Instalación (resumen)

1. **VM:** 2 CPU, 4 GB, adaptador puente a la red de clase. Discos SATA: 25 GB (sistema), 10 + 10 GB (A y B, RAID), 20 GB (C, copias).
2. **Ubuntu Server 24.04:** IP fija `192.168.1.50/24`, puerta de enlace `192.168.1.1`, DNS `192.168.1.1,1.1.1.1`. LVM solo en el disco de 25 GB, ampliado al máximo. Usuario `adminsrv` y OpenSSH.
3. **Desde el portátil, ANTES de `01-base.sh`:** crea la clave (`ssh-keygen -t ed25519`, con frase de paso guardada en KeePassXC), cópiala con `ssh-copy-id adminsrv@192.168.1.50` y comprueba que entras sin contraseña con `ssh -o PasswordAuthentication=no adminsrv@192.168.1.50`. Si `01-base.sh` no encuentra la clave, deja la contraseña activada (ver incidencia 1).
4. **En el servidor:**

```
git clone https://github.com/JRevii/almi-chain.git && cd almi-chain/erp/scripts
sudo bash 01-base.sh
sudo bash 02-raid.sh           # pide SI (borra sdb y sdc)
sudo bash 03-certificados.sh
sudo bash 04-odoo-docker.sh    # 10-15 min
sudo bash 05-copias.sh         # pide SI (borra sdd)
```

5. **Odoo:**
   - Configura la empresa y el IVA del 10 % por defecto.
   - Crea los usuarios por rol (Gerente, Barra 1 y Barra 2, que solo pueden usar el TPV) y los dos puntos de venta.
   - Importa `datos/*.csv`.
   - Crea la regla de automatización *TPV: factura automática de cada venta* (modelo «Pedido del punto de venta», desencadenante «Al crear», acción «Ejecutar código»):

```
cliente_contado = env['res.partner'].search([('name', '=', 'Cliente de contado')], limit=1)
for pedido in records:
    valores = {'to_invoice': True}
    if not pedido.partner_id:
        valores['partner_id'] = cliente_contado.id
    pedido.write(valores)
```

6. **En cada equipo cliente:**
   - Añade `192.168.1.50 odoo.almichain.lan` al fichero `hosts` (Windows: `C:\Windows\System32\drivers\etc\hosts` como administrador; Linux: `echo "192.168.1.50 odoo.almichain.lan" | sudo tee -a /etc/hosts`). Sin esto, el navegador da `DNS_PROBE_FINISHED_NXDOMAIN`.
   - Copia el certificado **público** de la CA (`scp adminsrv@192.168.1.50:almichain-ca.crt .`; nunca la `.key`) e instálalo como entidad raíz de confianza. Windows: doble clic → *Instalar certificado* → *Entidades de certificación raíz de confianza*. Chrome en Linux no usa el almacén del sistema: `chrome://certificate-manager` → *Local certificates* → *Installed by you* → *Trusted certificates* → *Import*. Sin esto sale `NET::ERR_CERT_AUTHORITY_INVALID`.
   - Abre `https://odoo.almichain.lan`.

7. **BD del local (`almichain_local`):** las tablas de Alejandro están en `../bd/scripts/Scripts_postgre/02_tablas.sql`. En este servidor se cargan en `almichain_local` con su dueño `local_app`, **nunca** en `almichain`, que es la BD de Odoo:

```
sudo docker compose -f /opt/almichain/docker-compose.yml exec -T db \
  psql -v ON_ERROR_STOP=1 -U local_app -d almichain_local < ~/almi-
chain/bd/scripts/Scripts_postgre/02_tablas.sql
sudo docker compose -f /opt/almichain/docker-compose.yml exec db psql -U local_app -d almichain_local
-c '\dt'
```

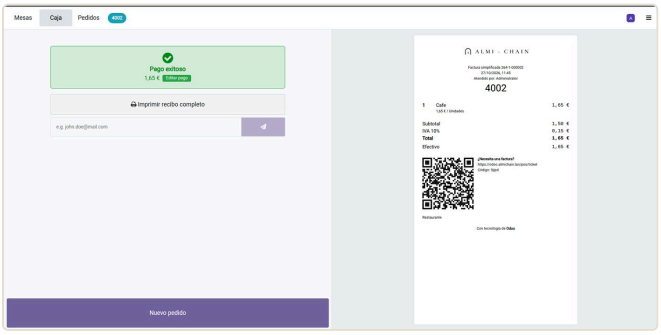
> Los comentarios de `02_tablas.sql` y `03_permisos.sql` usan la BD `almichain` y los roles `almichain_owner` / `almichain_app`. En este servidor esos nombres no aplican: `03_permisos.sql` **no se ejecuta** tal cual, porque daría acceso a la BD de Odoo. Pendiente de acordar con Alejandro.

Configuración de Odoo (26/10/2026)

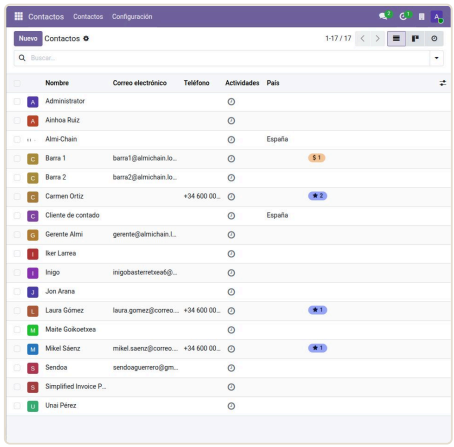
Configuración funcional hecha desde la interfaz web, con el usuario administrador.

Paso	Qué se ha hecho	Estado	Evidencia
Impuestos	Activos solo los necesarios: **IVA 10% (Bienes) · Ventas** (hostelería), IVA 21% (Bienes) · Ventas (general) y 10% IVA (bienes corrientes) · Compras. Los demás impuestos del plan español quedan desactivados para que nadie elija uno equivocado	✅	captura
Productos de prueba	*Cafe*: 1,50 € + IVA 10% (Bienes) = **1,65 €**. Antes llevaba el 21 % (1,82 €) y se corrigió el 27/10: en hostelería es el 10 %. La carta completa se importa después desde `datos/productos.csv`	✅ 27/10	ticket
Empresa y ticket	Empresa **Almi-Chain** con su logo (versión en negro para imprimir), que sale en la cabecera del ticket en lugar de «Your logo». **Impresión automática del recibo** activada (validada por el profesor, ver TPV)	✅ 27/10	ticket
Autofacturación por QR	**Se deja activada:** el ticket lleva el QR «¿Necesita una factura?» con un código, por si un cliente quiere factura completa con sus datos. El enlace va a `odoo.almichain.lan`, así que solo funciona desde la red del local	✅	ticket
Factura automática de cada venta	Al validar el pago en el TPV, Odoo genera e imprime el ticket solo (impresión automática del recibo). **El profesor lo validó como factura automática.** La regla *TPV: factura automática de cada venta* del apartado de instalación queda como mejora opcional, para publicar además la factura con número	✅ validado	vídeo · ticket
Carta, clientes y empleados	Importados desde CSV (platos, ingredientes, clientes y empleados) con **ID externo**, así que volver a importar actualiza los registros y no crea duplicados. En «Probar» sale «Todo parece válido» antes de importar	✅ 06/11	prueba · resultado

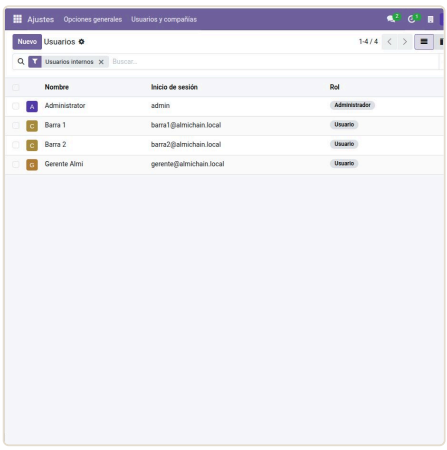
Paso	Qué se ha hecho	Estado	Evidencia
Usuarios por rol	**Gerente Almi** (facturación, marketing y empleados), **Barra 1** y **Barra 2**. Barra 1 y 2 solo usan el TPV: *Punto de venta: Usuario*; Ventas, Contabilidad, Contactos y Exportar en *No*; productos solo lectura. Barra 2 se creó duplicando Barra 1. Las contraseñas se asignan a mano y no están en el repositorio	✅ 06/11	usuarios · permisos de Barra 1
CRM	Flujo con 4 etapas (Nuevo, Calificado, Propuesta y Ganado) y 4 oportunidades de ejemplo con clientes de la BD: catering de empresa, menú de grupo, cena de empresa en Almi Casco Viejo y reserva de evento en Almi Centro (esta, ganada)	✅ 06/11	captura
Marketing	Lista de correo **Clientes Almi-Chain** (3 contactos) y campaña *Menú de otoño en Almi-Chain* asignada a esa lista. Queda en **borrador**, sin enviar	✅ 06/11	captura



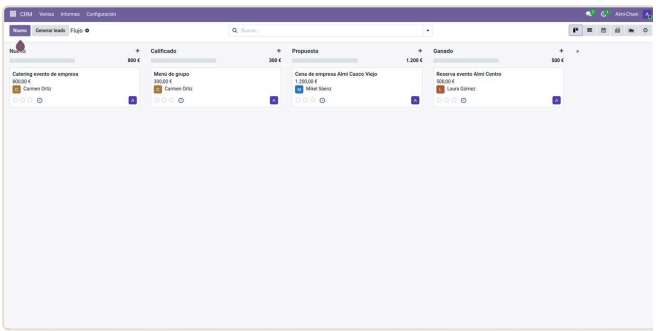
Productos de prueba · GitHub: erp/capturas/odoo_03_ticket_logo_iva10.png



Carta, clientes y empleados · GitHub: erp/capturas/odoo_06b_importacion_resultado.png



Usuarios por rol · GitHub: erp/capturas/odoo_07_usuarios.png



Flujo con 4 etapas (Nuevo, Calificado, Propuesta y Ganado) y 4 oportunidades de ejemplo co... · GitHub: erp/capturas/odoo_04_crm.png

> **Por qué IVA 10% (Bienes):** la localización española solo trae activo el 10 % de ventas como «Bienes». Servir comida es técnicamente una prestación de servicios, también al 10 %. El porcentaje y el cálculo son los correctos.

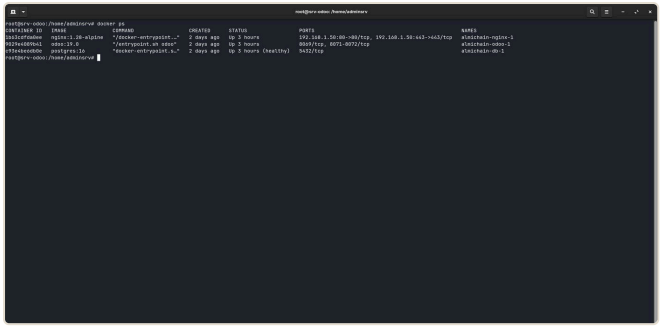
Seguridad aplicada

Capa	Medida
Sistema	Actualizaciones de seguridad automáticas. Administración con `adminsrv` y `sudo`. `adminsrv` no está en el grupo `docker`, que equivale a ser root
SSH	Solo con clave, sin root, solo `adminsrv` y 3 intentos. fail2ban banea 1 h
Cortafuegos	UFW: entrada cerrada salvo 22, 80 y 443 desde la red local. Docker se salta UFW, así que Nginx se publica **solo en la IP de la LAN**, y Odoo y PostgreSQL no se publican
HTTPS	TLS 1.2/1.3 con CA propia. HTTP → HTTPS, HSTS, cookie `secure` y `SameSite=Lax`, `X-Frame-Options`, `X-Content-Type-Options`, `Referrer-Policy`. Sin versión de Nginx
Odoo	Una sola BD (`dbfilter`), gestor de BD desactivado (`list_db = False`), contraseña maestra cifrada, `proxy_mode`. Usuarios por rol
PostgreSQL	Tres usuarios: `postgres` (superusuario, solo administración), `odoo` (crea su BD, no es superusuario) y `local_app` (solo `almichain_local`). `REVOKE CONNECT` en la BD de Odoo
Secretos	Generados al azar en `/root/secretos` (solo root) y en KeePassXC. Cada contenedor lee solo el suyo (Docker secrets). `odoo.conf` y `.env` reales fuera de Git
Datos	RAID 1. Docker no arranca si el RAID no está montado
Copias	restic cifrado (AES-256) en otro disco. Retención 7/4/6 y `restic check` diario

Pruebas

#	Prueba	Resultado esperado	Evidencia
1	`sudo docker compose ps` (en `/opt/almichain`)	`db` healthy, `odoo` y `nginx` running	✅ 26/10 · captura. Nginx publicado solo en `192.168.1.50`; Odoo y PostgreSQL sin publicar
2	`sudo ss -tlnp`	Solo 22, y 80/443 en `192.168.1.50`. Ni 5432 ni 8069	✅ 26/10 · captura
3	`sudo ufw status verbose`	`deny (incoming)`. 22/80/443 solo desde `192.168.1.0/24`	✅ 26/10 · captura
3b	Desde otro equipo de la LAN: `nmap -sV 192.168.1.50`	Solo 22 (OpenSSH), 80 (Nginx) y 443 abiertos; el resto filtrados	✅ 26/10: 997 puertos filtrados. PostgreSQL (5432) y Odoo (8069) no aparecen · captura
4	Navegador → `https://odoo.almichain.lan`	Candado sin avisos	✅ 26/10 · «La conexión es segura» · captura

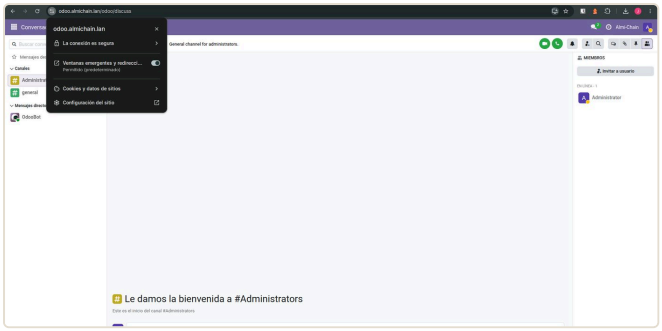
#	Prueba	Resultado esperado	Evidencia
5	`curl -I http://…` y `curl -I https://…/web/login`	`301`. Cabeceras de seguridad	✅ 26/10 · HTTP → `301` a HTTPS y `Server: nginx` sin versión (captura); HTTPS con HSTS, `X-Content-Type-Options`, `X-Frame-Options`, `Referrer-Policy` y cookie `Secure; HttpOnly; SameSite=Lax` (captura, cookie difuminada). Ver incidencia 2
6	`/web/database/manager`	«The database manager has been disabled»	✅ 26/10 · desde un PC de la LAN, sin avisos de certificado · captura
7	`ssh root@…` y `ssh -o PubkeyAuthentication=no adminsrv@…`	`Permission denied`	✅ 26/10 (repetida tras corregir la incidencia 1) · login con clave y contraseña prohibida (captura); `sshd -T`: `permitrootlogin no`, `passwordauthentication no`, `allowusers adminsrv` (captura); root y contraseña → `Permission denied (publickey)` sin pedir contraseña (captura)
8	`psql -U postgres -c '\du'` y `psql -U local_app -d almichain`	`odoo` solo «Create DB». `local_app` sin acceso	✅ 26/10 · `odoo` solo «Create DB», `local_app` sin atributos y `permission denied for database "almichain"` · captura
9	**Venta en TPV → factura automática**	Al validar el pago sale el ticket impreso, sin pulsar «Imprimir», con el logo y el IVA 10 %	✅ validado por el profesor como factura automática · vídeo de la venta y la impresión · ticket
10	Importar CSV y exportar a XLSX	Sin errores	✅ 06/11 · Exportación de Odoo hecha el 29/10: platos.csv, clientes.csv y platos.xlsx. Importación de `clientes.csv` sin errores: «Todo parece válido» (captura) y resultado sin duplicados (captura)
11-14	RAID, fallo de un disco, copia y restauración	11 y 12 ✅ 26/10 · 14 ✅ 27/10 · 13 ✅ 03/11 (ver incidencia 5) · ver `../backups/`	
15	`sudo fail2ban-client status sshd`	Jaula activa	✅ 26/10 · captura



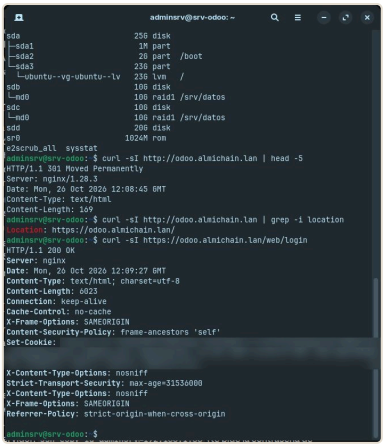
sudo docker compose ps (en /opt/almichain) · GitHub: erp/capturas/srv_01_docker_ps.png



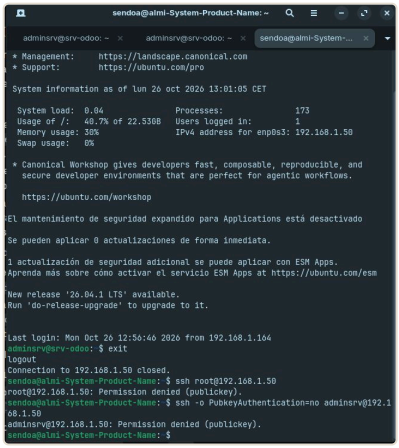
sudo ufw status verbose · GitHub: erp/capturas/srv_03_ufw.png



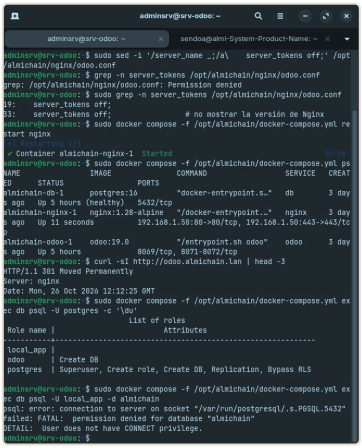
Navegador → https://odoo.almichain.lan · GitHub: erp/capturas/srv_04_candado.png



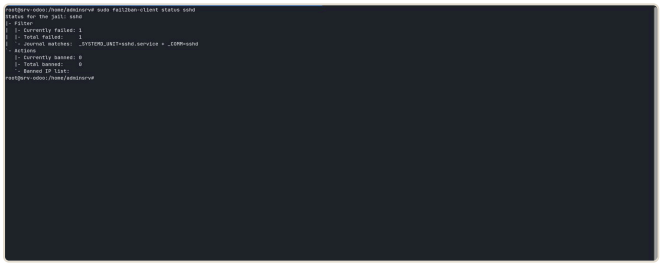
curl -I http://... y curl -I https://.../web/login · GitHub: erp/capturas/srv_05b_https_cabeceras.png



ssh root@... y ssh -o PubkeyAuthentication=no adminsrv@... · GitHub: erp/capturas/srv_07c_ssh_rechazado.png



psql -U postgres -c '\du' y psql -U local_app -d almichain · GitHub: erp/capturas/srv_08_postgres_roles.png



sudo fail2ban-client status sshd · GitHub: erp/capturas/srv_15_fail2ban.png

Capturas en `erp/capturas/`.

Incidencias

#	Qué pasó	Causa	Solución
1	La prueba 7 fallaba: el servidor pedía contraseña por SSH	`01-base.sh` se ejecutó antes de copiar la clave de `adminsrv`, así que dejó `PasswordAuthentication yes`. Además, alguien había añadido a mano `AllowUsers root` en `/etc/ssh/sshd_config.d/10-almichain.conf` (no lo bloqueaba `PermitRootLogin no`, pero sobraba)	Copia de seguridad del fichero, clave ed25519 con `ssh-copy-id`, comprobación con `ssh -o PasswordAuthentication=no`, quitar `AllowUsers root`, `PasswordAuthentication no`, `sshd -t` y `systemctl reload ssh` sin cerrar la sesión abierta. Volver a lanzar `01-base.sh` hace lo mismo, y ahora el script avisa al final si la contraseña sigue activa. `50-cloud-init.conf` también dice `yes`, pero no manda: sshd se queda con el primer valor y `10-` se lee antes
2	`curl -I http://…` mostraba `Server: nginx/1.28.3`	`server_tokens off` solo estaba en el bloque del 443	Añadido también al bloque del 80 en `docker/nginx/odoo.conf`. En un servidor ya montado: editar `/opt/almichain/nginx/odoo.conf` y `docker compose restart nginx` (no basta `reload`: `sed -i` crea un fichero nuevo y el contenedor seguiría viendo el viejo)
3	No existía `/srv/copias` ni el cron de copias	`05-copias.sh` no se llegó a ejecutar: `sdd` (20 GB) estaba vacío y sin montar (captura)	✅ 27/10 · `sudo bash 05-copias.sh` (pide escribir `SI` antes de formatear `sdd`). La clave de restic se guarda en KeePassXC. Después, pruebas 13 y 14 (`../backups/`)
4	En el servidor, `git pull` daba `dubious ownership` y luego `divergent branches`	La copia del repo en `~/almi-chain` se había clonado como root, seguía en la rama antigua `feat/servidor-odoo` (de antes de reorganizar el historial) y tenía un cambio local sin subir en `03-certificados.sh`, ya obsoleto	`sudo chown -R adminsrv:adminsrv ~/almi-chain`, `git restore` del cambio, `git switch main`, `git reset --hard origin/main` y `git config pull.ff only`, para que un `pull` nunca vuelva a mezclar historiales (captura). Clonar el repo siempre como `adminsrv`, nunca con `sudo`
5	La copia automática no se hacía nunca: `/var/log/almichain-copia.log` vacío (0 bytes) el 03/11	El cron está a las 02:30 (y la remota a las 03:30), pero la VM del servidor solo se enciende en horario de clase: `last -x` muestra arranques entre las 08:37 y las 13:44 y ningún apagado limpio (captura). En un restaurante el servidor estaría encendido toda la noche	Cron funciona: una tarea temporal a las 09:30 hizo la copia sola (captura). Arreglo: línea `@reboot` que copia 5 min después de arrancar (10 min la remota), además de la de las 02:30. Ya está en `05-copias.sh` y en la instalación de la copia remota (captura). Mejora pendiente: apagar la VM con `sudo poweroff`, no cerrando VirtualBox a la fuerza

7. Base de datos de los locales

Responsable: Alejandro

> **Dónde está en GitHub**
>
Guía de la base de datos	`bd/README.md`
Scripts SQL (00 a 05)	`bd/scripts/Scripts_postgre`
Diagrama E/R editable	`bd/er-entidades-relaciones.drawio`
Requisitos y modelo relacional (PDF)	`bd`
Capturas de las consultas	`bd/capturas`

Resumen

La base de datos de los locales es independiente de la de Odoo y guarda lo que Odoo no cubre en nuestra configuración: locales, mesas, reservas, recetas, proveedores y el stock mínimo por local. Partimos de un **modelo E/R** con atributos derivado (el total de una venta), multivaluado (los teléfonos de un local) y compuesto (la dirección), una **relación ternaria** entre proveedor, producto y local (`pedidos_suministro`) y un **historial de precios** de los platos. Se pasó a un modelo relacional de **17 tablas en 3FN**, con la normalización justificada paso a paso. Las tablas «espejo» (platos, clientes, empleados, ventas...) llevan una columna `odoo_id` con el identificador real de Odoo, así que la unión es por diseño y por carga de datos, no una sincronización en vivo. Los scripts SQL (reset, roles, tablas, permisos, datos y consultas) se ejecutan en orden, con **dos roles de mínimo privilegio**: `almichain_owner` , que solo modifica la estructura, y `almichain_app` , que solo lee y escribe datos; no son superusuarios y las contraseñas no están en ningún fichero. Las conexiones van cifradas con TLS 1.3. Con los datos de ejemplo se probaron **11 consultas** (ventas por local y día, platos más vendidos, precio de un plato en una fecha, pedidos por proveedor y local...) con el rol `almichain_app` , y las capturas están en `bd/capturas/` . Además, la justificación de Odoo y PostgreSQL explica por qué se eligieron.

Base de datos de los locales (carpeta `bd/`)

> **En GitHub:** `bd/README.md`

Base de datos PostgreSQL de la cadena de restaurantes Almi-Chain: locales, mesas, empleados, platos, ingredientes, proveedores, clientes, ventas, inventario y reservas. Convive con Odoo (ver Relación con Odoo).

- **Gestor:** PostgreSQL 16
- **Herramienta gráfica usada para el diseño físico:** pgAdmin 4 (el diagrama del modelo físico se genera con Graphviz desde el catálogo de la base de datos, para que salgan todas las relaciones)
- **Normalización:** 3FN (justificada en el documento del modelo relacional)

Contenido de la carpeta

Los scripts `.sql` están en `scripts/Scripts_postgre/` .

Fichero	Qué es	Quién lo ejecuta
`00_reset.sql`	Borra las 17 tablas **y sus datos**. Solo para desarrollo y pruebas	`almichain_owner`
`01_roles_y_bd.sql`	Crea el rol dueño y la base de datos	Administrador (`postgres`)

Fichero	Qué es	Quién lo ejecuta
`02_tablas.sql`	Crea las 17 tablas con claves primarias, claves externas y restricciones (DDL)	`almichain_owner`
`03_permisos.sql`	Crea el rol de uso diario y le da solo permisos de datos	Administrador (`postgres`)
`04_datos.sql`	Datos de ejemplo (ficticios). **Vacía las tablas al empezar**	`almichain_owner`
`05_consultas.sql`	Las 11 consultas útiles para el restaurante	`almichain_app`
`er-conceptual.png`, `er-entidades-relaciones.drawio`	Diagrama E/R conceptual y su fuente editable	
`er-fisico.png`, `er-fisico.dot`	Modelo físico con las 17 tablas y sus relaciones, generado desde el catálogo de PostgreSQL. Se regenera con `dot -Tpng er-fisico.dot -o er-fisico.png`	

Documentación asociada (en esta misma carpeta): requisitos, atributos y claves, modelo relacional con la normalización y capturas de las pruebas (`capturas/`).

Orden de ejecución

Desde la carpeta donde están los scripts. Las contraseñas **no están en ningún fichero**: se asignan a mano con `\password` y se guardan en un gestor de contraseñas.

```
# 1. Rol dueño y base de datos
sudo -u postgres psql < 01_roles_y_bd.sql
sudo -u postgres psql -c "\password almichain_owner"

# 2. Tablas
psql -h localhost -U almichain_owner -d almichain -f 02_tablas.sql

# 3. Rol de uso diario y permisos
sudo -u postgres psql -d almichain < 03_permisos.sql
sudo -u postgres psql -c "\password almichain_app"

# 4. Datos de ejemplo
psql -v ON_ERROR_STOP=1 -h localhost -U almichain_owner -d almichain -f 04_datos.sql

# 5. Consultas
psql -h localhost -U almichain_app -d almichain -f 05_consultas.sql
```

Comprobación rápida (con `almichain_app`): `SELECT count(*) FROM lineas_ventas;` debe devolver **32**.

Para repetir todo desde cero: `00_reset.sql`, `02_tablas.sql`, `03_permisos.sql` (`DROP TABLE` elimina también los permisos; el aviso `role "almichain_app" already exists` es inofensivo y los `GRANT` sí se aplican) y `04_datos.sql`.

> Los nombres `almichain`, `almichain_owner` y `almichain_app` son los del entorno de pruebas. Si en otro servidor cambian, basta con sustituirlos en los scripts.

Seguridad (mínimo privilegio)

- **Dos roles.** `almichain_owner` es dueño de las tablas y solo se usa para modificar la estructura. `almichain_app` solo puede `SELECT`, `INSERT`, `UPDATE` y `DELETE`.
- **Sin superusuario** ni permisos de crear bases de datos o roles en ninguno de los dos.

- **Acceso cerrado por defecto:** `REVOKE ALL ... FROM PUBLIC` sobre la base de datos; solo entran los roles autorizados.
- **Prueba:** con `almichain_app`, la lectura funciona y `DROP TABLE` / `CREATE TABLE` fallan (`must be owner of table`, `permission denied for schema public`).
- **Conexiones cifradas:** `psql` negocia TLS 1.3 al conectar por `localhost`.
- **Sin secretos en el repositorio:** ni contraseñas ni claves.

Modelo de datos

17 tablas:

- **Propias:** `locales`, `telefonos_local`, `mesas`, `categorias`, `proveedores`, `historial_precios`.
- **Espejo de Odoo** (llevan `odoo_id`): `platos`, `productos`, `clientes`, `empleados`, `ventas`, `lineas_ventas`.
- **Relaciones N:M** (PK compuesta): `recetas`, `suministras`, `inventarios`, `reservas`.
- **Relación ternaria** (proveedor, producto y local): `pedidos_suministro`.

Decisiones de diseño que cumplen la 3FN:

- `ventas` no guarda el local (se deduce por la mesa) ni el total (se calcula con `SUM(cantidad * precio_unitario)` de sus líneas).
- `lineas_ventas.precio_unitario` guarda el precio **en el momento de la venta**, distinto del precio actual del plato.
- `UNIQUE (local_id, numero)` en `mesas`: el número no se repite dentro del mismo local. La barra de cada local es la mesa 0.
- `UNIQUE (mesa_id, fecha, hora)` en `reservas`: una mesa no se puede reservar dos veces a la misma hora.
- `stock_minimo` está en `inventarios` (varía por local) y no en `productos`.
- **Relación ternaria `pedidos_suministro`:** qué proveedor sirve qué producto a qué local y cuándo. Su PK es (`proveedor_id`, `producto_id`, `local_id`, `fecha`) y la clave externa (`producto_id`, `proveedor_id`) apunta a `suministras`, así que solo se puede pedir a un proveedor un producto que suministre.
- **Historial de precios `historial_precios`:** guarda cada precio de un plato con su periodo (`fecha_desde` y `fecha_hasta`, vacía si es el vigente). `platos.precio` es el precio actual.
- **Atributo multivaluado:** un local puede tener varios teléfonos, por eso están en la tabla `telefonos_local` y no en `locales`.
- **Atributo compuesto:** la dirección del local se guarda partida en `calle`, `numero`, `codigo_postal` y `ciudad`.
- **Atributo derivado:** el total de una venta no se guarda; se calcula con las líneas.

Diagramas (el E/R conceptual se puede editar abriendo er-entidades-relaciones.drawio en draw.io):

Relación con Odoo

Odoo y esta base de datos son **dos bases de datos separadas**. Odoo gestiona facturación, CRM, empleados, inventario, marketing y TPV; esta base de datos añade lo que Odoo no tiene en su configuración (locales, mesas, reservas, recetas, proveedores y el stock mínimo por local).

La unión es la columna `odoo_id` de las tablas espejo: guarda el identificador numérico que ese elemento tiene en Odoo (`UNIQUE NOT NULL`). Las tablas propias se enlazan con ellas mediante claves externas normales, y así PostgreSQL puede comprobar la integridad.

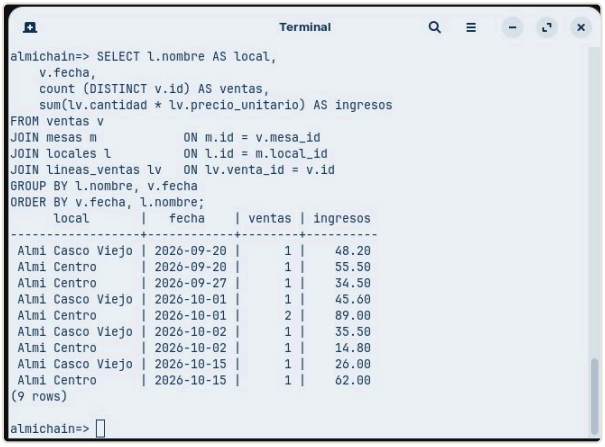
- Los `odoo_id` de platos, ingredientes, clientes y empleados son los reales de Odoo, obtenidos exportando el campo *ID* en CSV.
- Los de `ventas` y `lineas_ventas` son ficticios: todavía no hay ventas reales en Odoo.
- **Es una conexión por diseño y por carga de datos, no una integración en vivo.** Si un precio cambia en Odoo, esta base de datos no se entera hasta que se vuelvan a cargar los datos. En un entorno real habría que automatizarlo (un script programado o la API de Odoo).

Consultas

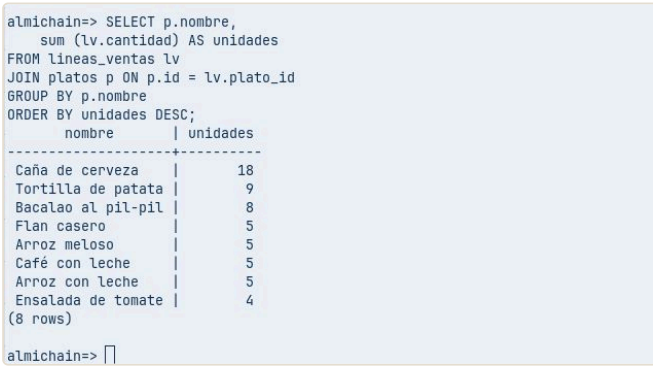
Resultados esperados con los datos de ejemplo:

#	Consulta	Resultado esperado
1	Ventas e ingresos por local y día	9 filas. El 1 de octubre en Almi Centro: 2 ventas y 89,00
2	Platos más vendidos	Caña de cerveza 18, Tortilla 9, Bacalao 8
3	Ingresos por plato	Bacalao 145,00, Tortilla 70,50, Arroz meloso 69,00… Café 9,00
4	Ventas e ingresos por mes	Septiembre: 3 ventas, 138,20. Octubre: 7 ventas, 272,90
5	Empleados por local	Almi Centro 3, Almi Casco Viejo 2
6	Ventas e ingresos de cada empleado	Jon 149,00, Maite 106,80, Unai 83,70, Ainhoa 71,60, Iker 0
7	Productos con poco stock	5 filas
8	Proveedores de un producto	Aceite de oliva: 2 proveedores
9	Stock de un producto por local	Bacalao: Almi Centro 3,00 y Almi Casco Viejo 6,00
10	Precio de un plato en una fecha concreta (`historial_precios`)	Tortilla de patata el 15/09/2026: 7,50 (vigente del 01/01 al 30/09)
11	Pedidos a cada proveedor por local (`pedidos_suministro`)	5 filas. Almi Centro, Bodega y Cafés Norte: 2 pedidos, 103,40

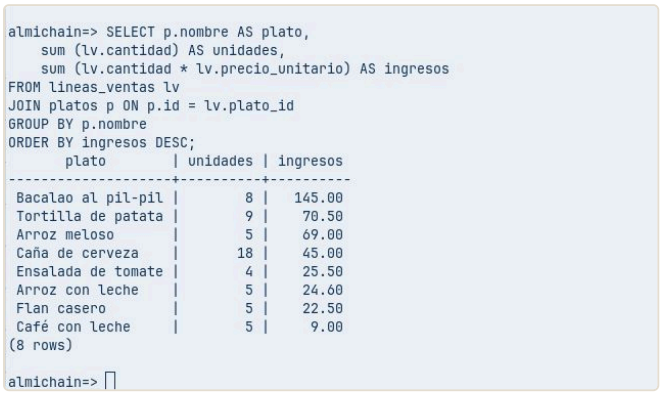
Capturas de las 11 consultas, ejecutadas con el rol `almichain_app`:



Consulta 1: ventas e ingresos por local y día · GitHub: bd/capturas/consulta-01.png



Consulta 2: platos más vendidos · GitHub: bd/capturas/consulta-02.png



Consulta 3: ingresos por plato · GitHub: bd/capturas/consulta-03.png



Consulta 4: ventas e ingresos por mes · GitHub: bd/capturas/consulta-04.png

```
almichain=> SELECT l.nombre AS local,
    e.nombre AS empleado,
    e.cargo
FROM empleados e
JOIN locales l ON l.id = e.local_id
ORDER BY l.nombre, e.nombre;
      local       |     empleado     |   cargo
------------------+------------------+-----------
 Almi Casco Viejo | Ainhoa Ruiz      | Camarera
 Almi Casco Viejo | Unai Pérez       | Encargado
 Almi Centro      | Iker Larrea      | Cocinero
 Almi Centro      | Jon Arana        | Camarero
 Almi Centro      | Maite Goikoetxea | Camarera
(5 rows)

almichain=>
```

Consulta 5: empleados por local · GitHub: bd/capturas/consulta-05.png

```
Terminal
 Almi Centro      | 2026-10-02 |      1 |    14.80
 Almi Casco Viejo | 2026-10-15 |      1 |    26.00
 Almi Centro      | 2026-10-15 |      1 |    62.00
(9 rows)

almichain=> SELECT e.nombre AS empleado,
    e.cargo,
    count (DISTINCT v.id)   AS ventas,
    coalesce (sum (lv.cantidad * lv.precio_unitario), 0) AS ingresos
FROM empleados e
LEFT JOIN ventas v ON v.empleado_id = e.id
LEFT JOIN lineas_ventas lv ON lv.venta_id = v.id
GROUP BY e.id, e.nombre, e.cargo
ORDER BY ingresos DESC, e.nombre;
     empleado     |   cargo   | ventas | ingresos
------------------+-----------+--------+----------
 Jon Arana        | Camarero  |      3 |   149.00
 Maite Goikoetxea | Camarera  |      3 |   106.80
 Unai Pérez       | Encargado |      2 |    83.70
 Ainhoa Ruiz      | Camarera  |      2 |    71.60
 Iker Larrea      | Cocinero  |      0 |        0
(5 rows)

almichain=>
```

Consulta 6: ventas e ingresos de cada empleado · GitHub: bd/capturas/consulta-06.png

```
almichain=> SELECT l.nombre AS local,
    p.nombre AS producto,
    p.unidad,
    i.cantidad_actual,
    i.stock_minimo,
    i.stock_minimo - i.cantidad_actual AS faltan
FROM inventarios i
JOIN locales l ON l.id = i.local_id
JOIN productos p ON p.id = i.producto_id
WHERE i.cantidad_actual < i.stock_minimo
ORDER BY l.nombre, faltan DESC;
      local       |    producto     | unidad | cantidad_actual | stock_minimo | faltan
------------------+-----------------+--------+-----------------+--------------+--------
 Almi Casco Viejo | Aceite de oliva | l      |            2.00 |         4.00 |   2.00
 Almi Casco Viejo | Cerveza         | l      |           18.00 |        20.00 |   2.00
 Almi Casco Viejo | Tomate          | kg     |            4.00 |         5.00 |   1.00
 Almi Centro      | Bacalao         | kg     |            3.00 |         5.00 |   2.00
 Almi Centro      | Café            | kg     |            1.50 |         2.00 |   0.50
(5 rows)

almichain=>
```

Consulta 7: productos con poco stock · GitHub: bd/capturas/consulta-07.png

```
Terminal
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression:
off)
Type "help" for help.

almichain=> \pset pager off
Pager usage is off.
almichain=> SELECT p.nombre AS producto,
    pr.nombre AS proveedor,
    pr.telefono,
    pr.correo
FROM productos p
JOIN suministras s      ON s.producto_id = p.id
JOIN proveedores pr     ON pr.id = s.proveedor_id
WHERE p.nombre ='Aceite de oliva'
ORDER BY pr.nombre;
    producto     |      proveedor       | telefono  |          correo
-----------------+----------------------+-----------+--------------------------
-
 Aceite de oliva | Bodega y Cafés Norte | 944100004 | info@bodegacafes.example
 Aceite de oliva | Despensa Bilbao      | 944100003 | comercial@despensa.example
(2 rows)

almichain=>
```

Consulta 8: proveedores de un producto (aceite de oliva) · GitHub: bd/capturas/consulta-08.png

```
Terminal
-----------------+----------------------+-----------+--------------------------
-
 Aceite de oliva | Bodega y Cafés Norte | 944100004 | info@bodegacafes.example
 Aceite de oliva | Despensa Bilbao      | 944100003 | comercial@despensa.example
(2 rows)

almichain=> SELECT p.nombre AS producto,
    l.nombre AS local,
    i.cantidad_actual,
    i.stock_minimo,
    p.unidad
FROM inventarios i
JOIN locales l ON l.id = i.local_id
JOIN productos p ON p.id = i.producto_id
WHERE p.nombre = 'Bacalao'
ORDER BY l.nombre;
 producto |      local       | cantidad_actual | stock_minimo | unidad
----------+------------------+-----------------+--------------+--------
 Bacalao  | Almi Casco Viejo |            6.00 |         5.00 | kg
 Bacalao  | Almi Centro      |            3.00 |         5.00 | kg
(2 rows)

almichain=>
```

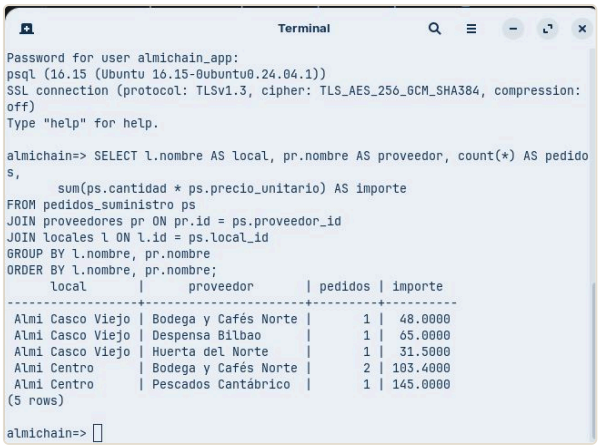
Consulta 9: stock de un producto por local (bacalao) · GitHub: bd/capturas/consulta-09.png

```
Terminal
alejandro@almi-System-Product-Name:~$ psql -h localhost -U almichain_app -d almi
chain
Password for user almichain_app:
psql (16.15 (Ubuntu 16.15-0ubuntu0.24.04.1))
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression:
off)
Type "help" for help.

almichain=> SELECT p.nombre AS plato, h.precio, h.fecha_desde, h.fecha_hasta
FROM historial_precios h
JOIN platos p ON p.id = h.plato_id
WHERE p.nombre LIKE 'Tortilla%'
  AND DATE '2026-09-15' BETWEEN h.fecha_desde AND COALESCE(h.fecha_hasta, 'infin
ity');
       plato        | precio | fecha_desde | fecha_hasta
--------------------+--------+-------------+-------------
 Tortilla de patata |   7.50 | 2026-01-01  | 2026-09-30
(1 row)

almichain=>
```

Consulta 10: precio de un plato en una fecha concreta · GitHub: bd/capturas/consulta-10.png



Consulta 11: pedidos a cada proveedor por local · GitHub: bd/capturas/consulta-11.png

Documentación

- Requisitos
- Atributos y claves
- Modelo relacional y normalización
- Justificación de Odoo y PostgreSQL

8. Disco redundante y copias de seguridad

Responsable: Sendoa

Dónde está en GitHub

Guía de RAID y copias	`backups/README.md`
Scripts de copia y restauración	`backups/scripts`
Bucket S3 con Terraform	`infra/terraform-copias`

Resumen

Las bases de datos del servidor (la de Odoo y la del local) y los adjuntos de Odoo están en un **RAID 1** con `mdadm` sobre dos discos. En la prueba de fallo se marcó un disco como roto: el RAID siguió funcionando con el otro, **Odoo siguió vendiendo** y, al reponer el disco, se reconstruyó solo en menos de 2 minutos. Como un RAID no protege de un borrado, además hay un **plan de copias** con **restic**: copia cifrada (AES-256), incremental y comprobada con `restic check`, en un tercer disco solo para root. La lanza **cron** a las 02:30 y también 5 minutos después de cada arranque, porque el servidor del aula no está encendido de noche (incidencia documentada). Guarda 7 copias diarias, 4 semanales y 6 mensuales. Para cumplir la regla **3-2-1**, otra tarea lleva las copias a un bucket de **AWS S3** creado con Terraform, cifrado y con otra clave. Las copias se han **verificado restaurándolas** en una base de datos aparte, tanto desde el disco local como desde S3, y los pedidos del TPV, los contactos, los productos y los adjuntos coincidían con los de producción. Las claves de restic solo están en el servidor (solo root) y en KeePassXC.

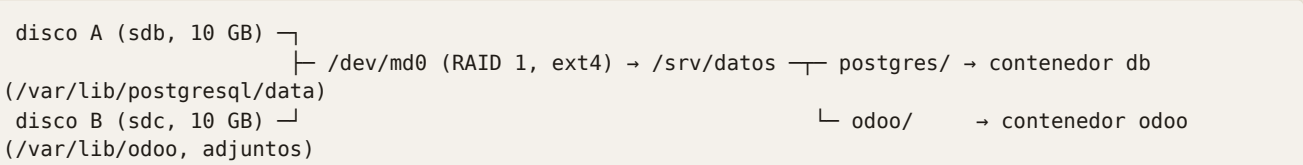
Disco redundante y copias de seguridad

> **En GitHub:** `backups/README.md`

Las bases de datos del servidor (`almichain`, la de Odoo, y `almichain_local`, la del local) y los adjuntos de Odoo están en un **RAID 1**. Además, cron hace una **copia diaria cifrada** con restic en un disco aparte, que se comprueba con una **prueba de restauración**. La instalación está en `../erp/`, con los scripts `02-raid.sh` y `05-copias.sh`.

Archivo	Qué es
`scripts/copia.sh`	Copia diaria. Se instala como `/usr/local/sbin/almichain-copia` y la lanza `/etc/cron.d/almichain-copia` a las 02:30
`scripts/restaurar_prueba.sh`	Prueba de restauración. Se instala como `/usr/local/sbin/almichain-restaurar-prueba`

1. RAID 1



Si un disco falla, todo sigue funcionando con el otro. Docker no arranca si el RAID no está montado. Para ver el estado: `cat /proc/mdstat` o `sudo mdadm --detail /dev/md0`.

> **La copia de configuración del router no se sube a GitHub:** contiene la contraseña de administración y la clave del Wi-Fi. Se guarda en KeePassXC como adjunto.

> **Un RAID no es una copia.** Si alguien borra una factura, se borra en los dos discos a la vez. Para eso está el plan de copias.

2. Plan de copias

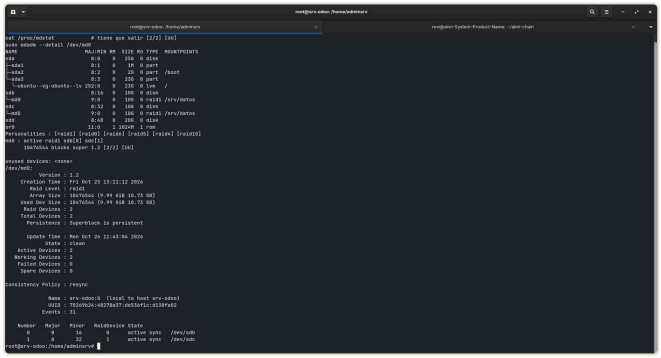
Pregunta	Respuesta
Qué	`pg_dump` de las dos BD y los usuarios de PostgreSQL, adjuntos de Odoo, stack de Docker (`/opt/almichain`), certificados, CA, RAID, UFW y SSH
Cómo	restic: copia **cifrada** (AES-256), incremental y deduplicada
Dónde	Disco C (`sdd`) en `/srv/copias`, aparte del RAID y solo para root
Cuándo	Cron, todos los días a las 02:30, y 5 minutos después de cada arranque por si el servidor estaba apagado a esa hora (incidencia 5). Registro en `/var/log/almichain-copia.log`
Retención	7 diarias, 4 semanales y 6 mensuales
Comprobación	`restic check` en cada copia y una prueba de restauración completa al mes
Clave	`/root/secretos/restic.txt` y KeePassXC. **Sin ella, la copia no se puede recuperar**

Copia fuera del local (regla 3-2-1): además, `restic copy` lleva las copias a un bucket de AWS S3. Ver el apartado 4.

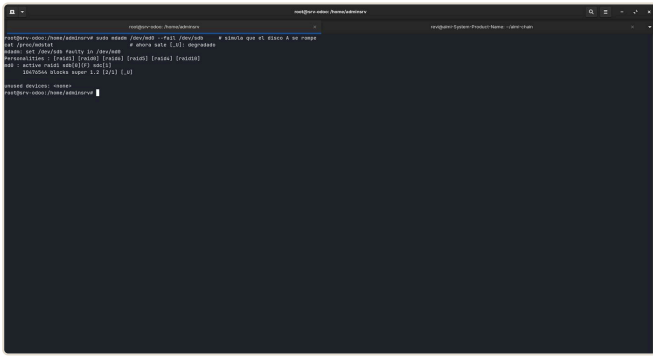
3. Pruebas

#	Prueba	Cómo	Resultado esperado	Evidencia
11	RAID sano	`cat /proc/mdstat`	`[2/2] [UU]`	✅ 26/10 · `sdb` y `sdc` *active sync*, estado *clean* · captura. Persistente: `ARRAY /dev/md0` en `mdadm.conf`, `/srv/datos` en `fstab` con `nofail`, y `postgres/` y `odoo/` dentro del RAID · captura
12	**Fallo de un disco**	Comandos de abajo	`[_U]` y Odoo sigue funcionando. Al reponer el disco vuelve a `[UU]`	✅ 26/10 · `sdb` marcado `(F)` y RAID en `[_U]` (captura); repuesto y reconstruido a `[UU]` en menos de 2 min (captura); Odoo siguió funcionando durante el fallo (captura)
13	Copia automática	`sudo grep "Copia terminada" /var/log/almichain-copia.log` y `sudo journalctl -u cron | grep almichain`	Copia lanzada por cron, sin nadie, y «Copia terminada correctamente»	✅ 03/11 · Instalada el 27/10: `sdd` en `/srv/copias` (captura), cron (captura) y primera copia `76f0e3dc` (captura). El 03/11 el registro estaba **vacío**: el servidor solo se enciende de día y nunca estaba a las 02:30 (captura, incidencia 5). Con una tarea temporal a las 09:30, `CRON[3134]` lanzó `almichain-copia` solo y terminó a las 09:30:13 con `restic check` sin errores y 4 instantáneas por la retención (captura). Después se borró la tarea temporal y se añadió la copia al arrancar (captura)

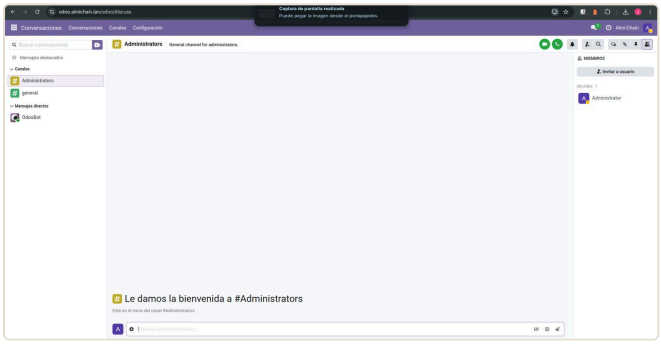
#	Prueba	Cómo	Resultado esperado	Evidencia
14	**Restauración**	Vende algo en el TPV, ejecuta `sudo almichain-copia` y luego `sudo almichain-restaurar-prueba`	`RESULTADO: RESTAURACIÓN CORRECTA`	✅ 27/10 · Venta en el TPV y copia manual `1e6be61a`, con `restic check` sin errores (captura). Restaurada en la BD aparte `almichain_restaurada`: 16 pedidos del TPV, 10 contactos, 6 productos y 484 adjuntos, iguales que en producción. Facturas 0 en las dos, porque cada venta sale como ticket del TPV, no como factura numerada de Contabilidad. La BD de prueba se borra al terminar (captura)



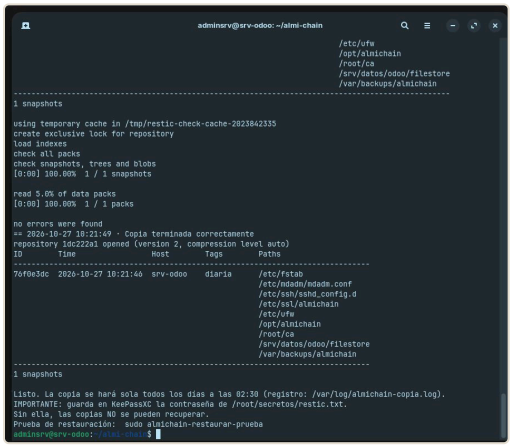
RAID sano · GitHub: erp/capturas/srv_11_raid_sano.png



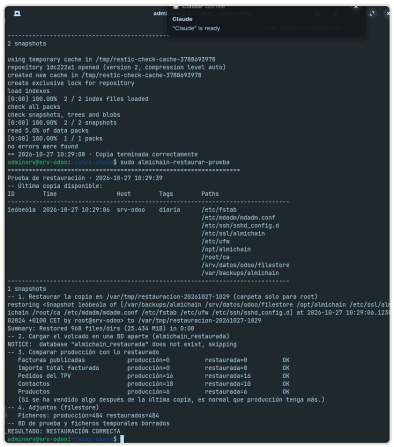
Fallo de un disco · GitHub: erp/capturas/srv_12a_raid_degradado.png



Fallo de un disco · GitHub: erp/capturas/srv_12c_odoo_con_raid_degradado.png



Copia automática · GitHub: erp/capturas/srv_13d_primera_copia.png



Restauración · GitHub: erp/capturas/srv_14b_restauracion_correcta.png

```
cat /proc/mdstat                          # antes: [UU]
sudo mdadm /dev/md0 --fail /dev/sdb       # simula que el disco A se rompe -> [_U]
#   (vender en el TPV: Odoo sigue funcionando)
sudo mdadm /dev/md0 --remove /dev/sdb
sudo mdadm /dev/md0 --add /dev/sdb        # disco "nuevo"
watch cat /proc/mdstat                    # hasta [UU]
```

4. Copia remota en AWS S3

Tercera copia, **fuera del local**: si el restaurante sufre un incendio o un robo, el RAID y el disco C se pierden a la vez, pero la copia de la nube no. Así se cumple la regla **3-2-1** (3 copias, en 2 soportes distintos y 1 fuera del local).

```
 RAID 1 (/srv/datos) ──restic, 02:30──► disco C (/srv/copias/restic) ──restic copy, 03:30──► AWS S3 (cifrado)
```

Pregunta	Respuesta
Dónde	Bucket S3 privado creado con Terraform: `infra/terraform-copias/`
Cómo	`scripts/copia_remota.sh` (`/usr/local/sbin/almichain-copia-remota`): `restic copy` del repositorio local al de S3, con la misma retención y `restic check`
Cifrado	Doble: restic cifra con AES-256 antes de subir, con una **clave distinta** a la local, y S3 cifra en reposo. Solo por HTTPS
Cuándo	Cron a las 03:30, después de la copia local, y 10 minutos después de cada arranque. Registro en `/var/log/almichain-copia-remota.log`
Secretos	`/root/secretos/aws-s3.env` (credenciales) y `/root/secretos/restic-remoto.txt` (clave), solo root. La clave, también en KeePassXC
Limitación del Learner Lab	Las credenciales caducan al cerrar la sesión: hay que renovarlas en `aws-s3.env` cada vez, y con el lab apagado la copia falla y lo deja en el registro. En una empresa se usaría un usuario IAM fijo con permiso solo sobre el bucket (política)

Instalación (una vez)

```
# En el servidor, como adminsrv, con el repo actualizado (git pull)
sudo install -m 750 backups/scripts/copia_remota.sh /usr/local/sbin/almichain-copia-remota
sudo install -m 600 /dev/null /root/secretos/aws-s3.env
```

```
sudo nano /root/secretos/aws-s3.env      # las 4 líneas de abajo
sudo almichain-copia-remota --iniciar    # crea el repositorio en S3 y su clave
printf '%s\n' '30 3 * * * root /usr/local/sbin/almichain-copia-remota >> /var/log/almichain-copia-
remota.log 2>&1' \
  '@reboot root sleep 600 && /usr/local/sbin/almichain-copia-remota >> /var/log/almichain-copia-
remota.log 2>&1' \
  | sudo tee /etc/cron.d/almichain-copia-remota
```

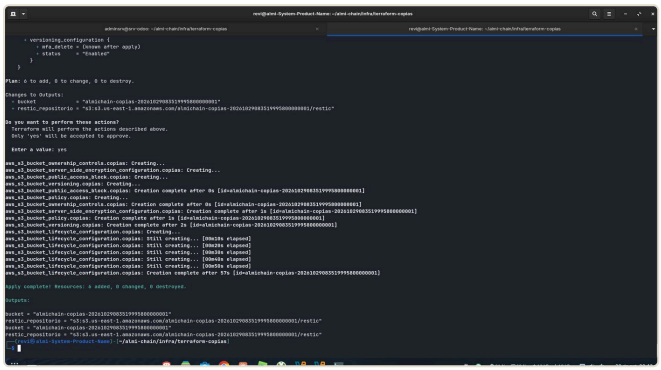
Contenido de `/root/secretos/aws-s3.env`: se puede pegar tal cual el bloque del Learner Lab (*AWS Details* → *AWS CLI*, con `[default]` y los nombres en minúsculas) y añadir debajo la línea `RESTIC_REPOSITORY` con el valor de `terraform output`. También vale en mayúsculas:

```
AWS_ACCESS_KEY_ID=...
AWS_SECRET_ACCESS_KEY=...
AWS_SESSION_TOKEN=...
RESTIC_REPOSITORY=s3:s3.us-east-1.amazonaws.com/almichain-copias-XXXX/restic
```

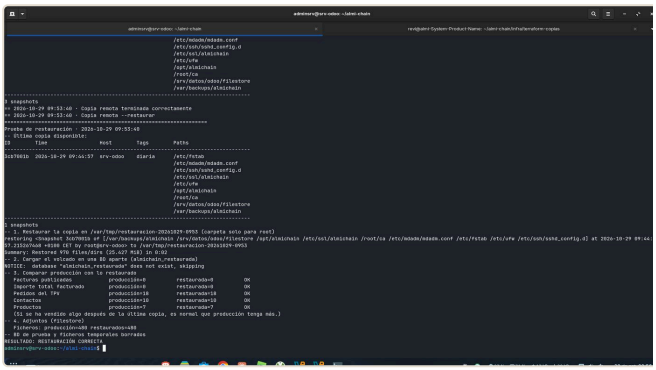
Pruebas

#	Prueba	Cómo	Resultado esperado	Evidencia
16	Bucket creado con Terraform	`terraform apply` y consola de S3	6 recursos; bucket privado, cifrado y con versionado	✅ 29/10 · `Apply complete! Resources: 6 added`. El bucket se crea con AWS CLI por la SCP del Learner Lab (incidencia) · captura
17	**Copia remota**	`sudo almichain-copia-remota`	Instantáneas copiadas, `restic check` sin errores y «Copia remota terminada correctamente»	✅ 29/10 · Las 3 instantáneas locales copiadas a S3 (repositorio `6243e12b`, distinto del local `1dc222a1`), retención aplicada y `restic check` sin errores · registro
18	**Restauración desde S3**	`sudo almichain-copia-remota --restaurar`	`RESULTADO: RESTAURACIÓN CORRECTA` leyendo de la nube	✅ 29/10 · Instantánea `3cb7001b` restaurada desde S3 en una BD aparte: 18 pedidos del TPV, 10 contactos, 7 productos y 480 adjuntos, iguales que en producción · captura · **10/11, repetida en directo delante del profesor:** copia remota terminada correctamente y restauración desde S3 de la instantánea `d61bd63b` (1031 ficheros, 77 MB, en 11 s), con 25 pedidos del TPV, 25 contactos, 24 productos y 521 adjuntos iguales en producción y en la copia (captura)

#	Prueba	Cómo	Resultado esperado	Evidencia
19	Sin credenciales válidas	Con el lab apagado: `sudo almichain-copia-remota`	Falla con un mensaje claro y no toca la copia local	✅ 29/10 · Con el lab apagado: «ERROR: no se puede abrir el repositorio de S3. ¿Learner Lab apagado o credenciales caducadas?» y sale sin tocar nada · captura



Bucket creado con Terraform · GitHub: docs/infra/capturas/aws_15_copias_apply.png



Restauración desde S3 · GitHub: erp/capturas/srv_18_restauracion_s3.png

Incidencias de la copia remota

Qué pasó	Causa	Solución
`terraform apply` fallaba con `AccessDenied` en `s3:GetBucketObjectLockConfiguration`	SCP de AWS Academy	Bucket con AWS CLI y el resto con Terraform (detalle)
`[default]: command not found`	El script cargaba `aws-s3.env` con `source` y el bloque del Learner Lab no es Bash	El script lee el fichero línea a línea y solo exporta las 4 variables esperadas, sin ejecutar nada
`client.BucketExists: 400 Bad Request`	El token de sesión se estropeó al pegarlo a mano en `nano`	Copiar `~/.aws/credentials` del portátil al servidor con `scp` y añadir la línea `RESTIC_REPOSITORY`
La clave del repositorio remoto salió en pantalla al pegar varios comandos a la vez	Se ejecutó el `cat` de la clave	Como el repositorio aún no existía, se borró la clave y se generó otra al repetir `--iniciar`. Lección: los comandos con secretos, de uno en uno

5. Recuperación real (sustituye la BD de producción)

Antes, haz siempre la prueba de restauración.

```
cd /opt/almichain
export RESTIC_REPOSITORY=/srv/copias/restic RESTIC_PASSWORD_FILE=/root/secretos/restic.txt
sudo -E restic restore latest --host srv-odoo --target /var/tmp/recuperar
sudo docker compose stop odoo
sudo docker compose exec -u postgres db dropdb almichain
sudo docker compose exec -u postgres db createdb -O odoo almichain
sudo sh -c 'docker compose exec -T -u postgres db pg_restore --no-owner --role=odoo -d almichain <
/var/tmp/recuperar/var/backups/almichain/almichain.dump'
sudo docker compose exec -u postgres db psql -c 'REVOKE CONNECT ON DATABASE almichain FROM PUBLIC'
sudo rsync -a --delete /var/tmp/recuperar/srv/datos/odoo/filestore/ /srv/datos/odoo/filestore/
sudo docker compose start odoo
sudo rm -rf /var/tmp/recuperar
```

Bucket S3 para la copia remota (Terraform)

> **En GitHub:** `infra/terraform-copias/README.md`

Crea el bucket de AWS S3 donde va la **copia remota** de restic del servidor Odoo. Con esto el plan de copias cumple la regla **3-2-1**: los datos en el RAID 1, una copia en el disco C del servidor y otra fuera del local, en la nube.

Está en una carpeta aparte de `../terraform/` (la web) a propósito: así un `terraform destroy` de la web nunca puede borrar las copias.

Qué crea

Recurso	Para qué
Bucket `almichain-copias-<sufijo>`	Se crea con **AWS CLI**, no con Terraform (ver la incidencia de abajo)
`aws_s3_bucket_public_access_block`	Bloquea cualquier acceso público
`aws_s3_bucket_ownership_controls`	Sin ACL: solo permisos de IAM
`aws_s3_bucket_server_side_encryption_configuration`	Cifrado en reposo (AES-256). Además restic cifra todo antes de subirlo
`aws_s3_bucket_versioning`	Si alguien borra o sobrescribe una copia, la versión anterior se conserva
`aws_s3_bucket_lifecycle_configuration`	Las versiones antiguas se borran a los 30 días, para no acumular coste
`aws_s3_bucket_policy`	Rechaza cualquier petición que no vaya por HTTPS

Pasos

```
# Learner Lab encendido y credenciales en ~/.aws/credentials (igual que para la web)
aws sts get-caller-identity

# 1. Crear el bucket (una sola vez). El nombre tiene que ser único en todo AWS
BUCKET="almichain-copias-$(date +%Y%m%d%H%M%S)"
aws s3api create-bucket --bucket "$BUCKET" --region us-east-1
echo "bucket = \"$BUCKET\"" > infra/terraform-copias/terraform.tfvars

# 2. Configurar su seguridad con Terraform
cd infra/terraform-copias
terraform init
terraform validate
terraform plan        # 6 recursos
terraform apply
terraform output      # bucket y restic_repositorio
```

El valor de `restic_repositorio` es el que va en el servidor (ver `backups/README.md`).

Incidencia: AccessDenied al crear el bucket con Terraform

Qué pasó	Causa	Solución
`terraform apply` creaba el bucket y luego fallaba con `AccessDenied ... s3:GetBucketObjectLockConfiguration ... explicit deny in a service control policy`	En AWS Academy, una política de la organización (SCP) prohíbe consultar el *object lock*. El recurso `aws_s3_bucket` del proveedor de AWS lo consulta siempre después de crear el bucket	El bucket se crea con `aws s3api create-bucket` y Terraform gestiona el resto de la configuración (acceso, cifrado, versionado, ciclo de vida y política), que no necesita ese permiso. En una cuenta normal de AWS, `aws_s3_bucket` funciona sin problema

Limitación de AWS Academy y cómo sería en una empresa

En el Learner Lab las credenciales son temporales: caducan al cerrar la sesión y no se puede crear un usuario IAM propio. Por eso la copia remota solo funciona con el lab encendido, y hay que renovar las credenciales del servidor en cada sesión.

En una empresa real se crearía un **usuario IAM solo para las copias**, con una clave fija y permiso **únicamente sobre este bucket** (mínimo privilegio), y el cron haría la copia remota cada noche sin intervención:

```
{
  "Version": "2012-10-17",
  "Statement": [
    { "Effect": "Allow", "Action": ["s3:ListBucket", "s3:GetBucketLocation"],
      "Resource": "arn:aws:s3:::almichain-copias-XXXX" },
    { "Effect": "Allow", "Action": ["s3:GetObject", "s3:PutObject", "s3:DeleteObject"],
      "Resource": "arn:aws:s3:::almichain-copias-XXXX/*" }
  ]
}
```

9. Página web y hosting en AWS

Responsable: Jose (bocetos y web) y Jon (AWS)

Dónde está en GitHub

Bocetos	`docs/bocetos`
Guía de la web	`docs/web/guia_web.md`
Código de la web	`web`
Despliegue en AWS	`docs/infra/despliegue_aws.md`
Terraform	`infra/terraform`

Resumen

Parte	Fuente
Bocetos (diseño conceptual)	`bocetos/bocetos.md`
Web: diseño, código, accesibilidad y pruebas	`web/guia_web.md`
Despliegue en AWS con Terraform, Nginx, HTTPS y DuckDNS	`infra/despliegue_aws.md` y `infra/terraform/README.md`

Antes de programar la web se hicieron 19 bocetos, que se valoraron con cinco criterios. Ganó el concepto **Latón** (fondo negro y detalles dorados), con un 8,8 sobre 10. La web es unidireccional: solo informa. Es responsiva y accesible, y tiene un slideshow de platos, un mapa de los locales con Leaflet y geolocalización, y un formulario de muestra que no envía datos. Está publicada en https://almichain.duckdns.org con HTTPS. En el validador del W3C da 0 errores y 0 avisos, y en Lighthouse saca entre 99 y 100.

El despliegue es **infraestructura como código**: un solo `terraform apply` crea en AWS una **EC2 t3.micro** con Ubuntu, su grupo de seguridad (80 y 443 abiertos, y el 22 solo desde la IP del administrador) y una **IP elástica**, para que la dirección no cambie entre sesiones del laboratorio. Al arrancar, el script de `user-data` instala **Nginx** con compresión y cabeceras de seguridad, da de alta el dominio gratuito en **DuckDNS** y saca el certificado de **Let's Encrypt** con **Certbot**, con redirección de HTTP a HTTPS y renovación automática. Se comprobó que es reproducible destruyéndolo todo y volviéndolo a crear: la web volvió a estar en marcha, con un certificado nuevo, en 1 minuto y 45 segundos. Ninguna credencial está en el repositorio: el token y el estado de Terraform se quedan fuera de Git.

Bocetos de la web (diseño conceptual)

> **En GitHub:** `docs/bocetos/bocetos.md`

Responsable: Jose · **Fechas:** 26–27 oct · **Carpeta en el repo:** `docs/bocetos/`

Objetivo

Definir la estructura y el aspecto de la web antes de programarla, en móvil, tablet y escritorio.

Secciones (en este orden)

1. Cabecera (`<header>`): logo y menú de navegación.
2. Inicio: imagen de portada, eslogan (`<h1>`) y botón "Ver carta".
3. Platos estrella: slideshow con flechas, puntos y texto alternativo en cada foto.
4. Carta resumida: entrantes, principales y postres.
5. Dónde estamos: mapa con Leaflet y botón de geolocalización.
6. Contacto: formulario de muestra que no envía datos.
7. Pie (`<footer>`): horario, dirección, teléfono y redes.

Decisiones de diseño

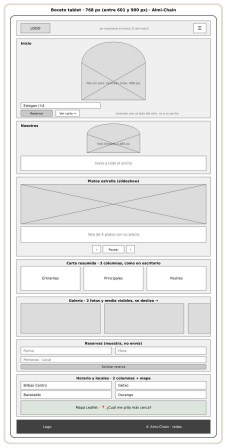
- **Pensado primero para el móvil:** la mayoría de los clientes buscan dónde comer desde el móvil, así que se dibujó primero a 375 px y luego se amplió.
- **Tres tamaños**, con dos cortes en el CSS (`@media`):

Tamaño	Ancho	Cómo se ve	Boceto
Móvil	hasta 600 px	Una columna. Menú plegado en un botón ☰ (`<button aria-expanded>`). La galería se desliza con el dedo	`boceto_movil.png`
Tablet	de 601 a 900 px	Sigue el menú ☰, pero se aprovecha el ancho: carta en 3 columnas, locales en 2, fotos centradas sin ocupar toda la pantalla y botones uno al lado del otro	`boceto_tablet.png`

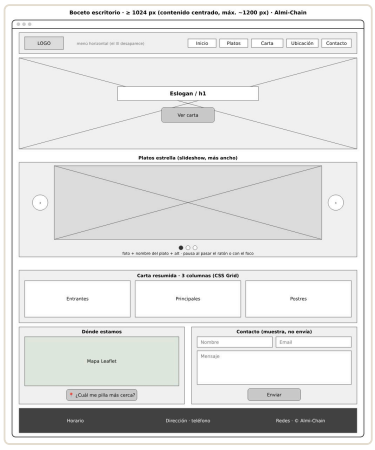
Tamaño	Ancho	Cómo se ve	Boceto
Escritorio	más de 900 px	Menú horizontal, carta en 3 columnas (CSS Grid), mapa y locales lado a lado y contenido centrado con un ancho máximo	`boceto_escritorio.png`



Móvil · GitHub: docs/bocetos/boceto_movil.png



Tablet · GitHub: docs/bocetos/boceto_tablet.png



Escritorio · GitHub: docs/bocetos/boceto_escritorio.png

- **Accesibilidad:** HTML semántico, `alt` en las imágenes, controles del slideshow usables con teclado y que se paran con el foco o el ratón.
- **Mínimo privilegio y privacidad:** la ubicación se pide solo al pulsar el botón; el formulario no recoge datos.

Por qué el diseño es usable

Para justificarlo usamos cuatro de las **10 heurísticas de usabilidad de Jakob Nielsen** (reglas generales para que una web sea fácil de usar) y el contraste de la WCAG.

Criterio	Qué dice	Cómo lo cumple el diseño
Visibilidad del estado del sistema (Nielsen 1)	El usuario siempre sabe dónde está y qué está pasando	El menú se queda fijo arriba al bajar por la página. En el slideshow, el plato que se ve está marcado en la lista y el botón cambia entre *Pausar* y *Reanudar*
Control y libertad del usuario (Nielsen 3)	El usuario puede deshacer o salir fácilmente	El slideshow se puede parar. El menú del móvil se cierra con la ✕, con la tecla Esc o tocando fuera. La ubicación solo se pide si el cliente pulsa el botón

Criterio	Qué dice	Cómo lo cumple el diseño
Consistencia y estándares (Nielsen 4)	Lo mismo se ve y funciona igual en toda la web	Todos los botones tienen la misma forma y color (latón), los títulos de sección siguen el mismo patrón y el ☰ es el icono de menú que todo el mundo conoce
Diseño estético y minimalista (Nielsen 8)	Mostrar solo lo necesario	Una sola acción principal en la portada (*Reservar mesa*) y cada sección con un único título y un único objetivo. Fondo negro sin adornos para que destaquen las fotos de los platos
Botones fáciles de pulsar	En el móvil, los botones deben medir al menos 44 × 44 px (WCAG 2.5.5)	Botones de 48 px de alto, iconos del pie de 44 × 44 px y el *Reservar mesa* a todo el ancho en el móvil
Contraste	WCAG AA pide 4,5:1 para el texto	Latón sobre negro: **8,5:1**. Texto principal: 16,2:1. Texto secundario: 7,4:1

Fuentes: Nielsen Norman Group, *10 Usability Heuristics for User Interface Design* (https://www.nngroup.com/articles/ten-usability-heuristics/) y WCAG 2.1 (https://www.w3.org/TR/WCAG21/).

Pasos para reproducirlo

1. Abrir Excalidraw (https://excalidraw.com) o similar.
2. Dibujar un marco de 375 px de ancho (móvil), otro de 768 px (tablet) y otro de unos 1024–1280 px (escritorio).
3. Colocar las secciones en el orden de arriba, con rectángulos y cruces para las imágenes.
4. Exportar en PNG y guardar en `docs/bocetos/`.

Evidencias

- `boceto_movil.png`
- `boceto_tablet.png` (y su `boceto_tablet.svg`, editable)
- `boceto_escritorio.png`
- `Almi-Chain_bocetos_web.pdf`: las opciones dibujadas.
- `Almi-Chain_bocetos_finalistas.pdf`: las finalistas y la valoración. Ganó **Latón** (8,8 sobre 10).

Página web de Almi-Chain

> **En GitHub:** `docs/web/guia_web.md`

Responsable	Fechas (Gantt)	Carpeta en el repo	Despliegue
Jose	28/10 – 04/11	`web/` · `docs/`	Jon (AWS, Nginx, HTTPS)

1. Objetivo

El enunciado pide una web **unidireccional** (solo informa), **atractiva y responsiva**, con **geolocalización**, un **formulario de contacto de muestra** y un **slideshow de platos estrella**, alojada en la nube. Además, el equipo se marcó como objetivo que fuera accesible y tuviera buen rendimiento (Lighthouse).

Resultado: una página única (`web/index.html`) con 9 secciones, una hoja de estilos y un archivo JavaScript, sin frameworks.

2. Requisitos previos

- Un editor (VS Code) y un navegador moderno.
- Python 3, para probarla en local con `python3 -m http.server`.
- El boceto elegido: concepto **Latón** (ver `docs/bocetos/`).

3. Decisiones y justificación

Decisión	Por qué
HTML, CSS y JS "a mano", sin frameworks	Es una web pequeña. Así carga rápido y el equipo puede explicar cada línea, que es lo que se evalúa
Una sola página con anclas (`#carta`, `#reservas`…)	Toda la información cabe en una página y la navegación es inmediata
Diseño *mobile first* y CSS Grid	Primero el móvil (la mayoría de los clientes buscan desde el móvil). Dos *media queries*: por debajo de 900 px una columna y menú ☰, y entre 601 y 900 px (tablet) se recuperan varias columnas donde caben
Paleta negro + latón	Transmite calidad; el latón sobre negro tiene 8,5:1 de contraste (WCAG AA pide 4,5:1)
Fuentes Cormorant Garamond + Jost **servidas desde nuestro servidor**	Si se cargaran desde Google, el navegador del cliente enviaría su IP a Google (RGPD). Así además carga más rápido
Leaflet + OpenStreetMap para el mapa	Gratis, sin clave de API y de código abierto. Google Maps pide una clave de API y tarjeta de crédito
Leaflet se carga **solo cuando el mapa va a verse**	No retrasa la carga inicial (mejor rendimiento)
Formulario de reservas **de muestra**	La web es unidireccional: no puede recoger datos

4. Pasos para reproducirlo

1. **Crear la estructura de carpetas** `web/css`, `web/js`, `web/img`, `web/fonts` y `web/vendor/leaflet`. *Por qué:* separar contenido (HTML), presentación (CSS) y comportamiento (JS) facilita el mantenimiento.
2. **Descargar las fuentes** en woff2 desde Google Fonts (por ejemplo, con *google-webfonts-helper*) y guardarlas en `web/fonts/`. En el CSS se declaran con `@font-face` y `font-display: swap`, para que el texto se vea mientras carga la fuente.
3. **Descargar Leaflet 1.9.4** (`npm pack leaflet@1.9.4`, o desde leafletjs.com) y copiar `leaflet.js`, `leaflet.css` e `images/` a `web/vendor/leaflet/`.
4. **Escribir `index.html`** con las etiquetas semánticas: `header`, `nav`, `main`, `section` (una por apartado) y `footer`.
5. **Escribir `css/styles.css`**: primero las variables de color (`:root`), después la base y luego cada sección. Al final, la *media query* para móvil y la de `prefers-reduced-motion`.
6. **Escribir `js/main.js`**: menú móvil, slideshow, opiniones, formulario de muestra, mapa con geolocalización y animaciones.
7. **Probar en local:** `cd web && python3 -m http.server 8000` y abrir `http://localhost:8000`. La geolocalización funciona en `localhost` porque el navegador lo trata como contexto seguro. 📸 *Captura:* `capturas/web_01_escritorio_completa.png`, `web_02_movil_completa.png` y `web_04_tablet_768_completa.png` (ventana a 768 px de ancho).
8. **Validar el HTML.** Pasar `index.html` por el validador del W3C (https://validator.w3.org/nu/, opción "File upload"). 📸 *Captura:* resultado "Document checking completed. No errors or warnings to show."
9. **Pasar Lighthouse** (Chrome → F12 → Lighthouse), en móvil y en escritorio. 📸 *Captura:* las cuatro puntuaciones.
10. **Subir a GitHub** la carpeta `web/` y esta guía en `docs/`. Jon la despliega en la EC2 con Nginx y HTTPS.



Escritorio completa · GitHub: docs/web/capturas/web_01_escritorio_completa.png



Movil completa · GitHub: docs/web/capturas/web_02_movil_completa.png



Tablet 768 completa · GitHub: docs/web/capturas/web_04_tablet_768_completa.png

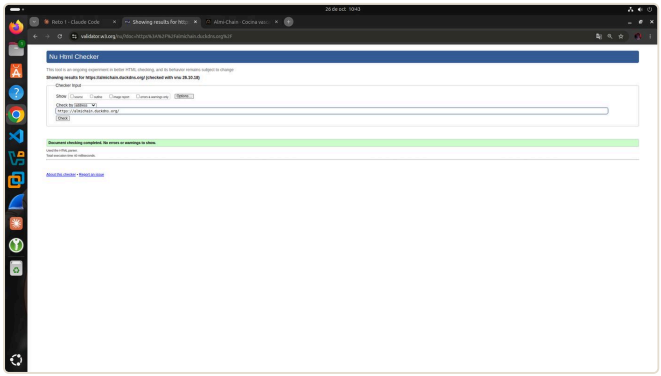
5. Seguridad y mínimo privilegio

- **No se recoge ningún dato.** `form-action 'none'` en la CSP bloquea cualquier envío de formulario, y además el JS cancela el `submit`.
- **Política de seguridad de contenidos (CSP):** solo se ejecutan scripts y se cargan estilos y fuentes de nuestro dominio. Si alguien inyectara un script de otro sitio, el navegador lo bloquearía.
- **Ninguna clave, token ni contraseña** en el código. Por eso se eligió OpenStreetMap: no necesita clave de API.
- **Ubicación con permiso explícito.** Solo se pide al pulsar el botón, y la distancia se calcula en el navegador; la posición no se envía a ningún sitio.
- **Cabeceras de seguridad en Nginx** (las configura Jon; vienen en `web/README.md`).

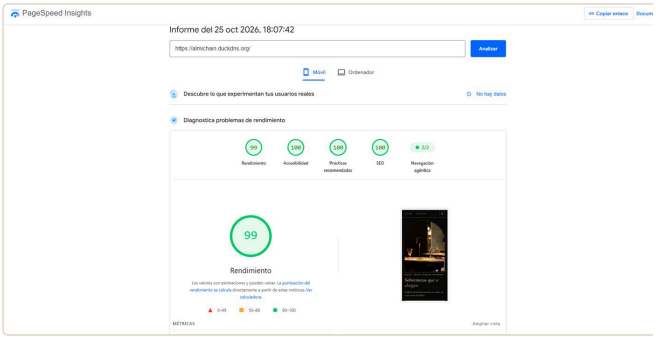
6. Pruebas y evidencias

Prueba	Resultado esperado	Resultado obtenido	Evidencia
La web se ve bien a 1440 px y a 390 px	Sin desbordes ni texto cortado	✅	`capturas/web_01…`, `web_02…`
La web se ve bien en tablet (768 px)	Sin desbordes ni texto cortado. Carta en 3 columnas y locales en 2	✅ 05/11. Sin el corte de tablet se veía como un móvil estirado (fotos enormes y carta en una columna), así que se añadió el `@media` de 601 a 900 px	`capturas/web_04_tablet_768_completa.png`

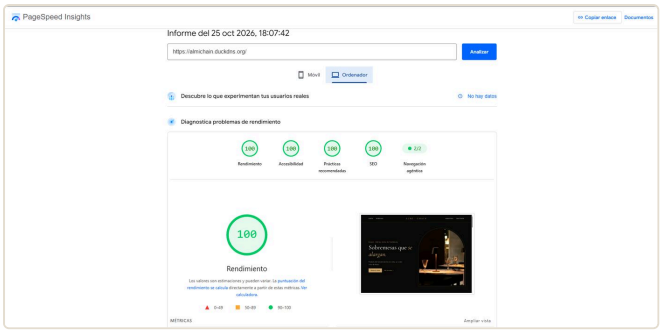
Prueba	Resultado esperado	Resultado obtenido	Evidencia
Menú móvil: abrir, Tab, Esc	Se abre, el foco queda dentro y se cierra con Esc	✅	`capturas/web_03_menu_movil.png`
Slideshow: clic, flechas ↑↓ y Pausar	Cambia el plato y la foto; se puede parar	✅	— (hacer vídeo corto)
Formulario de muestra	No envía nada y muestra el aviso	✅	—
Geolocalización (simulada en Getxo)	"El más cercano es Getxo, a unos 3,1 km"	✅	—
Validación del HTML (W3C)	0 errores	✅ Con la URL publicada (26/10/2026): 0 errores y 0 avisos. Subiendo el archivo salían 8 avisos de la CSP (ver apartado 8)	`capturas/w3c_validacion_publicada.png`, `capturas/w3c_validacion.png`
Lighthouse en local (móvil)	≥ 90	✅ Rendimiento 99 · Accesibilidad 100 · Buenas prácticas 100 · SEO 100	`evidencias/lighthouse_local_movil.html`
Lighthouse en local (escritorio)	≥ 90	✅ 100 · 100 · 100 · 100	`evidencias/lighthouse_local_escritorio.html`
Lighthouse de la web publicada, con PageSpeed Insights sobre `https://almichain.duckdns.org` (Jon)	≥ 90	✅ 25/10 · Móvil: Rendimiento 99 · Accesibilidad 100 · Buenas prácticas 100 · SEO 100. Ordenador: 100 · 100 · 100 · 100	`evidencias/pagespeed_publicada_movil.png`, `evidencias/pagespeed_publicada_escritorio.png`
Web publicada en AWS: carga por `https://almichain.duckdns.org` con las fotos reales	Candado y página completa, sin imágenes provisionales	✅ 22/10	`capturas/aws_01_portada_https.png`, `aws_02_nosotros.png`, `aws_03_platos_slideshow.png`
Web publicada: formulario de muestra	Muestra el aviso y no envía nada	✅ 22/10	`capturas/aws_04_reservas_formulario_muestra.png`
Web publicada: mapa con los 4 locales	Aparecen los 4 marcadores sobre OpenStreetMap	✅ 22/10	`capturas/aws_05_mapa_locales.png`



Validación del HTML (W3C) · GitHub: docs/web/capturas/w3c_validacion_publicada.png



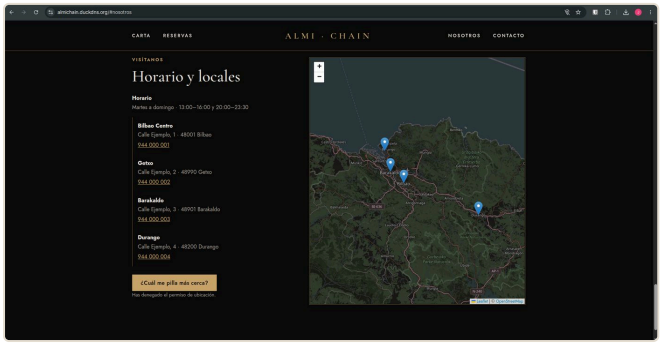
Lighthouse de la web publicada, con PageSpeed Insights sobre https://almichain.duckdns.org… · GitHub: docs/web/evidencias/pagespeed_publicada_movil.png



Lighthouse de la web publicada, con PageSpeed Insights sobre https://almichain.duckdns.org... · GitHub: docs/web/evidencias/pagespeed_publicada_escritorio.png



Web publicada en AWS: carga por https://almichain.duckdns.org con las fotos reales · GitHub: docs/web/capturas/aws_01_portada_https.png



Web publicada: mapa con los 4 locales · GitHub: docs/web/capturas/aws_05_mapa_locales.png

Nota: las pruebas en local se hicieron con fotos provisionales en SVG. Con las fotos reales hay que repetir Lighthouse: si las fotos pesan mucho, el rendimiento baja (de ahí la recomendación de WebP de menos de 150 KB).

7. Explicación del código (para la defensa)

HTML: estructura semántica

- `<header>`, `<nav>`, `<main>`, `<section>` y `<footer>` le dicen al navegador y a los lectores de pantalla qué es cada parte. Cada `section` tiene `aria-labelledby` apuntando a su título.
- **"Saltar al contenido"** (`<a class="skip" href="#main">`): es el primer enlace, oculto hasta que recibe el foco. Permite a quien navega con teclado saltarse el menú.
- **Imágenes** con `width` y `height` (así el navegador reserva el hueco y la página no "salta" al cargar), con `alt` descriptivo y con `loading="lazy"` en todas menos la principal, que lleva `fetchpriority="high"`.
- **Botones de verdad** (`<button>`) para todo lo que hace algo, y enlaces (`<a href>`) para lo que lleva a otro sitio. Así funcionan con teclado sin código extra.

CSS

- **Variables** (`--bg`, `--ac`...): toda la paleta en un sitio. Cambiar el color de la marca es cambiar una línea.
- **`clamp()`** en los tamaños de letra: `clamp(2.75rem, 5.5vw, 5rem)` significa "como mínimo 2,75 rem, como máximo 5 rem, y entre medias el 5,5 % del ancho de la pantalla". El texto se adapta sin necesidad de *media queries*.
- **El arco** es `border-radius: 999px 999px 0 0` (esquinas superiores totalmente redondas). **El marco interior** es `outline: 1px solid; outline-offset: -12px` (un borde dibujado 12 px hacia dentro). No hacen falta imágenes extra.
- **Responsive:** por debajo de 900 px las rejillas pasan a una columna, el menú se convierte en botón y la galería se desliza en horizontal (`scroll-snap`). Entre 601 y 900 px (tablet) hay un segundo `@media` que,

sobre lo anterior, vuelve a poner la carta en 3 columnas y los locales en 2, centra las fotos con un ancho máximo de 480 px y deja los botones de la portada uno al lado del otro.
- `prefers-reduced-motion`: si el sistema del usuario pide menos movimiento, se quitan las animaciones y el slideshow no avanza solo.
- **Mapa oscuro** con `filter: invert(1) hue-rotate(180deg)` sobre las teselas: invierte los colores y recoloca los tonos para que el mapa sea oscuro sin contratar otro servicio.

JavaScript

- **Menú móvil:** al abrirlo cambia `aria-expanded` a `true` (el lector de pantalla anuncia "expandido") y el foco entra en el menú. Se cierra con Esc, con la X o con el fondo, y el foco vuelve al botón. Con Tab, el foco no se escapa del cajón ("trampa de foco").
- **Slideshow:** cada plato es un `<button>` de una lista ordenada. Al pulsarlo se marca con `aria-current="true"` y la foto cambia con un fundido. Avanza solo cada 6 s, pero se para al pasar el ratón o el foco, y tiene botón **Pausar** (WCAG 2.2.2 exige poder parar lo que se mueve solo más de 5 s).
- **Formulario de muestra:** `e.preventDefault()` cancela el envío y se muestra un aviso en una zona `role="status"`, que el lector de pantalla lee.
- **Mapa:** un `IntersectionObserver` detecta cuándo el mapa está cerca de verse y solo entonces carga Leaflet.
- **Geolocalización:** `navigator.geolocation.getCurrentPosition()` pide la posición (el navegador muestra el permiso). Con la **fórmula de Haversine** se calcula la distancia en km a cada local y nos quedamos con la menor (`reduce`). Se tratan los errores: permiso denegado, sin señal o tiempo agotado. `window.isSecureContext` comprueba que estamos en HTTPS, porque si no la API no funciona.

Preguntas que puede hacer el profesor

- **¿Por qué la geolocalización necesita HTTPS?** Porque la ubicación es un dato sensible; los navegadores solo la permiten en contextos seguros, para que nadie pueda interceptarla.
- **¿Cómo garantizas que la web no recoge datos?** El formulario no tiene `action`, el JS cancela el envío y la CSP con `form-action 'none'` lo bloquea aunque se quitara el JS.
- **¿Qué es la CSP?** Una lista blanca, enviada al navegador, de los orígenes desde los que puede cargar cada tipo de recurso. Protege contra la inyección de scripts (XSS).
- **¿Por qué no Bootstrap o WordPress?** Para una web de una página añadiría peso y superficie de ataque (WordPress necesita PHP y una base de datos) sin aportar nada. Con HTML estático, el servidor solo entrega archivos.
- **¿Cómo se comprobó la accesibilidad?** Con el contraste calculado con la fórmula de la WCAG, Lighthouse (100) y probando la página solo con teclado.

Más preguntas, con respuestas cortas, en `defensa_web.md`.

8. Problemas encontrados y solución

- **Leaflet usa una imagen `data:` para las teselas vacías**, que la CSP bloqueaba. Solución: añadir `data:` a `img-src`, solo para imágenes.
- **Fondo gris del mapa antes de cargar las teselas.** El selector era `.map .leaflet-container`, pero la clase está en el propio `#map`. Solución: `.map.leaflet-container`.
- **Avisos de la CSP en el validador del W3C.** Al subir `index.html` como archivo, el validador da 0 errores pero 8 avisos del tipo *"Resource violates Content Security Policy"* para `styles.css`, `main.js` y las imágenes. Pasa porque un archivo subido suelto no tiene dominio, así que `'self'` (nuestro dominio) no coincide con nada. En el servidor real, CSS, JS e imágenes vienen del mismo dominio y la CSP los permite (Lighthouse da 100 en Buenas prácticas, sin errores en la consola). Comprobado el 26/10/2026: al validar la URL publicada (https://almichain.duckdns.org/) sale *"No errors or warnings to show"*.

9. Fuentes

- WCAG 2.1, contraste y 2.2.2 “Pausar, detener, ocultar”: https://www.w3.org/TR/WCAG21/ (consultado el 22/10/2026)
- Documentación de Leaflet: https://leafletjs.com/reference.html
- API de geolocalización (MDN): https://developer.mozilla.org/es/docs/Web/API/Geolocation_API
- Content Security Policy (MDN): https://developer.mozilla.org/es/docs/Web/HTTP/CSP
- Licencias: Cormorant Garamond y Jost (SIL Open Font License), Leaflet (BSD-2), teselas © colaboradores de OpenStreetMap (ODbL)

Código de la web

> **En GitHub:** `web/README.md`

Web estática **unidireccional**: solo informa. No tiene backend, base de datos ni formularios que envíen datos.

Estructura

```
web/
├── index.html              Página única con las 9 secciones
├── css/styles.css          Todos los estilos (variables de color al principio)
├── js/main.js              Menú móvil, slideshow, opiniones, formulario de muestra, mapa y
geolocalización
├── fonts/                  Cormorant Garamond y Jost (woff2, licencia SIL OFL), servidas desde aquí
├── vendor/leaflet/         Leaflet 1.9.4 (mapa), servido desde aquí (licencia BSD-2)
└── img/                    Fotos en WebP; fuente y licencia de cada una en img/CREDITOS.md
```

Pasos para reproducir

La guía completa (decisiones, pasos, pruebas y explicación del código para la defensa) está en `docs/web/guia_web.md`.

Probarla en local

La geolocalización solo funciona en HTTPS o en `localhost`, así que no basta con abrir el archivo con doble clic:

```
cd web
python3 -m http.server 8000
# abrir http://localhost:8000
```

Imágenes

Las fotos están en WebP, recortadas a su proporción (4:5 o 3:4) y con menos de 150 KB cada una. La fuente y la licencia de cada foto se anotan en `img/CREDITOS.md`. Si se cambia una foto, hay que cambiar también su texto `alt` en `index.html` y, en el slideshow, el atributo `data-img` del botón.

Seguridad y mínimo privilegio

- **No recoge datos.** El formulario de reservas es de muestra: el JavaScript cancela el envío y, además, la cabecera CSP tiene `form-action 'none'`, así que el navegador bloquea cualquier envío.
- **Sin claves, tokens ni contraseñas** en el código.
- **Sin dependencias de terceros en tiempo de carga.** Las fuentes y Leaflet se sirven desde nuestro servidor. Lo único externo son las teselas del mapa de OpenStreetMap (permitidas en la CSP).

- **Ubicación.** Solo se pide al pulsar el botón y nunca sale del navegador: la distancia se calcula en el propio JavaScript.
- **CSP** (`<meta http-equiv="Content-Security-Policy">`): solo se permiten scripts, estilos y fuentes de nuestro dominio.

Para Jon (despliegue en Nginx)

Lighthouse local da 99–100 en las cuatro categorías. Lo único que propone es comprimir y cachear, y eso se configura en el servidor:

```
# dentro del server { listen 443 ssl; ... }
root /var/www/almichain;
index index.html;

gzip on;
gzip_types text/css application/javascript image/svg+xml;

location ~* \.(css|js|woff2|svg|png|webp|jpg)$ {
    expires 30d;
    add_header Cache-Control "public";
}

# Cabeceras de seguridad (la CSP también va en el HTML; en cabecera admite frame-ancestors)
add_header Content-Security-Policy "default-src 'self'; script-src 'self'; style-src 'self'; img-src
'self' data: https://tile.openstreetmap.org; font-src 'self'; connect-src 'self'; form-action 'none';
base-uri 'self'; object-src 'none'; frame-ancestors 'none'" always;
add_header X-Content-Type-Options "nosniff" always;
add_header Referrer-Policy "strict-origin-when-cross-origin" always;
add_header Permissions-Policy "geolocation=(self), camera=(), microphone=()" always;
add_header Strict-Transport-Security "max-age=31536000" always;
```

Datos de ejemplo que hay que revisar

Las direcciones (`Calle Ejemplo…`), los teléfonos (`944 000 00X`), los precios, las opiniones y el tipo de cocina (vasca de temporada) son de ejemplo. Las coordenadas del mapa son aproximadas, del centro de cada municipio. Todo está en `index.html`, y las coordenadas en `js/main.js` (constante `LOCALES`).

Despliegue de la web en AWS con Terraform

> **En GitHub:** `docs/infra/despliegue_aws.md`

Responsable: Jon · Código: `infra/terraform/` · Web desplegada: `web/`

Este documento explica **qué se ha montado, por qué y cómo comprobarlo**. Los pasos exactos para repetirlo desde cero están en `infra/terraform/README.md`.

1. Qué pide el reto

> «La página debe servirse de forma segura, por HTTPS y con dominio propio, y estar alojada en la nube con un despliegue reproducible, a poder ser automatizado mediante scripts o infraestructura como código.»

Requisito	Cómo se cumple
Alojada en la nube (AWS)	Instancia EC2 en AWS Academy Learner Lab, región `us-east-1`
Dominio propio	`almichain.duckdns.org` (DuckDNS, gratuito)

Requisito	Cómo se cumple
HTTPS	Certificado de Let's Encrypt con Certbot; HTTP redirige a HTTPS
Despliegue reproducible	Terraform: un `terraform apply` crea todo y publica la web
Seguridad por defecto	Cortafuegos mínimo, SSH solo desde una IP, disco cifrado, secretos fuera de Git

2. Arquitectura

```
Visitante ──HTTPS (443)──► almichain.duckdns.org
                                  │  DuckDNS apunta a la IP elástica
                                  ▼
                           Elastic IP (fija)
                                  │
                  ┌───────────────┴──────────────┐
                  │ EC2 t3.micro · Ubuntu 24.04  │
                  │ Nginx · Certbot · cron (DuckDNS) │
                  └──────────────────────────────┘
        Security Group: 80 y 443 abiertos · 22 solo IP del administrador
```

Diagrama de despliegue completo (con la parte local y VirtualBox): `docs/diagramas/`.

3. Qué crea Terraform (5 recursos)

Recurso	Para qué
`aws_security_group.web`	Cortafuegos. HTTP y HTTPS para todos; SSH solo desde `admin_cidr`
`aws_instance.web`	Servidor Ubuntu 24.04 `t3.micro`, disco de 8 GB cifrado, IMDSv2 obligatorio
`aws_eip.web`	IP fija, para que DuckDNS no cambie aunque la instancia se pare
`aws_eip_association.web`	Une la IP elástica al servidor
`terraform_data.web`	Sube la carpeta `web/` al servidor. Se repite sola en cada `apply` si cambia algún archivo

Al **primer arranque** (`user_data.sh.tftpl`) el servidor se configura solo:

1. Instala Nginx, Certbot y herramientas.
2. Guarda el token de DuckDNS en `/etc/duckdns/token` (solo lo lee root) y lo actualiza cada 5 minutos con cron.
3. Configura Nginx con compresión, caché de 30 días y las cabeceras de seguridad de `web/README.md`.
4. Espera a que el dominio apunte a la IP del servidor y saca el certificado HTTPS.
5. Activa la renovación automática del certificado (`certbot.timer`).

4. Decisiones y por qué

Decisión	Motivo
EC2 + Nginx, no S3	El enunciado admite «S3, máquinas EC2». Con EC2 se controla Nginx, las cabeceras de seguridad y el certificado de Let's Encrypt, y se demuestra más administración de sistemas
Elastic IP	Sin ella, la IP cambia cada vez que el laboratorio para la instancia y habría que reconfigurar el dominio

Decisión	Motivo
DuckDNS + Let's Encrypt	Gratuitos. Route 53 obliga a pagar un dominio y ACM necesita un balanceador de carga o CloudFront, que gastan el crédito del laboratorio
`t3.micro`	Una web estática no necesita más. Es lo más barato que admite el laboratorio
Ubuntu 24.04 LTS oficial	Soporte hasta 2029. La imagen se busca automáticamente (la más reciente de Canonical)
Terraform	Es lo que pidió el profesor. Nota: desde 2023 no es software libre; OpenTofu es un reemplazo compatible
SSH solo desde nuestra IP	Mínimos privilegios: el servidor no ofrece administración remota a todo internet. Terraform rechaza `0.0.0.0/0`

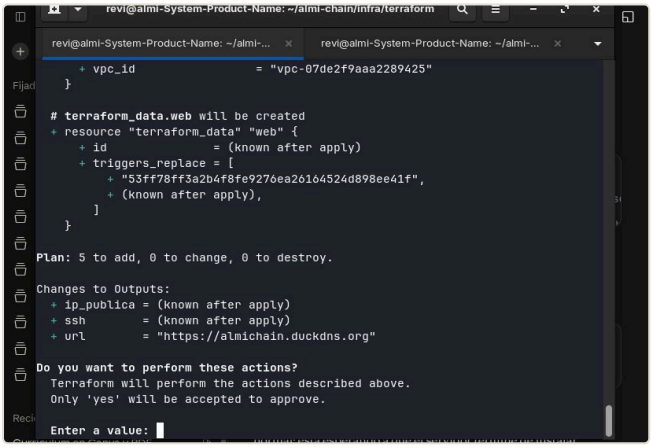
5. Seguridad

- **Secretos fuera de Git.** El token de DuckDNS va en `terraform.tfvars` y las credenciales de AWS en `~/.aws/credentials`. El `.gitignore` excluye `*.tfvars`, `*.tfstate` y `*.pem`; en el repositorio solo está `example.tfvars`, con valores falsos.
- **Estado de Terraform.** El `terraform.tfstate` puede contener el token en texto plano, así que **no se sube a Git**.
- **Cortafuegos mínimo.** Solo 80, 443 y 22 (restringido).
- **Disco cifrado** (EBS) e **IMDSv2 obligatorio**, para que un fallo en la web no permita robar credenciales de la instancia.
- **Cabeceras de seguridad** en Nginx: `Content-Security-Policy`, `Strict-Transport-Security`, `X-Content-Type-Options`, `Referrer-Policy` y `Permissions-Policy`. No se muestra la versión de Nginx (`server_tokens off`).
- **Web unidireccional.** El formulario es de muestra y la CSP lleva `form-action 'none'`.

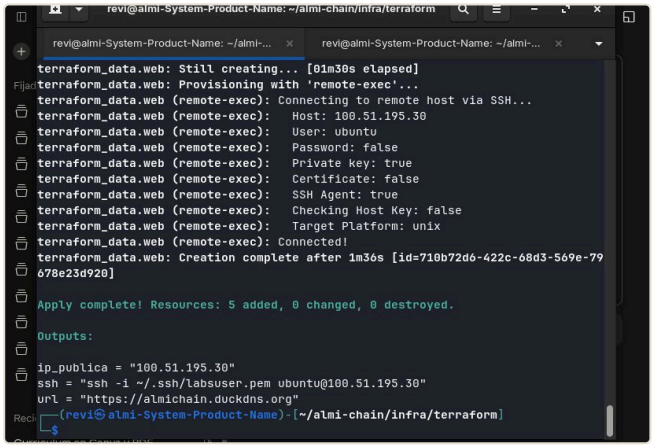
6. Pruebas y evidencias

Prueba	Resultado	Evidencia
`terraform plan`	5 recursos a crear, 0 a cambiar, 0 a destruir. El SSH se limita a una IP `/32`	ver captura «plan» debajo
`terraform apply`	`Apply complete! Resources: 5 added`. Salidas: IP pública, URL y comando SSH	ver captura «apply» debajo
La web carga por `https://almichain.duckdns.org` con las fotos reales	✅ 22/10	`docs/web/capturas/aws_01…05`
Revisión por el profesor	✅ 22/10: lo vio funcionando y le pareció correcto	—
HTTPS con certificado válido: el navegador muestra «La conexión es segura» en `almichain.duckdns.org`	✅ 23/10	ver captura «candado» debajo
Cabeceras de seguridad (`curl -I https://almichain.duckdns.org`): `200 OK`, `Content-Security-Policy`, `Strict-Transport-Security`, `X-Content-Type-Options`, `Referrer-Policy` y `Permissions-Policy`	✅ 23/10	ver captura «curl» debajo

Prueba	Resultado	Evidencia
Reproducibilidad (`destroy` y `apply` desde cero)	✅ 26/10: se destruyen los 5 recursos y se vuelven a crear en **1 min 45 s**, con una IP nueva y un certificado nuevo, y la web vuelve igual. Detalle en el apartado 6.1	ver captura «apply» debajo



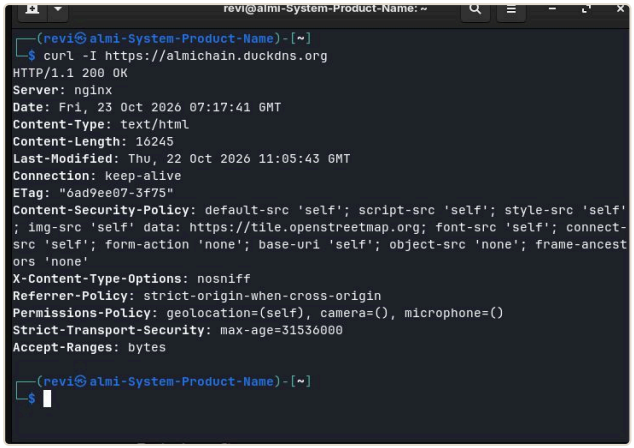
plan · GitHub: docs/infra/capturas/aws_01_terraform_plan_5_recursos.png



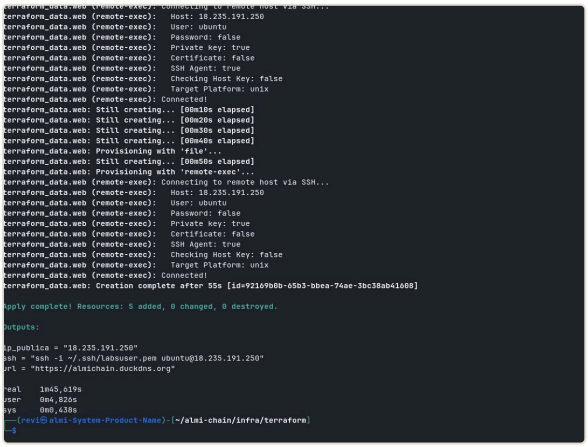
apply · GitHub: docs/infra/capturas/aws_02_terraform_apply_completo.png



candado · GitHub: docs/infra/capturas/aws_04_https_candado_navegador.png



curl · GitHub: docs/infra/capturas/aws_05_curl_cabeceras_seguridad.png

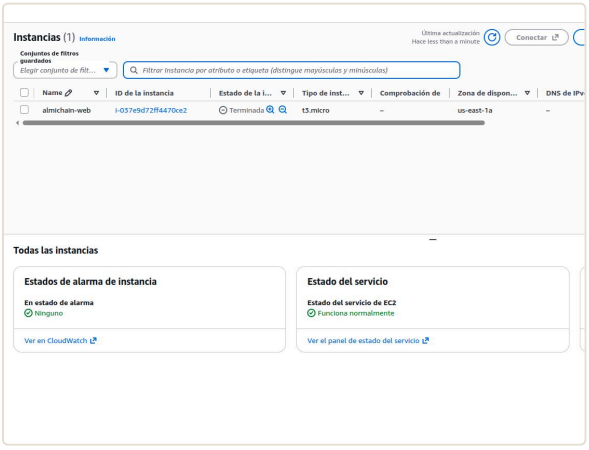


apply · GitHub: docs/infra/capturas/aws_12_apply_completo_1m45s.png

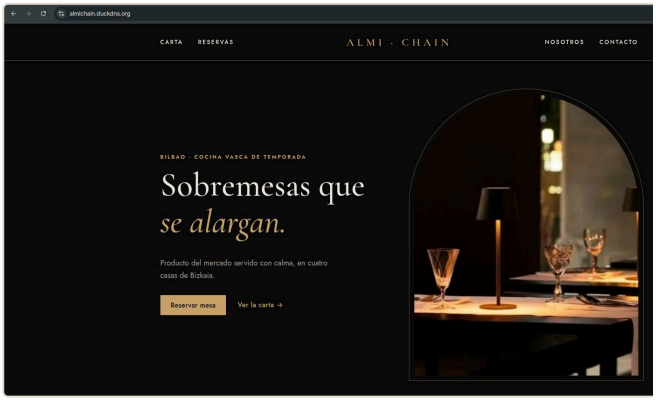
6.1 Prueba de reproducibilidad: `destroy` + `apply` (26/10/2026)

El enunciado pide un **despliegue reproducible**. La prueba es borrar toda la infraestructura y volver a crearla con un solo comando, sin tocar nada a mano.

Paso	Qué se hace	Resultado	Evidencia
1	`terraform state list`	Terraform gestiona la Elastic IP, su asociación, la instancia, el grupo de seguridad y la subida de la web	captura «state»
2	`terraform destroy`	Plan: `0 to add, 0 to change, 5 to destroy`. Se confirma con `yes`	captura «destroy» captura «confirmación»
3	Consola de EC2	La instancia `almichain-web` aparece como **Terminada**	ver captura «terminada» debajo
4	`time terraform apply`	`Destroy complete! Resources: 5 destroyed`, y el nuevo plan: `5 to add, 0 to change, 0 to destroy`	captura «apply» captura «plan»
5	Fin del `apply`	`Apply complete! Resources: 5 added` en **1 min 45 s**. Nueva IP pública: `18.235.191.250` (antes `100.51.195.30`)	ver captura «completo» debajo
6	Certificado (`openssl s_client`)	Emitido por Let's Encrypt el **26/10/2026 a las 06:31 GMT**: es nuevo, lo ha pedido el servidor recién creado	captura «certificado»
7	Navegador → `https://almichain.duckdns.org`	La web carga igual que antes	ver captura «web» debajo



terminada · GitHub: docs/infra/capturas/aws_09_consola_instancia_terminada.png



web · GitHub: docs/infra/capturas/aws_14_web_recreada.png

Qué demuestra: la IP cambió y nadie tocó DuckDNS ni el certificado a mano. El propio servidor actualizó el dominio, esperó a que apuntara a él y pidió el certificado. Toda la infraestructura está descrita en código, así que cualquiera del equipo puede reconstruirla en menos de dos minutos.

Incidencia durante el despliegue

El primer `terraform plan` falló: AWS no admite apóstrofos en las descripciones de las reglas del cortafuegos, y la descripción decía «Let's Encrypt». Se corrigió quitando el apóstrofo (commit `fix(infra)`) y el segundo intento salió bien. Terraform detectó el error **antes** de crear nada, así que no se creó ningún recurso a medias.

7. Costes y límites del laboratorio

Concepto	Coste aproximado
Servidor `t3.micro` encendido	~0,01 $/hora

Concepto	Coste aproximado
IP elástica y disco, aunque el laboratorio esté cerrado	~0,14 $/día
Presupuesto del Learner Lab	50 $

- Al cerrar el laboratorio, la instancia se para sola. Al iniciarlo de nuevo, hay que **arrancarla en EC2** y la web vuelve con el mismo dominio.
- Las credenciales del laboratorio **caducan en cada sesión**: hay que volver a copiarlas antes de usar Terraform.
- Si cambia la IP desde la que se administra, hay que actualizar `admin_cidr` y volver a hacer `terraform apply`.
- Para borrar todo: `terraform destroy`, y comprobar en la consola que no queda ninguna IP elástica sin asociar.

8. Qué se guarda y dónde

Qué	Dónde	¿En Git?
Código de Terraform y plantilla `example.tfvars`	`infra/terraform/`	Sí
Web	`web/`	Sí
Este documento y las capturas	`docs/infra/`	Sí
`terraform.tfvars` (token de DuckDNS, tu IP)	Solo en el equipo de quien despliega	**No**
`terraform.tfstate`	Solo en el equipo de quien despliega. Copia de seguridad cifrada fuera de Git	**No**
Credenciales de AWS y `labsuser.pem`	Las genera el laboratorio en cada sesión	**No**

Si se pierde el `terraform.tfstate`, Terraform no sabe qué ha creado y habría que borrar a mano en la consola de EC2 la instancia, la IP elástica, el disco y el grupo de seguridad `almichain-web-sg`.

9. Preguntas que pueden hacer en la defensa

- **¿Por qué EC2 y no S3?** Por el control sobre Nginx, las cabeceras y el certificado (ver sección 4).
- **¿Qué pasa si se borra la instancia?** Se vuelve a levantar igual con `terraform apply`, con el mismo dominio y la web publicada.
- **¿Dónde está el token de DuckDNS?** En `terraform.tfvars`, que no se sube a Git, y en el servidor, en un archivo que solo lee root.
- **¿Cómo se renueva el certificado?** Los certificados de Let's Encrypt duran 90 días. Certbot instala un temporizador (`certbot.timer`) que lo renueva solo cuando quedan menos de 30.
- **¿Cómo se actualiza la web?** Se cambia `web/`, se hace pull request y se ejecuta `terraform apply`, que solo vuelve a subir la web.
- **¿Qué falló y cómo se arregló?** La incidencia de la sección 6.

Infraestructura en AWS (Terraform)

> **En GitHub:** `infra/terraform/README.md`

Despliegue reproducible de la web en AWS con Terraform, Nginx con HTTPS y dominio DuckDNS. Las credenciales nunca se suben: usar `example.tfvars` como plantilla.

```
Internet ──HTTPS──► almichain.duckdns.org ──► Elastic IP ──► EC2 t3.micro · Ubuntu 24.04 · Nginx + Certbot
                                                          Security Group: 80 y 443 abiertos · 22 solo IP del admin
```

Un solo `terraform apply` crea el servidor, lo configura, saca el certificado HTTPS y publica la carpeta `web/`.

La explicación completa (decisiones, seguridad, pruebas, costes y preguntas de la defensa) está en `docs/infra/despliegue_aws.md`.

Qué crea

Recurso	Para qué
`aws_security_group.web`	Cortafuegos: HTTP/HTTPS para todos, SSH solo desde la IP del administrador (validado: no admite `0.0.0.0/0`)
`aws_instance.web`	Servidor Ubuntu 24.04 `t3.micro`, disco cifrado, IMDSv2 obligatorio
`aws_eip.web`	IP fija, para que el dominio no cambie al parar y arrancar
`user_data.sh.tftpl`	Primer arranque: Nginx con compresión, caché y cabeceras de seguridad; DuckDNS cada 5 min; certificado Let's Encrypt y redirección HTTP → HTTPS; renovación automática
`terraform_data.web`	Sube `web/` al servidor. Se repite sola en cada `apply` si cambia algún archivo de la web

Requisitos

- Terraform ≥ 1.5 y AWS CLI en el portátil.
- Cuenta en duckdns.org con un subdominio creado y su token.
- Learner Lab de AWS Academy iniciado (semáforo en verde).

Pasos para reproducir

1. **Credenciales del laboratorio.** Caducan en cada sesión, así que hay que repetirlo cada vez. En el Learner Lab: *AWS Details* → *AWS CLI* → *Show*. Copia el bloque en `~/.aws/credentials` y comprueba con `aws sts get-caller-identity`.
2. **Clave SSH.** En *AWS Details* → *Download PEM*:

   ```
   mv ~/Descargas/labsuser.pem ~/.ssh/ && chmod 400 ~/.ssh/labsuser.pem
   ```

3. **Variables.**

   ```
   cd infra/terraform
   cp example.tfvars terraform.tfvars
   curl ifconfig.me          # tu IP → admin_cidr = "x.x.x.x/32"
   ```

   Rellena `terraform.tfvars`. No se sube a Git.

4. **Desplegar.**

   ```
   terraform init
   terraform plan
   terraform apply
   ```

   El `apply` tarda unos 5–10 minutos, porque espera a que el servidor termine de configurarse antes de subir la web.

5. **Comprobar.** Abre la dirección de `terraform output url`. Si algo falla, mira el registro del primer arranque:

```
ssh -i ~/.ssh/labsuser.pem ubuntu@$(terraform output -raw ip_publica) 'sudo cat /var/log/almichain-setup.log'
```

6. **Actualizar la web.** Tras cambiar algo en `web/`, basta con volver a ejecutar `terraform apply`.

Ahorrar crédito del laboratorio

- Al terminar la sesión, el laboratorio para la instancia solo. La Elastic IP y el disco se mantienen, y al volver la web sigue en el mismo dominio.
- **Si cambia tu IP** (otra red, clase o casa), actualiza `admin_cidr` y haz `terraform apply`. Si no, SSH y la subida de la web no conectan.
- Para borrarlo todo: `terraform destroy`.
- `terraform.tfstate` se queda **solo en el portátil de quien despliega**. No se sube a Git, porque guarda el token. Si se pierde, hay que borrar los recursos a mano en la consola.

Pruebas (evidencias para la documentación)

```
IP=$(terraform output -raw ip_publica); URL=$(terraform output -raw url)
curl -I http://$IP                      # 301 → https
curl -I $URL                            # 200, con Content-Security-Policy y Strict-Transport-Security
curl -sI -H 'Accept-Encoding: gzip' $URL/css/styles.css | grep -i -e content-encoding -e expires
echo | openssl s_client -connect ${URL#https://}:443 2>/dev/null | openssl x509 -noout -issuer -dates
nmap -Pn -p 22,80,443 $IP               # desde otra red: 22 filtrado
```

Para comparar con la prueba local, pasa Lighthouse sobre la URL pública. También se pueden revisar las cabeceras con securityheaders.com.

10. Máquina recreativa

Responsable: Jon

> **Dónde está en GitHub**
>
> | | |
> |---|---|
> | Guía de la recreativa | `recreativa/README.md` |
> | Medidas de rendimiento | `recreativa/medidas/resultados.md` |
> | Scripts | `recreativa/scripts` |

Resumen

La recreativa es una máquina virtual con los mismos límites que el equipo real: **1 núcleo, 1 GB de RAM y sin swap**. Lleva **antiX 26.1 core** (5 servicios y unos 160 MB de RAM al arrancar) y **MAME 0.276**, con una ROM legal y gratuita (Gridlee). Al encenderla entra sola en el menú de MAME a pantalla completa, sin escritorio ni contraseña, y ese menú hace de front-end para elegir juego. La optimización ajusta MAME a un solo núcleo (vídeo por software, salto de fotogramas y sonido más ligero), quita consolas sobrantes y acorta el arranque. Jugando, usa un **20,3 % de CPU** y MAME ocupa **364 MB**. Para el mínimo privilegio, el usuario que juega no tiene `sudo`, se administra con otro usuario y SSH no admite root. Las incidencias (instalador, SSH y permisos de antiX) están documentadas con sus capturas.

Máquina recreativa (MAME)

> **En GitHub:** `recreativa/README.md`

Máquina virtual de VirtualBox que imita el equipo de la recreativa: **1 núcleo y 1 GB de RAM, sin swap**. Funciona con **antiX 26.1 core** y **MAME 0.276**. Al encenderla entra sola y abre el menú de MAME, sin escritorio y sin pedir contraseña.

Resultado: jugando a Gridlee usa un **20,3 % de CPU** y MAME ocupa **364 MB de RAM**. Ver `medidas/resultados.md`.

Contenido de la carpeta

Ruta	Qué es
`scripts/01-base.sh`	Puesta a punto tras instalar antiX: sudo, actualización y zona horaria
`scripts/02-ssh.sh`	Instalación de SSH, con el arreglo del grupo `_ssh`
`scripts/03-mame.sh`	Xorg mínimo + MAME + carpeta de ROMs
`scripts/04-optimizar.sh`	Optimización: `mame.ini`, autologin, arranque directo, GRUB y limpieza
`scripts/05-seguridad.sh`	Mínimos privilegios: usuario `adminrec`, `recreativa` sin `sudo`, permisos de antiX recortados y SSH sin root (aplicado el 27/10)
`scripts/medir.sh`	Recoge las medidas de rendimiento en un `.txt`
`config/`	Ficheros de configuración que acaban en la VM
`medidas/`	Resultados de las pruebas de rendimiento
`capturas/`	Capturas del proceso (ver el apartado 7)

> No se suben al repositorio contraseñas, ROMs ni el disco `.vdi`. La ROM de prueba (Gridlee) es legal y gratuita: se descarga de mamedev.org.

1. Máquina virtual

Ajuste	Valor
Nombre	Recreativa-MAME
Tipo	Linux / Debian (64-bit)
CPU	1 núcleo, límite de ejecución 100 %
RAM	1024 MB
Disco	10 GB VDI dinámico, una partición `sda1` ext4 con tabla MBR
Pantalla	VMSVGA, 16 MB, sin aceleración 3D
Red	NAT, reenvío **127.0.0.1:2222 → 22** para administrarla por SSH
USB	Controlador USB 1.1 (OHCI), para el arcade stick
Audio	ICH AC97

Instantáneas de VirtualBox, una por fase: `01 - antiX instalado y actualizado`, `02 - MAME sin optimizar` y `03 - optimizada`.

2. Instalación de antiX core

1. Arrancar la ISO de antiX 26.1 core y elegir **`antiX-26.1 x64-core (sysvinit)`**. El autoarranque se configura en `/etc/inittab`, que solo existe con sysvinit.
2. Entrar en el live como `root` y ejecutar `cli-installer`.
3. Responder así al instalador:
   - **Reparticionar:** sí. Aunque la pregunta sale en español, el instalador solo acepta `y` / `n` (con `s` dice «Inválido»).
   - **Tabla:** `dos` (MBR), porque la VM arranca en modo BIOS.
   - **Partición:** `sda1`, 10 GB, tipo Linux (83), marcada como arranque.
   - **Sistema de archivos:** ext4, sin `/home` aparte.
   - **GRUB:** en el **MBR** (opción 1). En la partición raíz, que es lo que viene por defecto, la VM no arranca.
   - **Idioma:** solo `es_ES.UTF-8`. **Teclado:** español.
   - **Equipo y usuario:** `recreativa`.
4. Al terminar, quitar la ISO (*Dispositivos → Unidades ópticas → Eliminar disco*) antes de reiniciar. Si no se quita, vuelve a arrancar el live.

3. Puesta a punto, SSH y MAME

Ejecutar los scripts por orden. El `01` va como root; el resto, como `recreativa`:

```
sh scripts/01-base.sh          # como root
sh scripts/02-ssh.sh
sh scripts/03-mame.sh
```

Para copiar la ROM desde el PC anfitrión:

```
scp -P 2222 gridlee.zip recreativa@127.0.0.1:~/mame/roms/
ssh -p 2222 recreativa@127.0.0.1
```

Para probar el juego: `startx /usr/games/mame gridlee`. Antes de optimizar, se toman las medidas «antes» con `sh scripts/medir.sh antes`.

4. Optimización

```
sh scripts/04-optimizar.sh
sudo reboot     # tiene que aparecer el menú de MAME directamente
```

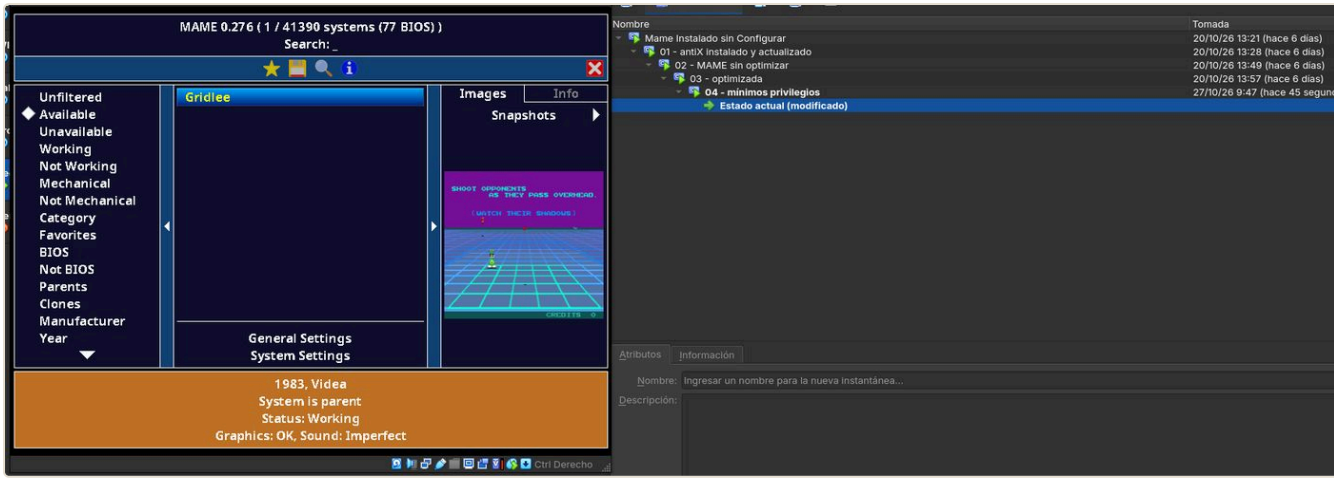
Cambio	Para qué
`video soft`	Dibujo por software: lo más estable sin aceleración 3D
`autoframeskip 1`	Si no llega a tiempo, salta fotogramas en vez de ir lento
`samplerate 22050`	Sonido más ligero para un solo núcleo
`skip_gameinfo 1`	Sin pantallas de aviso al cargar el juego
`window 0`	Pantalla completa
`joystick 1`	Mando activado, para el arcade stick
Autologin en tty1 (`/etc/inittab`)	Arranca sin pedir usuario ni contraseña
`.bash_profile` → `startx` → `.xinitrc`	Abre MAME directamente, sin escritorio
tty3 a tty6 desactivadas	4 procesos menos; tty2 queda como consola de emergencia (`Ctrl+Alt+F2`)
`GRUB_TIMEOUT=1`	Arranque más rápido

Cambio	Para qué
Sin swap	Requisito del profesor; la RAM sobra (ver medidas)

Front-end gráfico: el menú de MAME

Como front-end se usa la **interfaz propia de MAME**, que arranca sola al encender. No se instala un front-end aparte (Attract-Mode, EmulationStation…) porque cada uno ocupa RAM y CPU, y con 1 núcleo y 1 GB lo que sobra se deja para el juego.

Lo que pide un front-end	Cómo se cumple
Arranque directo, sin escritorio ni contraseña	Autologin en tty1 → `startx` → `.xinitrc` abre MAME
Selección de juego	Menú de MAME con la lista de juegos disponibles, filtros (*Available*, *Working*…) y búsqueda, manejable con el arcade stick
Pantalla completa	`window 0` en `mame.ini`
Información del juego	Panel lateral con la captura del juego, el año, el fabricante y el estado
Sin pantallas de aviso	`skip_gameinfo 1`
Volver al menú sin teclado	Mantener START 2 segundos en el arcade stick envía `Esc` (ver `../arcade-stick/`)



Menú de MAME a pantalla completa, con Gridlee seleccionado · GitHub: recreativa/capturas/rec_28_mame_e_instantanea_04.png

5. Incidencias resueltas

Incidencia	Causa	Solución
El instalador rechaza la respuesta `s`	Solo acepta respuestas en inglés	Responder `y`
Al reiniciar vuelve a salir el live	La ISO seguía montada	Quitarla en *Dispositivos* → *Unidades ópticas*
`openssh-server` no se instala (error de `dpkg`)	El script de `openssh-client` no consigue crear el grupo `_ssh`	`sudo groupadd -r _ssh && sudo dpkg --configure -a`
SSH: `Connection refused`	La regla de reenvío aún no estaba activa	Activarla en *Red* → *Avanzadas* → *Reenvío de puertos*
Aviso `XDG_RUNTIME_DIR`	Lanzar MAME por SSH	Es inofensivo

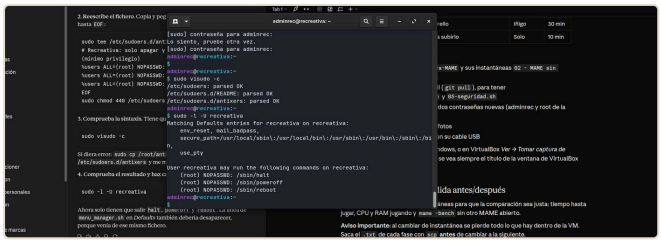
6. Seguridad (mínimos privilegios)

- MAME y la parte gráfica se ejecutan con el usuario `recreativa`, sin ser root (`xserver-xorg-legacy` + `config/Xwrapper.config`).

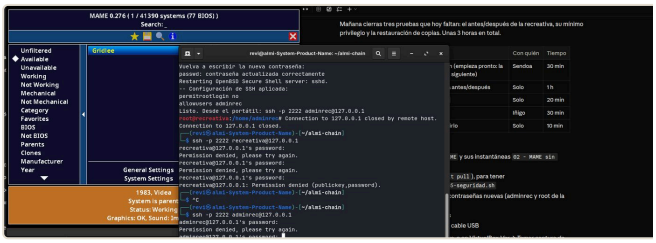
- SSH solo es accesible desde el propio anfitrión (reenvío NAT en 127.0.0.1), así que nadie de la red puede conectarse.
- Solo queda abierta una consola de emergencia (tty2), que pide usuario y contraseña.

Aplicado el 27/10/2026 con `scripts/05-seguridad.sh` (instantánea `04 - mínimos privilegios`):

Medida	Antes	Después	Captura
Administración	`recreativa` (el usuario que entra solo) tenía `sudo`	Usuario aparte `adminrec`, con `sudo` y contraseña en KeePassXC	rec_24
Permisos sin contraseña de antiX	`/etc/sudoers.d/antixers` daba al grupo `users` 13 comandos como root sin contraseña, entre ellos el instalador (`minstall`), `fdisk` y la red (`ceni`)	Solo `halt`, `poweroff` y `reboot`. Comprobado con `visudo -c`	rec_27
SSH	Entraba `recreativa`	`PermitRootLogin no` y `AllowUsers adminrec`: `recreativa` recibe *Permission denied*	rec_25 · rec_26
Contraseña de root	La del instalador	Nueva, en KeePassXC	rec_25
Arranque	—	Sigue entrando sola en MAME tras reiniciar	rec_28



Permisos sin contraseña de antiX · GitHub: recreativa/capturas/rec_27_sudo_solo_apagar_reiniciar.png



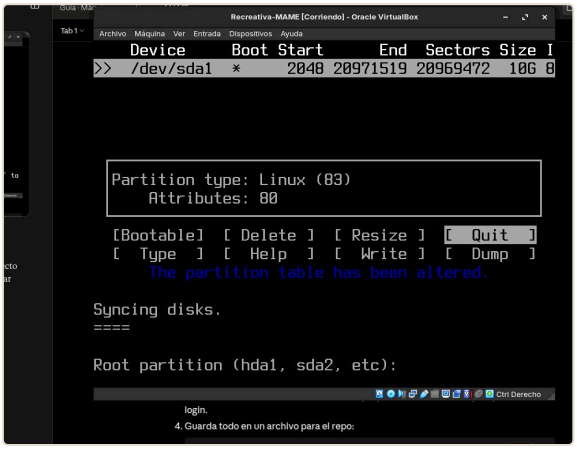
Entraba recreativa · GitHub: recreativa/capturas/rec_26_ssh_recreativa_rechazado.png

Incidencia: `Ctrl + Alt + F2` no cambia de consola dentro de VirtualBox, porque el anfitrión se queda la combinación. Se usa `Ctrl derecho + F2`, o se hace todo por SSH como se hizo aquí.

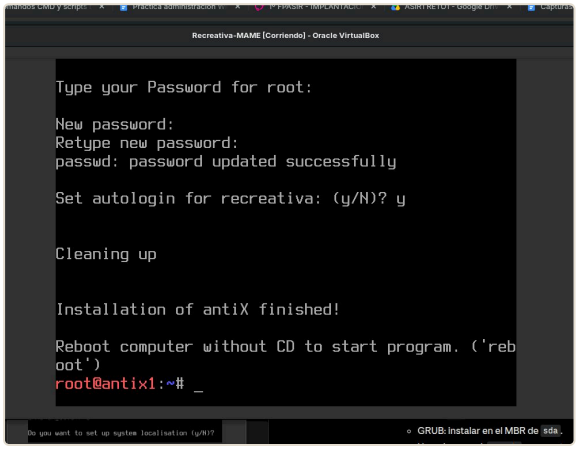
7. Capturas del proceso

Paso	Captura
Menú de arranque de la ISO: `antiX-26.1 x64-core (sysvinit)`	rec_01
Sesión live, como root, para lanzar `cli-installer`	rec_02
Disco nuevo: reparticionar	rec_03
Incidencia: la pregunta sale en español, pero solo acepta `y` / `n`	rec_04
Tabla de particiones `dos` (MBR), porque la VM arranca en modo BIOS	rec_05
Una partición `sda1` de 10 GB, Linux (83), de arranque	rec_06
ext4, sin `/home` aparte	rec_07
GRUB en el MBR (opción 1); en la partición raíz no arranca	rec_08
Solo el idioma `es_ES.UTF-8`	rec_09
Teclado español	rec_10
Fin de la instalación: equipo y usuario `recreativa`	rec_11

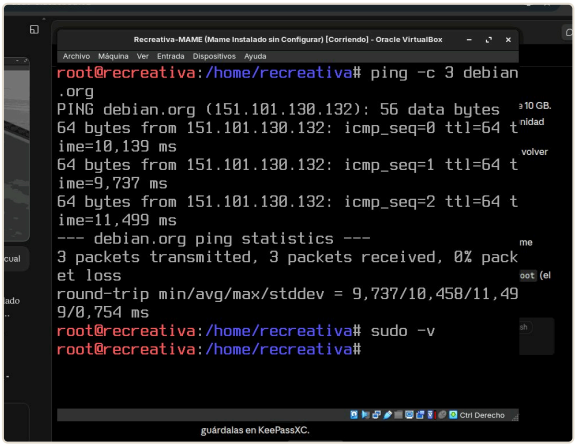
Paso	Captura
Incidencia: vuelve a salir la ISO porque seguía montada	rec_12
Red NAT funcionando (`ping -c 3 debian.org`)	rec_13
`sudo` , actualización y zona horaria `Europe/Madrid`	rec_14
Incidencia: falla la instalación de SSH (`dpkg` y `_ssh`)	rec_15
Dependencias de OpenSSH y arranque con sysvinit (`cat /proc/1/comm`)	rec_16



Una partición sda1 de 10 GB, Linux (83), de arranque · GitHub: recreativa/capturas/rec_06_particion_sda1.png



Fin de la instalación: equipo y usuario recreativa · GitHub: recreativa/capturas/rec_11_fin_instalacion.png



Red NAT funcionando (ping -c 3 debian.org) · GitHub: recreativa/capturas/rec_13_red_ping.png

El funcionamiento y el rendimiento los validó el profesor (ver `medidas/resultados.md`); el juego con el arcade stick está en `../arcade-stick/` . Las capturas de mínimo privilegio son las `rec_24` a `rec_28` del apartado 6.

Relacionado

- Arcade stick con Arduino (tareas 44 y 45) → carpeta `../arcade-stick/`

Sacar los ficheros reales de la VM

Desde el portátil, en la raíz del repo y con la VM encendida:

```
scp -P 2222 recreativa@127.0.0.1:.mame/mame.ini recreativa/config/mame.ini
scp -P 2222 recreativa@127.0.0.1:/etc/inittab recreativa/config/inittab
scp -P 2222 'recreativa@127.0.0.1:medidas-*.txt' recreativa/medidas/
# Desde la instantánea 04, recreativa ya no entra por SSH: usa adminrec@127.0.0.1:/home/recreativa/...
git add recreativa && git commit -m "feat(recreativa): ficheros reales de la VM" && git push
```

Pruebas de rendimiento · Recreativa MAME

> **En GitHub:** `recreativa/medidas/resultados.md`

VM con 1 núcleo y 1 GB de RAM (965 MB útiles), sin swap. Medidas tomadas con `uptime`, `free -m`, `ps`, `df -h`, `sar` (paquete sysstat) y `mame -bench`, mediante `../scripts/medir.sh`.

1. Resultado: el juego va sobrado con 1 núcleo y 1 GB

Medido con la versión optimizada y **Gridlee en marcha**:

Medida	Resultado	Qué significa
CPU media (`sar`, 60 s)	**20,3 %** (14,75 % usuario y 5,42 % sistema)	Queda un 79,7 % de CPU libre
RAM de MAME (RSS)	**364 MB**	Cabe de sobra en 1 GB
RAM disponible	575 MB	Sin necesidad de swap
Swap	0	Requisito del profesor: no se usa
`mame -bench 60 gridlee`	**1353–1613 %**	Emula entre 13 y 16 veces más rápido de lo necesario (100 % = velocidad de la máquina original)
Juego	Sin tirones ni cortes de sonido	Prueba de uso real

2. Estado del sistema en cada fase

Estas dos columnas **no son una comparación directa**: se tomaron en momentos distintos (la primera sin MAME abierto y la segunda con el menú de MAME abierto). Sirven para ver cuánto ocupa cada parte.

Medida	Sistema base, sin MAME abierto	Optimizada, con el menú de MAME abierto
Tiempo desde el arranque (`uptime`)	52 s	62 s
RAM usada	162 MB	389 MB (320 de caché)
Procesos	85	92 (X y MAME suman procesos; hay 4 consolas menos)
Servicios activos	5 (dbus, opensysusers, seatd, ssh, udev)	Los mismos 5
Disco usado	2,3 GB (25 %)	3,0 GB (33 %, por MAME y su ROM)

La diferencia de RAM es MAME funcionando, no un empeoramiento. El sistema base ya era mínimo (antiX core con 5 servicios), así que la optimización no se centra en quitar servicios, sino en preparar la máquina para jugar.

3. Qué se ha optimizado

Cambio	Efecto
Arranque directo a MAME (autologin, `startx` y `.xinitrc`)	Al encender, la máquina va sola al juego, sin escritorio ni usuario y contraseña
4 consolas de texto desactivadas (tty3–tty6)	4 procesos menos
`GRUB_TIMEOUT=1`	Arranque más rápido
`video soft` y `autoframeskip`	Vídeo estable sin aceleración 3D y sin ralentizaciones
`samplerate 22050`	Sonido más ligero para un solo núcleo
`skip_gameinfo`	Sin pantallas de aviso al cargar el juego

Cambio	Efecto
Sin swap	La RAM sobra, como muestra el apartado 1

4. Validación

✅ **Validado por el profesor el 29/10/2026:** las medidas de los apartados 1 y 2 son suficientes como constancia de las pruebas de rendimiento. No se repite la comparación con las instantáneas `02` y `03`.

✅ **Confirmado de nuevo el 04/11/2026 con la rúbrica final** (nivel 5 del criterio de la recreativa: «monitoriza el rendimiento con herramientas adecuadas y deja constancia documentada de las pruebas»).

Si se quisiera ampliar, se mediría lo mismo en las dos instantáneas: el tiempo desde encender hasta poder jugar, `sh medir.sh antes-jugando` / `despues-jugando` con Gridlee en marcha y `mame -bench 60 gridlee` sin otro MAME abierto.

5. Cómo interpretarlo (para la defensa)

- **El requisito se cumple:** con 1 núcleo y 1 GB, Gridlee usa el 20 % de la CPU y 364 MB, sin swap ni tirones.
- `mame -bench` mide la emulación sin vídeo ni sonido. Más del 100 % significa que va más rápido que la máquina original. El valor más bajo (1353 %) se tomó con otra instancia de MAME abierta, por eso no se usa para comparar.
- **La mejora principal es de uso:** la máquina arranca sola en el juego, con vídeo y sonido ajustados a un solo núcleo. Al encender, la máquina va sola al juego sin pasar por escritorio ni contraseña.

Los ficheros en bruto (`medidas-*.txt`) se añaden a esta carpeta al sacarlos de la VM.

11. Arcade stick con Arduino

Responsable: Jon

> **Dónde está en GitHub**
>
Guía del arcade stick (versión final)	`arcade-stick/README.md`
Código de la versión final	`arcade-stick/mejora-leds-altavoz/mando_arcade_leds/mando_arcade_leds.ino`
Vídeo de las luces y el START	`arcade-stick/mejora-leds-altavoz/capturas/prueba_luces_start.mp4`
Vídeos de las pruebas	`arcade-stick/capturas`

Resumen

El arcade stick usa un **Arduino Leonardo**, que tiene USB nativo y el ordenador reconoce como un **teclado USB, sin drivers**. Por eso funciona en MAME y en cualquier otro programa. Se eligió frente al *encoder* USB del kit porque el *encoder* tiene la asignación de botones fija, y el enunciado pide input personalizable. La **versión final** tiene un joystick, 4 botones y un botón **START** en el lateral, que envían las teclas de MAME (flechas, disparo, moneda, pausa, salir y *start*). Además lleva **tres luces** en la tapa (un LED RGB, uno verde y uno rojo) y un **altavoz**: al conectarlo se encienden las luces y suena una melodía, cada botón da un pitido, y START enciende un modo de colores en bucle y lo apaga con un aviso en rojo. El código usa las resistencias internas para los botones (`INPUT_PULLUP`) y resistencias de 220 Ω para los LED, tiene antirrebote de 10 ms, controla las

luces con `millis()` para no bloquear los botones y guarda todas las teclas en una sola tabla, así que cambiar un control es cambiar un valor. También se puede remapear desde el propio emulador sin reprogramar. Cuesta 58,45 € entre el Arduino y el kit; las luces, el altavoz y las resistencias son material del aula.

Arcade stick con Arduino

> **En GitHub:** `arcade-stick/README.md`

Mando arcade controlado por un **Arduino Leonardo** que el ordenador reconoce como un **teclado USB**. No necesita drivers ni configuración, así que funciona en cualquier emulador y en cualquier sistema operativo. Las teclas se pueden cambiar en el código o en el propio emulador.

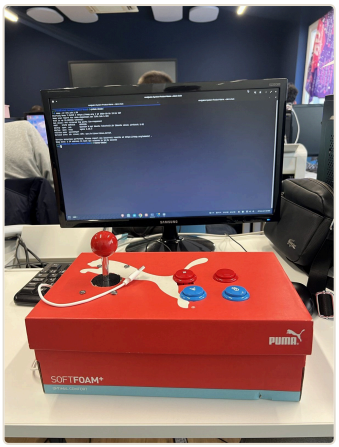
La **versión final** tiene:

- un joystick de 4 direcciones y 4 botones (B1 a B4);
- un botón **START** en el lateral de la caja;
- tres luces en la tapa: un **LED RGB** (izquierda), un **LED verde** (centro) y un **LED rojo** (derecha);
- un **altavoz**, que toca una melodía al conectar el mando y da un pitido distinto con cada botón.

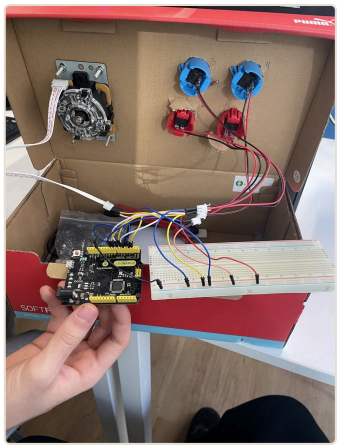


Arcade stick final, con el LED RGB y el verde encendidos · GitHub: arcade-stick/img/mando_final_luces.jpg

Archivo	Qué es
`mejora-leds-altavoz/mando_arcade_leds/mando_arcade_leds.ino`	**Código de la versión final** (el que lleva el mando)
`mejora-leds-altavoz/img/montaje_leds_altavoz_boton.png`	Esquema de conexiones de las luces, el altavoz y el botón START
`img/montaje_desde_arriba.png`	Esquema de conexiones del joystick y los botones
`img/mando_final_luces.jpg`	Foto del mando final con las luces encendidas
`img/mando_terminado.jpg` · `img/mando_fisico.jpg`	Fotos del mando por fuera y del montaje por dentro
`mejora-leds-altavoz/capturas/prueba_luces_start.mp4`	Vídeo de la prueba de las luces y el START
`capturas/`	Vídeos y fotos de las pruebas del joystick y los botones
`mando_arcade/mando_arcade.ino`	Versión 1: solo joystick y botones (ver apartado 11)
`mejora-leds-altavoz/README.md`	Diario de la mejora del 3 y 4 de noviembre



Fotos del mando por fuera y del montaje por dentro · GitHub: arcade-stick/img/mando_terminado.jpg



Fotos del mando por fuera y del montaje por dentro · GitHub: arcade-stick/img/mando_fisico.jpg



▶ **Vídeo:** Vídeo de la prueba de las luces y el START · verlo en GitHub (o con el QR): arcade-stick/mejora-leds-altavoz/capturas/prueba_luces_start.mp4

1. Qué hace

Control	Pin	Tecla que envía	Para qué (en MAME)	Además
Joystick ↑ ↓ ← →	3, 4, 5, 2	Flechas	Mover	—
B1 (rojo)	9	Ctrl izquierdo	Disparar	Pitido
B2 (rojo)	8	`5`	Moneda	Pitido
B3 (azul)	7	`P`	Pausa	Pitido
B4 (azul)	6	`Esc`	Salir	Pitido
START (lateral)	A0	`1`	*Start*	Pitido y luces

- **Al conectar el USB:** se encienden las tres luces y suena una melodía corta. Sirve para comprobar que todo funciona y, de paso, da 2 segundos de margen para volver a subir el sketch.
- **Primera pulsación de START:** envía la tecla `1` y activa el modo fiesta: el LED RGB cambia de color en bucle y el verde se queda encendido.
- **Segunda pulsación de START:** se apagan el RGB y el verde, y el rojo se enciende durante 1 segundo.

2. Material y presupuesto

Pieza	Cantidad	Para qué	Precio (presupuesto)
Arduino Leonardo (ATmega32U4)	1	Lee los botones y hace de teclado USB	25,40 €
Kit arcade de 1 jugador: joystick de 4 direcciones (5 hilos), pulsadores y cableado	1	Joystick y botones B1 a B4	33,05 €
LED RGB de cátodo común (4 patas)	1	Colores en bucle	Material del aula
LED verde y LED rojo de 5 mm	1 + 1	Mando encendido / aviso de apagado	Material del aula

Pieza	Cantidad	Para qué	Precio (presupuesto)
Altavoz de placa base (conector SPEAKER)	1	Melodía y pitidos	Material del aula
Pulsador	1	START en el lateral	Material del aula
Resistencias de 220 Ω	6	Una por cada color del RGB, una por LED y una para el altavoz	Material del aula
Protoboard, cables dupont y cable micro-USB	—	Conexiones y GND común	Material del aula
Total			**58,45 €**

Los precios son los del presupuesto. En el aula se montó con un Leonardo de Keyestudio (compatible) y el material disponible.

3. Por qué un Leonardo y por qué como teclado

3.1 Frente a las alternativas comerciales

El arcade stick es un pequeño **sistema embebido**: un microcontrolador que lee los pulsadores, se comunica con el PC por USB y controla luces y sonido.

Opción	Ventajas	Inconvenientes	¿Cumple el enunciado?
Mando USB comercial	Se enchufa y funciona	No tiene joystick arcade y no se puede reprogramar	No: el input no es personalizable
Encoder USB del kit (*zero delay*)	Viene en el kit y no hay que programarlo	La asignación de botones la fija el fabricante, y no puede manejar luces ni sonido	Solo en parte
Raspberry Pi	Muy potente	Es un ordenador entero: más caro, tarda en arrancar y es demasiado para leer 9 entradas	Sí, pero sobredimensionado
Arduino Leonardo (elegido)	USB nativo (HID), sin drivers. Barato (≈ 25 €). Las teclas están en una tabla del código. Le sobran pines para luces y altavoz	Hay que cablearlo y programarlo	**Sí**: personalizable, documentado y versionado
Arduino Uno	Muy común en clase	Sin USB nativo: no puede hacer de teclado sin cambiar el firmware	No sin modificarlo

La justificación completa está en el documento de digitalización (tecnologías habilitadoras: IoT y sistemas embebidos).

3.2 Por qué como teclado

- **Leonardo (chip ATmega32U4):** tiene USB nativo, así que puede presentarse al ordenador como un dispositivo de entrada (HID). Un Arduino Uno no puede hacerlo sin cambiar su firmware.
- **Como teclado, y no como mando:** la biblioteca `Keyboard.h` viene incluida en el IDE de Arduino. Todos los sistemas y todos los emuladores aceptan un teclado sin drivers. Para un mando arcade, que solo tiene entradas digitales (pulsado o no), un teclado no pierde nada respecto a un mando USB.

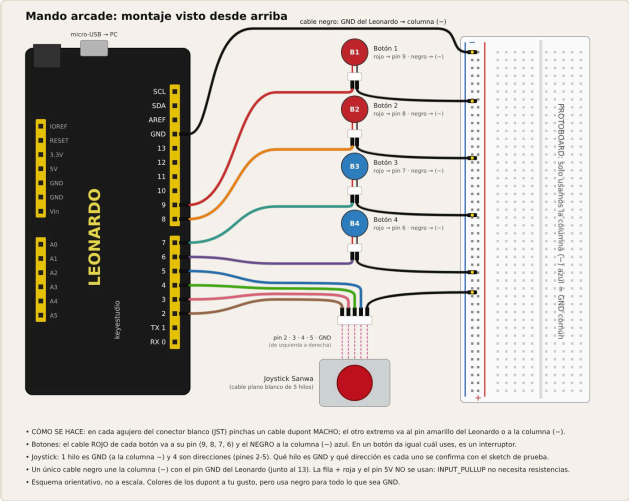
3.3 Input personalizable

1. **Sin reprogramar:** en el emulador, cada uno asigna las teclas a lo que quiera, por ejemplo en MAME con *Tab → Input (este juego)* o en RetroArch con *Ajustes → Entrada*.

2. **En el código:** todas las teclas están en una sola tabla, `teclas[]`, y los pines en `pines[]`. Cambiar un control es cambiar un valor de la tabla, sin tocar la lógica.

4. Conexiones

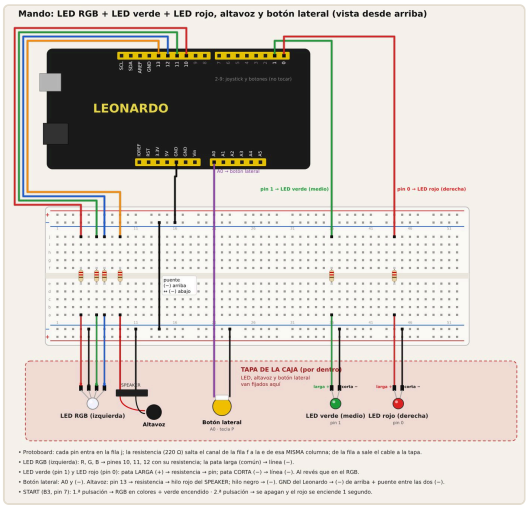
4.1 Joystick y botones



Esquema de conexiones del joystick y los botones · GitHub: arcade-stick/img/montaje_desde_arriba.png

- Cada entrada va a **su pin** (tabla del apartado 1) por un lado y a **GND** por el otro. Todas las GND se juntan en la línea (−) de la protoboard, y un único cable negro la une al GND del Leonardo.
- No hacen falta resistencias ni 5V: el código activa la resistencia interna de cada pin (`INPUT_PULLUP`). El botón START, en A0, funciona igual.
- En el joystick, qué hilo es GND y qué dirección es cada uno se comprobó con el propio sketch.

4.2 Luces, altavoz y botón START



Esquema de conexiones de las luces, el altavoz y el botón START · GitHub: arcade-stick/mejora-leds-altavoz/img/montaje_leds_altavoz_boton.png

Elemento	Pin	Detalle
LED RGB rojo / verde / azul	10 / 11 / 12	Cada uno con su resistencia. La pata larga (común) va a (−)
LED verde	1	Pin → resistencia → pata larga (+). La pata corta va a (−)
LED rojo	0	Pin → resistencia → pata larga (+). La pata corta va a (−)

Elemento	Pin	Detalle
Altavoz	13	Pin → resistencia → hilo rojo. El hilo negro va a (–)
Botón START	A0	Un hilo a A0 y el otro a (–)

- Los LED, el altavoz y el botón van fijados en la tapa. En la protoboard solo están las resistencias: cada pin entra por la fila j, la resistencia salta el canal central (de la fila f a la e) y de la fila a sale el cable hacia la tapa.
- **Por qué la resistencia:** un LED casi no tiene resistencia propia. Sin ella, el pin daría más de los 40 mA que aguanta. Con 220 Ω pasan unos 14 mA.
- **La pata larga cambia de sitio:** en los LED de un color, la larga es el (+) y va hacia el pin. En el RGB de cátodo común, la larga es la común y va a (–).

5. Cómo funciona el código

1. **`INPUT_PULLUP`:** sin pulsar, el pin lee `HIGH`. Al pulsar, el botón une el pin con GND y lee `LOW`.
2. **Bucle:** recorre las 9 entradas sin parar. Si una cambia de estado, envía `Keyboard.press()` al pulsar y `Keyboard.release()` al soltar. Así se pueden mantener varias a la vez (por ejemplo, diagonal + disparo).
3. **Antirrebote de 10 ms:** un pulsador mecánico «rebota» y da varios cambios en milisegundos. Solo se acepta un cambio si han pasado 10 ms desde el anterior.
4. **Luces y sonido sin `delay()`:** el RGB, el LED rojo y los pitidos se controlan con `millis()`, así que el joystick y los botones siguen respondiendo mientras el RGB cambia de color.
5. **Ajustes al principio del código:** `ANODO_COMUN = true` si el RGB es de ánodo común; `ZUMBADOR_ACTIVO = true` si se usa un zumbador activo en lugar del altavoz; `SONIDO = false` para silenciar los pitidos.

6. Pasos para reproducirlo

1. Monta las conexiones del apartado 4.
2. En el IDE de Arduino: *Herramientas → Placa → Arduino Leonardo* y elige su puerto.
3. Abre `mejora-leds-altavoz/mando_arcade_leds/mando_arcade_leds.ino` y súbelo.
4. Al terminar de subirse, se encienden las tres luces y suena la melodía.
5. Pruébalo en cualquier programa: cada control tiene que escribir su tecla y el START tiene que encender las luces.
6. **En la recreativa (VirtualBox):** con la VM apagada, *Configuración → USB → USB 1.1 (OHCI)* y añade un filtro para el dispositivo *Arduino Leonardo*. Al arrancar, la VM lo recibe como teclado. Con USB 1.1 basta, porque un teclado es un dispositivo de baja velocidad.
7. **En MAME:** las teclas ya son las de MAME (flechas, Ctrl, `5`, `1`, `P` y `Esc`), así que se juega sin configurar nada. Para cambiar un control: *Tab → Input (este juego)*.

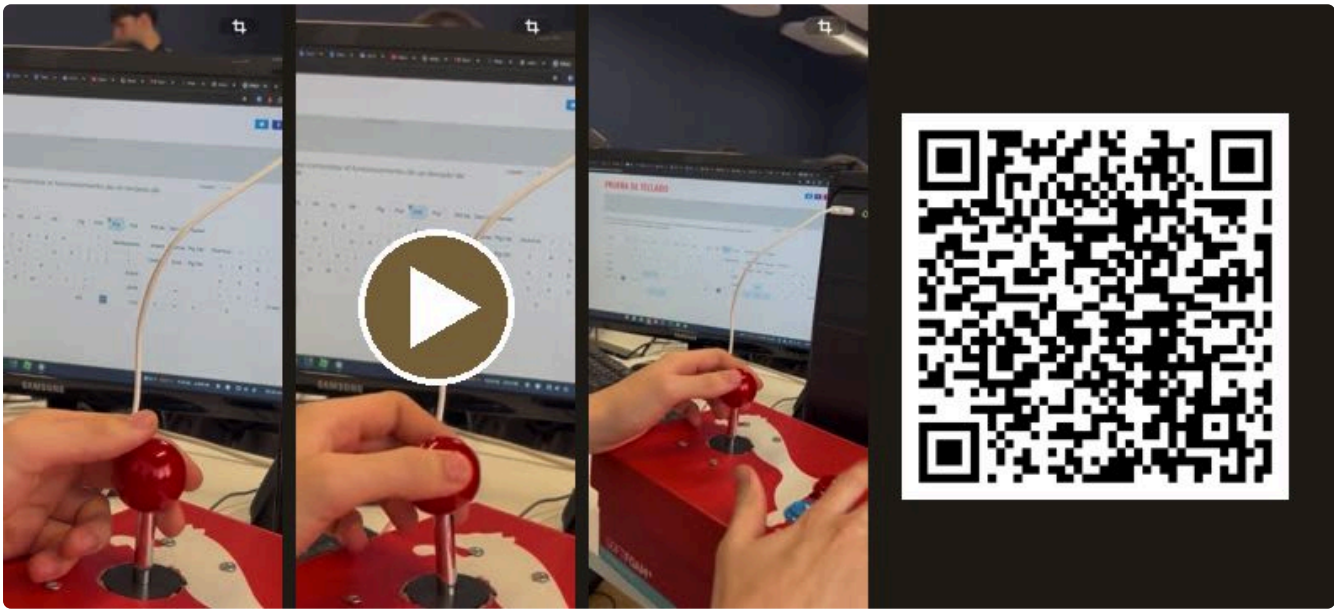
7. Seguridad y mínimo privilegio

- Es un dispositivo de clase **HID estándar**: no instala drivers ni necesita permisos de administrador en el equipo.
- En la recreativa, MAME lo usa con el usuario `recreativa`, sin root.
- Solo envía las 9 teclas de su tabla. No puede ejecutar nada por su cuenta.

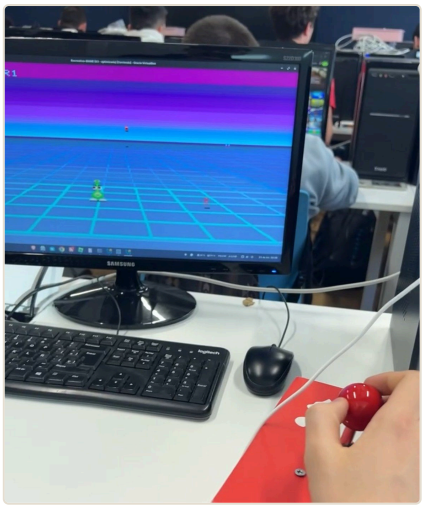
8. Pruebas

#	Prueba	Resultado esperado	Estado	Evidencia
1	Revisión del montaje y funcionamiento por el profesor	Funciona	✅ Validado por el profesor	—

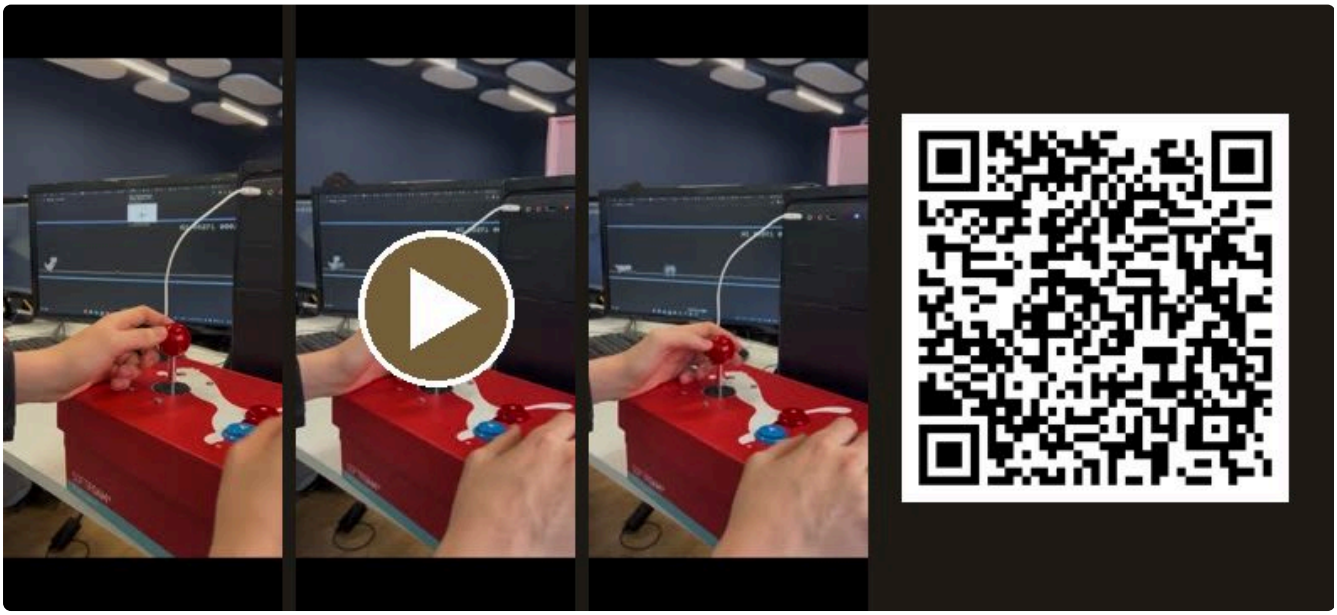
#	Prueba	Resultado esperado	Estado	Evidencia
2	Probador de teclado en el navegador: pulsar todas las entradas	Cada entrada marca su tecla	✅ 26/10 (versión 1): se marcan las 4 flechas y Z, X, C y V	vídeo
3	Jugar a Gridlee en la VM recreativa (instantánea `03 - optimizada`)	El juego responde al joystick y a los botones	✅ Validado por el profesor en clase	ver captura «Gridlee con el arcade stick» debajo
4	Otro programa: el juego del dinosaurio de Chrome (`chrome://dino`)	Responde igual, sin configurar drivers	✅ 26/10: arriba salta y abajo se agacha	vídeo
5	**Cada control por separado:** las 4 direcciones y B1 a B4 en un probador de teclado	Cada uno marca su tecla	✅ 04/11: cada control se ilumina en el probador	vídeo
6	**Otro emulador: RetroArch** (versión web), sin configurar drivers	El joystick y los botones responden en un juego	✅ 04/11: se juega a un *shooter* en primera persona moviéndose y disparando con el stick	vídeo
7	Conectar el USB (versión final)	Se encienden las 3 luces y suena la melodía	✅ 04/11	—
8	Pulsar START	El RGB cambia de colores y el verde se enciende	✅ 04/11	vídeo
9	Pulsar START otra vez	Se apagan el RGB y el verde, y el rojo se enciende 1 s	✅ 04/11	vídeo



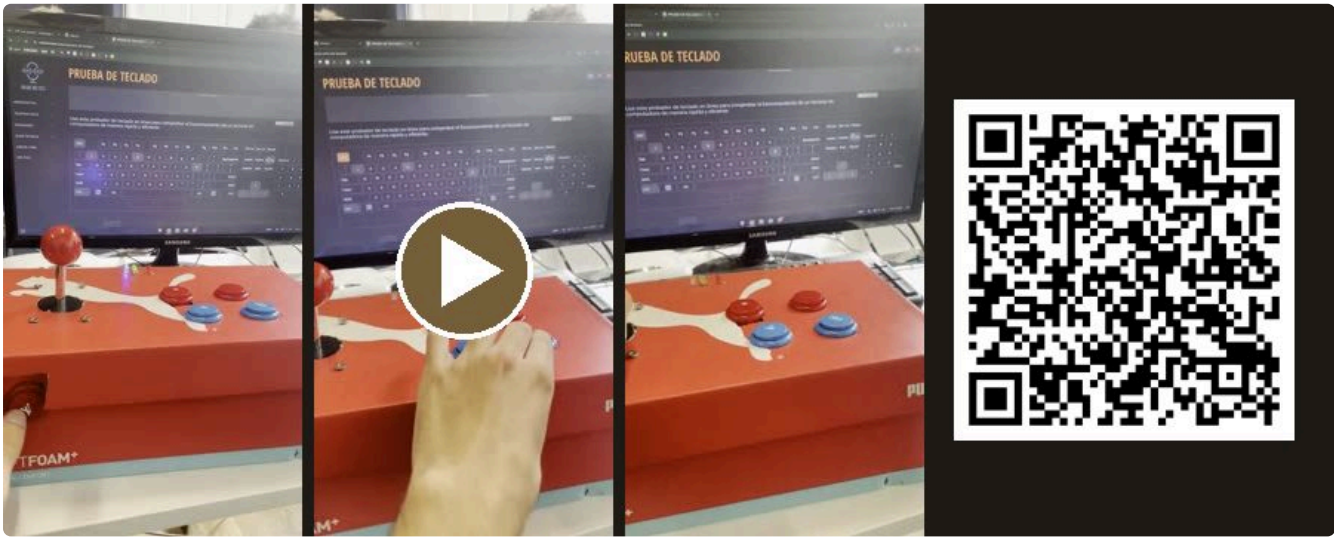
▶ **Vídeo:** Probador de teclado en el navegador: pulsar todas las entradas · verlo en GitHub (o con el QR): arcade-stick/capturas/stick_01_probador_teclado.mp4



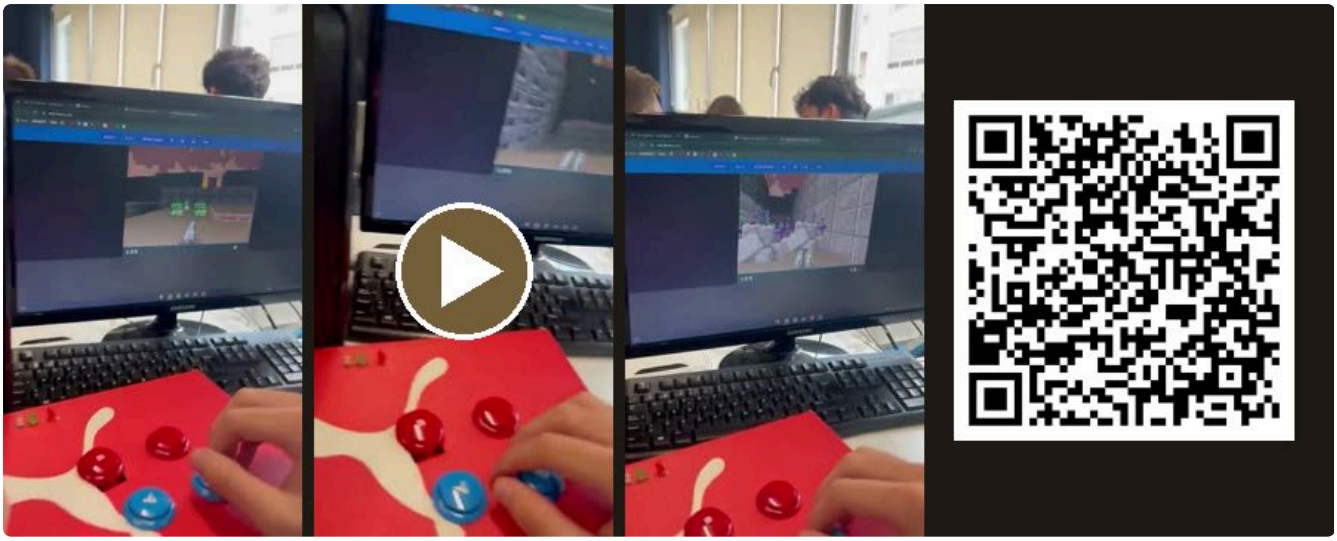
Gridlee con el arcade stick · GitHub: arcade-stick/capturas/stick_03_mame_gridlee.jpg



▶ **Vídeo:** Otro programa: el juego del dinosaurio de Chrome (chrome://dino) · verlo en GitHub (o con el QR): arcade-stick/capturas/stick_04_chrome_dino.mp4



▶ **Vídeo:** Cada control por separado: las 4 direcciones y B1 a B4 en un probador de teclado · verlo en GitHub (o con el QR): arcade-stick/capturas/stick_07_cada_control.mp4



▶ **Vídeo:** Otro emulador: RetroArch (versión web), sin configurar drivers · verlo en GitHub (o con el QR): arcade-stick/capturas/stick_08_retroarch.mp4

9. Problemas encontrados y soluciones

Problema	Causa	Solución
`Keyboard` subrayado en rojo al compilar	Placa no seleccionada como Leonardo o biblioteca sin instalar	*Herramientas → Placa → Arduino Leonardo* e instalar `Keyboard` desde el gestor de bibliotecas
Error `butterfly_recv(...) failed` al subir	El Leonardo no entra a tiempo en el cargador; en Ubuntu, ModemManager ocupa el puerto `ttyACM`	Pulsar RESET dos veces al ver «Uploading...», revisar el puerto y `sudo systemctl stop ModemManager`
Del joystick solo funcionaba «derecha»	El hilo GND del joystick estaba en el pin 2 y una dirección en la línea (–)	Intercambiar esos dos cables
Arriba e izquierda cambiadas	Los microinterruptores del joystick están girados respecto a la palanca	Intercambiar 3 y 5 en `pines[]`, sin tocar cables
En Gridlee ningún botón disparaba (versión 1)	MAME dispara con Ctrl izquierdo y los botones enviaban `Z`, `X`, `C`, `V`	En la versión final, B1 envía `KEY_LEFT_CTRL`
Dos LED RGB en A0-A5 no se encendían	Lo más probable, la línea (–) partida de la protoboard o la pata común en otra fila	Se cambiaron por un LED verde (pin 1) y uno rojo (pin 0)
El altavoz no sonaba	Sus dos hilos estaban en la línea (+)	Hilo negro a la línea (–)
El altavoz suena bajo	Un pin de Arduino da poca corriente	Tonos agudos. Para más volumen: un transistor BC547/2N2222 o un zumbador activo
El botón START, en el pin 0, chocaba con el LED rojo	Hacía falta el pin 0 para el LED	Se movió a A0

10. Preguntas que pueden hacer en la defensa

- **¿Por qué no un Arduino Uno?** Porque no tiene USB nativo y no puede hacerse pasar por teclado sin cambiar su firmware.
- **¿Por qué los botones no llevan resistencia y los LED sí?** Los botones usan la resistencia interna del chip (`INPUT_PULLUP`). Los LED necesitan una externa para limitar la corriente a unos 14 mA.
- **¿Por qué pulsado es `LOW`?** Porque el botón une el pin con GND, y sin pulsar la resistencia interna lo mantiene en `HIGH`.
- **¿Qué es el antirrebote?** Ignorar los cambios de menos de 10 ms que produce el contacto mecánico al pulsar.

- **¿Por qué `millis()` y no `delay()`?** Porque `delay()` para el programa: mientras el RGB cambiara de color, el mando no leería los botones.
- **¿Cómo cambio un botón?** En la tabla `teclas[]` del código o en el menú de entradas del emulador.

11. Versión 1 (historial)

La primera versión, del 26 de octubre, solo tenía el joystick y los 4 botones, que enviaban `Z`, `X`, `C` y `V`. Su código sigue en `mando_arcade/mando_arcade.ino` y con ella se hicieron las pruebas 2 y 4. El 3 y el 4 de noviembre se añadieron las luces, el altavoz y el botón START, y se cambiaron las teclas a las de MAME: es la versión final, descrita arriba. El diario de esa mejora está en `mejora-leds-altavoz/README.md`.

12. Digitalización

Responsable: Jose

> **Dónde está en GitHub**
>
> | | |
> |---|---|
> | Documento de digitalización | `docs/digitalizacion/digitalizacion.md` |

Resumen

La solución usa varias tecnologías habilitadoras digitales: un ERP (Odoo), análisis de datos (PostgreSQL), la nube (AWS), IoT con Arduino y ciberseguridad. Con ellas el restaurante cobra antes, controla el stock al momento, se da a conocer en internet y protege su información frente a averías y cortes de luz.

Cada tecnología se eligió comparándola con otras opciones: AWS frente a Azure, Google Cloud y un hosting compartido; Odoo frente a SAP Business One, Holded y un TPV suelto; PostgreSQL frente a MySQL y SQL Server; y Arduino Leonardo frente a un encoder USB comercial y una Raspberry Pi. Los motivos principales son el coste (software libre y Learner Lab gratis), que Odoo solo funciona con PostgreSQL y que el Leonardo se reconoce como teclado USB sin drivers.

Digitalización del restaurante

> **En GitHub:** `docs/digitalizacion/digitalizacion.md`

Responsable	Fechas (Gantt)	Carpeta en el repo
Jose	5/11 – 9/11	`docs/`

1. Qué es la digitalización

Digitalizar una empresa no es solo usar ordenadores. Es cambiar la forma de trabajar gracias a la tecnología, para hacerlo más rápido, con menos errores y llegando a más clientes.

Las **tecnologías habilitadoras digitales** son las que hacen posible ese cambio: la nube, el IoT, la ciberseguridad, los sistemas de gestión (ERP), el análisis de datos o la inteligencia artificial.

2. Cómo trabajaba el restaurante antes

Sin el sistema que proponemos, el restaurante funcionaría así:

- **Cobro:** el camarero apunta la comanda en papel, suma la cuenta a mano o con una caja registradora y cobra con el datáfono o en efectivo, según el cliente. Es lento y es fácil equivocarse al sumar o dejarse algo sin cobrar.
- **Control del negocio:** el gerente revisa el inventario contando a mano lo que hay en el almacén y junta los tickets del día para saber cuánto se ha vendido. Le lleva tiempo, la información llega tarde y no sabe con facilidad qué plato se vende más o cuándo hay que pedir género.
- **Clientes nuevos:** el restaurante solo se da a conocer por el boca a boca o pagando publicidad (carteles, folletos, prensa). Sin web, quien busca dónde comer en internet no lo encuentra.

Además, toda la información está en papel o en un solo sitio: si se pierde o se estropea, no hay copia.

3. Tecnologías habilitadoras del proyecto

Tecnología	Dónde está en el proyecto	Problema que resuelve
Sistema de gestión (ERP)	Odoo con su TPV	Cobro lento, inventario a mano
Datos	PostgreSQL, consultas de la BD	No saber qué se vende ni cuándo pedir
Nube	Web en AWS	Sin web, los clientes no te encuentran
IoT / sistemas embebidos	Arduino del arcade stick	Ocio para clientes, mando personalizable
Ciberseguridad	HTTPS, cortafuegos, usuarios sin permisos, copias	Pérdida o robo de la información

ERP (Odoo). Es un solo programa para cobrar, facturar, controlar el almacén, el personal y los clientes. El camarero cobra en el TPV táctil, la factura sale automáticamente por la impresora de tickets y **el inventario se descuenta solo** con cada venta. El gerente ya no tiene que contar a mano: ve el stock al momento y sabe cuándo pedir.

Datos (PostgreSQL). Todo lo que se vende queda guardado en la base de datos. Con consultas se puede saber qué plato se vende más, qué día hay más clientes o qué local factura más, y tomar decisiones con datos reales en vez de a ojo.

Nube (AWS). La web está en un servidor de Amazon, disponible las 24 horas aunque pase algo en el local, sin comprar ni mantener un servidor propio (unos 10 € al mes). Muestra la carta, los platos y dónde están los locales con un mapa, así que el restaurante se da a conocer sin pagar publicidad.

IoT y sistemas embebidos (Arduino). El arcade stick de la recreativa lleva un Arduino Leonardo que el ordenador reconoce como un teclado. Es un pequeño ejemplo de cómo un aparato físico se conecta y se programa para hacer lo que necesita el negocio: aquí, ofrecer ocio a los clientes mientras esperan.

Ciberseguridad. Al digitalizar, la información pasa a ser lo más valioso del restaurante. Por eso la web va por HTTPS, el servidor tiene cortafuegos, los TPV y el gerente usan cuentas sin permisos de administrador, los discos están en espejo (RAID 1) y se hacen copias de seguridad cifradas.

4. Alternativas que comparamos y por qué elegimos cada una

Para cada tecnología miramos otras opciones y elegimos según lo que necesita el restaurante: **poco dinero** (es una cadena pequeña), **que funcione aunque falle algo**, **que lo podamos montar y mantener nosotros** y **que no dependa de licencias de pago**.

Tecnología	Elegida	Alternativas	Por qué la elegida
Nube (web)	**AWS** (EC2 + S3, montado con Terraform)	Microsoft Azure, Google Cloud, hosting compartido	El centro nos da **AWS Academy Learner Lab gratis** y es lo que vemos en clase. Con EC2 controlamos el servidor entero (Nginx, HTTPS, cortafuegos) y con S3 guardamos la copia de seguridad fuera del local. Con Terraform el servidor se vuelve a crear con un solo comando.
ERP y TPV	**Odoo Community**	SAP Business One, Holded, un TPV suelto	Es **libre y sin licencias**, trae el **TPV integrado** y se le añaden módulos (inventario, facturación, empleados) según se necesitan. Además lo instalamos en nuestro servidor, así que los datos son del restaurante.
Base de datos	**PostgreSQL**	MySQL / MariaDB, Microsoft SQL Server	**Odoo solo funciona con PostgreSQL**, así que es la opción obligada. Además es libre, muy fiable y es la que **recomienda el profesor**.
Sistema embebido (recreativa)	**Arduino Leonardo**	Encoder USB comercial, Raspberry Pi	Su chip tiene **USB nativo (HID)**: el ordenador lo ve como un teclado sin drivers. Es **personalizable** (las teclas se cambian en el código) y **barato** (unos 25 €).

Nube: AWS frente a Azure, Google Cloud o hosting compartido. Azure y Google Cloud ofrecen lo mismo que AWS (servidores virtuales y almacenamiento) a precios parecidos, así que técnicamente valdrían. Elegimos AWS porque el centro nos da el Learner Lab sin coste y es la nube que estudiamos. Un hosting compartido es más barato, pero no deja configurar el servidor (ni el cortafuegos ni las cabeceras de seguridad), no se puede automatizar con Terraform y no ofrece un sitio como S3 para las copias.

ERP: Odoo frente a SAP Business One, Holded o un TPV suelto. SAP Business One es muy completo pero está pensado para empresas más grandes: tiene licencias de pago por usuario y necesita un instalador oficial, demasiado caro para un restaurante. Holded es un programa español en la nube muy fácil de usar, pero se paga una cuota mensual y los datos están en sus servidores, no en los nuestros. Un TPV suelto solo cobra: el inventario y la facturación seguirían en papel o en otro programa, que es justo el problema que queremos resolver. Odoo junta todo en un solo programa y es gratis.

Base de datos: PostgreSQL frente a MySQL o SQL Server. MySQL también es libre y muy conocido, y SQL Server es el de Microsoft, de pago a partir de cierto tamaño. Pero la documentación de Odoo indica que solo funciona con PostgreSQL, así que elegir otra base de datos obligaría a cambiar de ERP. Además PostgreSQL es libre y es la que recomienda el profesor.

Embebido: Arduino Leonardo frente a encoder USB comercial o Raspberry Pi. Un encoder USB comercial (la placa que traen los kits de mando arcade) es aún más barato y se enchufa sin programar, pero sus teclas vienen fijas y no aprendemos nada de sistemas embebidos. Una Raspberry Pi es un ordenador entero con Linux: cuesta más, tarda en arrancar y para leer cuatro botones y un joystick es demasiado. El Leonardo está en el punto medio: barato, se programa a nuestro gusto y el ordenador lo reconoce como teclado. Un Arduino Uno no serviría, porque no tiene USB nativo.

Fuentes consultadas

- AWS Academy Learner Lab: https://aws.amazon.com/training/awsacademy/
- Amazon EC2 y Amazon S3: https://aws.amazon.com/ec2/ · https://aws.amazon.com/s3/
- Microsoft Azure: https://azure.microsoft.com/es-es/ · Google Cloud: https://cloud.google.com/
- Terraform (proveedor de AWS): https://registry.terraform.io/providers/hashicorp/aws/latest/docs
- Odoo Community y TPV (Punto de venta): https://www.odoo.com/es_ES/app/point-of-sale
- Instalación de Odoo, requisito de PostgreSQL: https://www.odoo.com/documentation/19.0/administration/on_premise/source.html
- SAP Business One: https://www.sap.com/spain/products/erp/business-one.html
- Holded: https://www.holded.com/es
- PostgreSQL: https://www.postgresql.org/about/ · MySQL: https://www.mysql.com/ · SQL Server: https://www.microsoft.com/es-es/sql-server
- Arduino Leonardo y biblioteca Keyboard: https://docs.arduino.cc/hardware/leonardo/ · https://docs.arduino.cc/language-reference/en/functions/usb/Keyboard/
- Raspberry Pi: https://www.raspberrypi.com/products/
- Documentación del propio proyecto: `infra/terraform`, `erp`, `arcade-stick` y fichas HW/SW

5. Cómo cambia el restaurante

Para el cliente - **Menos espera y mejor servicio:** el camarero cobra en el TPV táctil y el ticket sale solo, así que se atiende con más agilidad. - **Sin sorpresas al pedir:** como el stock está actualizado al momento, el camarero sabe si un plato se ha acabado antes de que el cliente lo pida. - **Encuentra el restaurante en internet:** desde la web ve la carta, los platos y cómo llegar con el mapa, en el móvil o en el ordenador. - **Ocio mientras espera:** la máquina recreativa.

Para el negocio - **Menos errores** al cobrar y facturar, porque el cálculo lo hace el programa. - **Ahorro de tiempo:** nadie cuenta el inventario ni junta tickets a mano. - **Decisiones con datos:** qué plato funciona, cuándo pedir género o qué local va mejor. - **Publicidad más barata:** la web da a conocer el restaurante sin pagar anuncios. - **La información no se pierde:** está en el servidor, en dos discos y con copias.

6. Riesgos y cómo los cubrimos

Al digitalizar, el restaurante depende de la tecnología. Si algo falla, puede quedarse sin cobrar o perder información. Estos son los riesgos y lo que hemos montado contra cada uno:

Riesgo	Qué pasaría	Cómo lo cubrimos
Se rompe un disco del servidor	Se perderían los datos de Odoo	**RAID 1:** el otro disco tiene lo mismo y todo sigue funcionando
Se borra algo por error, o un virus cifra los datos	El RAID copia también el error a los dos discos	**Copias de seguridad** cifradas en otro disco, con restauración probada
Se rompe el servidor entero	Los TPV no pueden registrar ventas en Odoo hasta repararlo	Se restaura la última copia en otro equipo; el servidor es profesional y tiene **SAI** contra cortes de luz
Alguien entra sin permiso	Robo de datos de clientes o manipulación de cobros	Cuentas **sin permisos de administrador**, TPV en modo quiosco, cortafuegos, SSH solo con claves y Wi-Fi de clientes separado
Se cae el local o internet	—	La **web sigue funcionando** porque está en la nube
Contraseñas filtradas	Acceso a todo el sistema	Gestor de contraseñas **KeePassXC**; nunca se suben a GitHub

7. Conclusión

Al restaurante le conviene digitalizarse por tres motivos. Primero, **agiliza los procesos**: se cobra antes, se controla el stock al momento y eso **mejora el servicio** al cliente. Segundo, **está protegido ante imprevistos**: con el RAID, las copias de seguridad y el SAI, una avería o un corte de luz no le hacen perder la información. Y tercero, **no se queda anticuado**: con una web en la nube y un sistema de gestión moderno puede competir con otros restaurantes y llegar a clientes nuevos.

Todo esto con una inversión moderada y con software libre, sin pagar licencias.

13. Organización y seguimiento del equipo

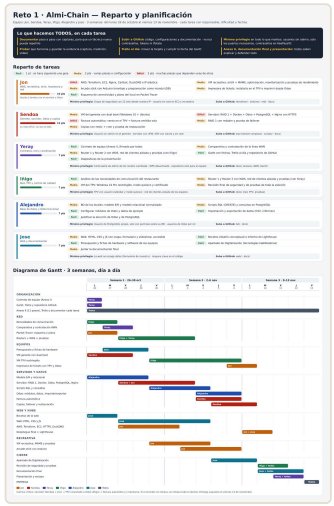
Responsable: Yeray

Dónde está en GitHub

Gestión, anexos y validaciones	`gestion/README.md`
Anexos I y II	`gestion/anexos`
Gantt y reparto	`docs/diagramas/reparto_y_gantt.png`

Resumen

- Contrato de equipo (Anexo I), documento de los 11 pasos (Anexo II), Gantt y Trello.
- Reparto de tareas: `diagramas/reparto_y_gantt.png`
- Normas del repositorio: `CONTRIBUTING.md`



Reparto y gantt · GitHub: docs/diagramas/reparto_y_gantt.png

Somos seis, con los roles del contrato de equipo (Anexo I), firmado por todos el 26/10: Jon coordina, Yeray es el secretario, Iñigo el portavoz, Jose lleva la documentación, y Sendoa y Alejandro son técnicos. Como coordinador, Jon tenía sus propias tareas (nube en AWS, recreativa, arcade stick e impresora) y, además, ayudaba a los demás cuando se atascaban y revisaba que lo subido a GitHub estuviera completo; cada parte es de su responsable, que es quien la ha hecho y quien la defiende. Cada tarea tiene un responsable, un apoyo y una dificultad en puntos, y se planificó en un Gantt de tres semanas (del 26/10 al 13/11). El día a día se siguió en Trello. Todo el trabajo está en GitHub: cada cambio entra con una rama y una pull request revisada, así que queda registrado quién hizo cada cosa y cuándo. Los profesores han validado las tareas según se

terminaban (ver la tabla de validaciones en `gestion/`). Cambios sobre el plan: repetidor en lugar de WDS, Windows 11 en el gerente y la factura automática, que pasó de Sendoa a Yeray e Iñigo. La metodología de los 11 pasos está en el Anexo II.

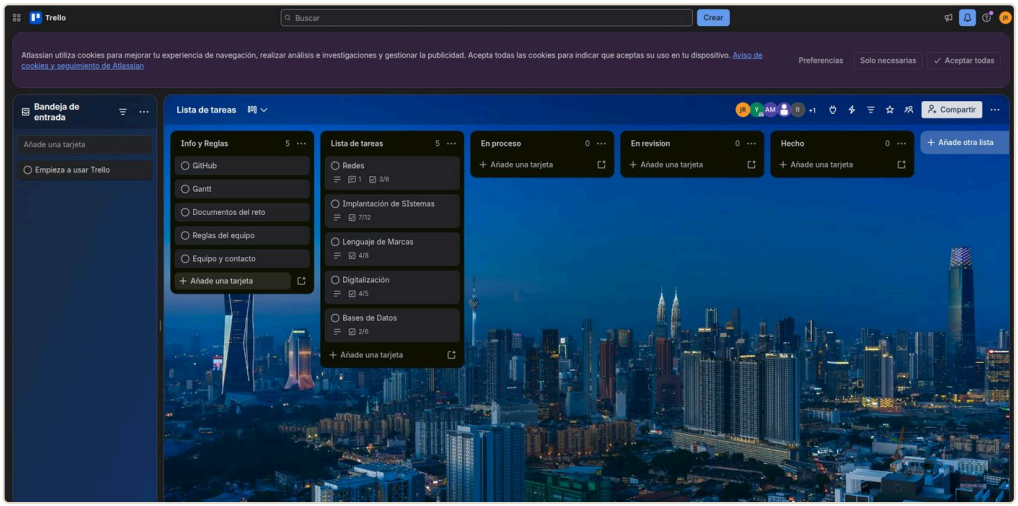
Gestión del proyecto

> **En GitHub:** `gestion/README.md`

Anexos I (contrato de equipo), II (11 pasos) y IX, y enlace al tablero Kanban/Gantt.

Anexos

Anexo	Documento	Estado
I · Contrato de equipo	`anexos/Anexo_I_Contrato_de_Equipo.pdf`: roles, reparto de tareas (responsable y apoyo), normas, consecuencias y código de conducta	✅ 26/10, firmado por los seis
II · Documento de 11 pasos	`anexos/Anexo_II_11_pasos.xlsx`: la plantilla de los 11 pasos del reto introductorio rellenada con este reto (activación del equipo, plantear, identificar, parámetros, información, alternativas, selección, planificación, ejecución, presentación y reflexión)	✅ 12/11
IX · Seguimiento (Gantt/Kanban) y control de versiones	Gantt planificado: `../docs/diagramas/reparto_y_gantt.png` · tablero de Trello: Lista de tareas · control de versiones: este repositorio (ramas y PR, ver `CONTRIBUTING.md`)	✅ 27/10 · captura del tablero



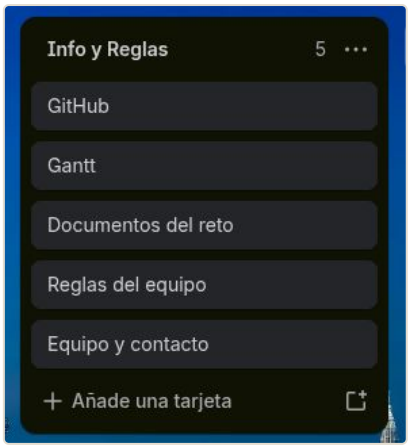
captura del tablero · GitHub: gestion/capturas/trello_tablero.png

Capturas del tablero de Trello

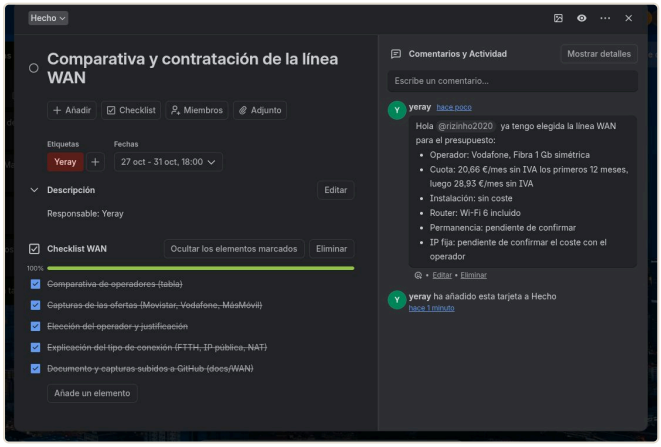
Captura	Qué muestra
trello_tablero · 02_trello_tablero	El tablero con sus listas
03_trello_info_reglas	Tarjetas de información y reglas del equipo
04_trello_tarjeta_wan	Una tarjeta por dentro: la de la línea WAN
05_trello_etiquetas	Etiquetas usadas



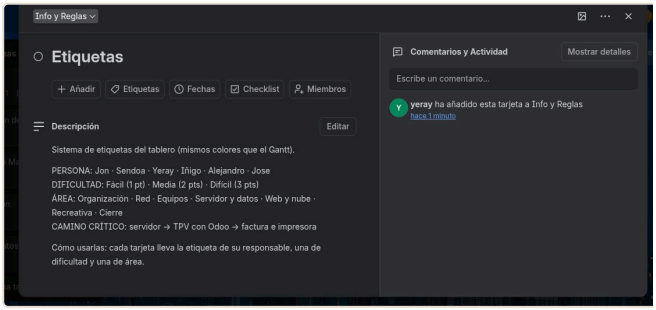
El tablero con sus listas · GitHub: gestion/capturas/02_trello_tablero.png



Tarjetas de información y reglas del equipo · GitHub: gestion/capturas/03_trello_info_reglas.png



Una tarjeta por dentro: la de la línea WAN · GitHub: gestion/capturas/04_trello_tarjeta_wan.png



Etiquetas usadas · GitHub: gestion/capturas/05_trello_etiquetas.png

Validaciones del profesor

Lo que el profesor (o la profesora de Redes) ha revisado y dado por bueno, con la fecha.

Fecha	Criterio o parte	Resultado
12/11	**Ampliación de la cobertura Wi-Fi**: router y repetidor con la red de clientes aislada del servidor · profesora de Redes (routers)	✅ **Validado**
12/11	**Diseño y esquema de conectividad**: versión final del Packet Tracer · profesora de Redes (`almichain.pkt`)	✅ **Validado**
10/11	**Instalación de SO y arranque dual** (Windows 11 + Ubuntu, cifrados) · profesor de Sistemas	✅ **Validado**
10/11	**Restricción de cuentas y equipos** (mínimo privilegio en el gerente, el TPV y la recreativa) · profesor de Sistemas	✅ **Validado**
10/11	**Optimización y monitorización de la máquina recreativa** · profesor de Sistemas	✅ **Validado**
10/11	**Plan de copias de seguridad** (local y en AWS S3, con restauración en directo) · profesor de Sistemas	✅ **Validado**

Fecha	Criterio o parte	Resultado
10/11	**Disco redundante (RAID 1)** · profesor de Sistemas	✅ **Validado**
10/11	**Configuración de la impresora** · profesor de Sistemas	✅ **Validado**
10/11	**Diseño web**	✅ **5**
10/11	**Implementación web** (HTTPS, dominio y AWS con Terraform)	✅ **5**
10/11	**ERP (Odoo)**	✅ **5**
10/11	**Copias de la BD**, con copia remota a S3 y restauración desde la nube hechas en directo delante del profesor (prueba 18)	✅ **5**
10/11	**Digitalización: tecnologías habilitadoras**	✅ **5**
10/11	**Digitalización: arcade stick**	✅ **5**
29/10	Medidas de rendimiento de la recreativa (resultados)	✅ Suficientes como constancia
27/10	Venta en el TPV → ticket impreso solo, como impresión y factura automáticas (TPV, prueba 8)	✅ Válido
26/10	Packet Tracer: esquema, plano y configuración (profesora de Redes) (prueba 9)	✅ Correcto
26/10	Windows 11 en lugar de Windows 10 en el equipo del gerente (dual boot)	✅ Válido
—	Repetidor (*Range Extender*) en lugar de WDS, porque el firmware del TL-WR841N no tiene WDS (repetidor)	✅ Indicado por el profesor
—	Montaje y funcionamiento del arcade stick (prueba 1)	✅ Funciona

Planificación

El Gantt es la **planificación inicial** (26/10 – 13/11). Cambios respecto al plan: la factura automática la llevan Yeray e Iñigo (en el plan, Sendoa), y la fecha de entrega del contrato es el 09/11.

Roles

Persona	Rol	Área técnica
Jon Revillas	Coordinador	Nube (AWS), recreativa, Arduino e impresora
Yeray Arratibel	Secretario	Gantt, Trello y línea WAN
Jose Rizo	Documentación	Web y presupuesto
Iñigo Basterrechea	Portavoz	Redes y TPV
Sendoa Guerrero	Técnico	Servidor, RAID y copias
Alex Marquez	Técnico	Base de datos y ERP

Como coordinador, Jon tenía sus propias tareas (las de la tabla) y, además, se encargaba de que el trabajo avanzara: resolver dudas, echar una mano a los compañeros cuando se atascaban y revisar que lo que se subía a GitHub estuviera completo. Cada parte es de su responsable, que es quien la ha hecho y quien la

defiende.

Cómo se lleva el seguimiento

1. **Planificar:** el Gantt reparte las tareas por persona y fecha, y el contrato (Anexo I) fija el responsable y el apoyo de cada una.
2. **Seguir el día a día:** cada tarea es una tarjeta de Trello que pasa de lista en lista hasta «Hecho».
3. **Entregar el trabajo:** cada cambio va en una rama de GitHub y entra en `main` con una Pull Request revisada (`CONTRIBUTING.md`). El historial deja constancia de quién hizo cada cosa y cuándo.
4. **Documentar:** cada parte tiene su guía con pruebas y capturas, y el documento final las resume.

14. Revisión de seguridad y pruebas

Responsable: Iñigo y todos

Resumen de las pruebas de cada parte. El detalle, los comandos y las capturas están en la guía enlazada en cada fila.

14.1 Pruebas de funcionamiento

Parte	Qué se prueba	Resultado esperado	Resultado	Evidencia
Gerente	Arranque dual con GRUB, BitLocker y LUKS	Se elige el sistema y el disco pide su clave antes del inicio de sesión	✅	dual boot §9
Gerente	Servicios y puertos hacia la red	Ningún puerto abierto hacia la red y cortafuegos activo, también tras reiniciar	✅ 04/11 · de 1 puerto a 0	dual boot §11.1
TPV	IP fija, quiosco de Edge y Odoo por HTTPS sin avisos	Edge abre solo el punto de venta y el certificado es de confianza	✅	TPV, pruebas 1–3
TPV	Salir del quiosco con atajos de teclado	No se puede salir	✅ 06/11 · ninguna de las 7 combinaciones saca del quiosco	TPV, prueba 4
Impresora	Venta en el TPV → ticket impreso solo, y legible	Se imprime al validar el pago; se leen €, ¿? y el QR	✅ 27/10 · validado por el profesor	TPV, pruebas 5–8
Recreativa	Rendimiento con 1 núcleo y 1 GB	Juego fluido sin swap	✅ CPU 20,3 %, MAME 364 MB, `mame -bench` 1353–1613 % · validado por el profesor	medidas
Arcade stick	Cada control, y en MAME, Chrome y RetroArch sin drivers	Cada entrada marca su tecla y los juegos responden	✅ validado por el profesor	arcade stick §7
Red (Packet Tracer)	Aislamiento: el móvil de clientes hace ping al servidor	Falla	✅ 05/11 · 100 % de pérdida · proyecto validado por la profesora el 26/10	redes §7, pruebas 3 y 9

Parte	Qué se prueba	Resultado esperado	Resultado	Evidencia
Red (routers del aula)	Cliente de la Wi-Fi: internet y aislamiento del servidor	Navega y no llega al servidor	✅ 12/11 · internet responde y el servidor no (100 % de pérdida). Validado por la profesora	routers §6
Web	Responsiva (1440, 768 y 390 px), menú, slideshow, formulario y geolocalización	Funciona en las tres anchuras	✅	web §6
Web	Validador W3C y Lighthouse de la web publicada	0 errores y ≥ 90	✅ 0 errores · 99–100	web §6
AWS	HTTPS, cabeceras de seguridad y reproducibilidad (`destroy` + `apply`)	Candado válido y la web vuelve igual	✅ vuelve en 1 min 45 s	AWS §6
Servidor	Contenedores, puertos, UFW, `nmap`, HTTPS y gestor de BD desactivado	Solo 22, 80 y 443 hacia la LAN; PostgreSQL y Odoo sin publicar	✅ 26/10	ERP, pruebas 1–6
ERP	Venta en el TPV → factura automática	El ticket sale solo con logo e IVA 10 %	✅ validado por el profesor	ERP, prueba 9
ERP	Importar CSV y exportar a XLSX; CRM, Marketing y usuarios por rol	Sin errores	✅ 06/11	ERP, prueba 10 y configuración
Base de datos	Scripts 00–05 y las 11 consultas	Sin errores y los resultados esperados	✅ 17 tablas, 32 líneas de venta	BD, consultas
RAID 1	Fallo de un disco	Odoo sigue funcionando y el RAID se reconstruye	✅ 26/10 · en menos de 2 min	copias, pruebas 11–12
Copias	Copia automática con cron y restauración	«Copia terminada» y `RESTAURACIÓN CORRECTA`	✅ 27/10 y 03/11	copias, pruebas 13–14
Copia en S3	Copia remota, restauración desde S3 y fallo sin credenciales	Restaura desde la nube y avisa si el lab está apagado	✅ 29/10	copias, pruebas 16–19

14.2 Mínimo privilegio en cada parte

Parte	Cuenta de uso diario	Quién administra	Comprobado
Gerente	`gerente`: estándar en Windows y sin `sudo` en Ubuntu	`tecnico`	UAC pide las credenciales del técnico
TPV	`tpv`: estándar y encerrado en el quiosco	Administrador local	El cajero de Odoo no puede crear productos
Recreativa	`recreativa`: sin `sudo`	`adminrec`	SSH rechaza a `recreativa` y a root
Servidor	Servicios en contenedores	`adminsrv` con clave SSH, fuera del grupo `docker`	Root y contraseña rechazados por SSH
Odoo	Barra 1 y Barra 2: solo punto de venta	Gerente y administrador	Permisos en la ficha de cada usuario

Parte	Cuenta de uso diario	Quién administra	Comprobado
PostgreSQL	`almichain_app` : solo leer y escribir datos	`almichain_owner`	`DROP TABLE` y `CREATE TABLE` fallan con `almichain_app`
Secretos	Ninguna contraseña ni clave en el repositorio	KeePassXC y `/root/secretos` (solo root)	Revisado en cada guía

15. Conclusiones

Qué ha funcionado. Hemos montado entera la informática de un restaurante: equipos, red, web en la nube, ERP con TPV, base de datos y copias de seguridad. Cada parte tiene su guía en el repositorio y se puede volver a montar desde cero. Todo el software es libre o gratuito, así que el restaurante solo paga el hardware, la fibra y el servidor de AWS. Además, cada parte se ha probado y las evidencias están en las guías. Por ejemplo, la web da 0 errores en el W3C y entre 99 y 100 en Lighthouse, y el servidor sigue funcionando al quitarle un disco del RAID.

Problemas y cómo se resolvieron. Los fallos que fueron saliendo se apuntaron en cada guía, con la causa y la solución. Por ejemplo, el servidor de Odoo aceptaba contraseñas por SSH porque el script de seguridad se ejecutó antes de copiar la clave, y se corrigió dejando solo el acceso con clave. Las copias automáticas no se hacían porque la máquina virtual del servidor está apagada por la noche, a la hora programada, y se resolvió haciendo también una copia unos minutos después de cada arranque. También hubo que adaptarse a cambios de planteamiento, como pasar de Windows 10 a Windows 11 en el equipo del gerente al quedarse Windows 10 sin soporte. Trabajar con GitHub y con ramas y Pull Requests nos ayudó a no pisarnos el trabajo entre seis personas.

Qué mejoraríamos. En un restaurante real, el servidor estaría encendido siempre, con su SAI, para que las copias se hagan a su hora. También tendríamos un dominio propio en lugar de DuckDNS y una cuenta normal de AWS en lugar del Learner Lab, que se apaga al cerrar la sesión. Y la web, que ahora solo informa, podría tener reservas de verdad conectadas con Odoo.

16. Anexos

- Anexo I: contrato de equipo, firmado por los seis → `gestion/anexos/Anexo_I_Contrato_de_Equipo.pdf`
- Anexo II: documento de los 11 pasos → `gestion/anexos/Anexo_II_11_pasos.xlsx`
- Reparto y Gantt → `diagramas/reparto_y_gantt.png`